

Data Scout - the standalone lab, explained

This document contains the complete, annotated source of the bundle: the web server `api_server.py` and the whole scouting engine `scout_engine.py`, every block shown as real code with a plain-English note on what it does and how it works. It follows the same order as the files.

How the pieces fit together

```
Browser (the lab UI)                                Python (this bundle)
-----
index.html -> loads  app.css + app.js  (the real React lab, pre-built)
|
|  fetch("/api/data-scout", {command: "..."})
v
+-----+
| api_server.py  (FastAPI web server)                |
| * serves the UI files                             |
| * receives each {command}, calls scout_engine.COMMANDS[.] |
| * overrides the engine's get_connection() so all its SQL |
| runs against an in-memory SQLite DB, not PostgreSQL    |
+-----+
|
| scout_engine.py (the course engine, verbatim)
v
in-memory SQLite <- loaded from - data/players.csv.gz
                                data/supplementary.csv.gz
```

The trick that makes the numbers match: `scout_engine.py` is the production engine, unchanged. All of its data access goes through one function, `get_connection()`. `api_server.py` replaces that function with one that talks to a local SQLite database loaded from the CSVs. Nothing in the scoring maths changes, so every result is the same as the live course.

Part 1 - `api_server.py` (complete, annotated)

Data Scout - Day 5 capstone, standalone. Runs the EXACT same engine as the live course lab, but over a local CSV snapshot instead of a database - so anyone can download this folder and reproduce the Day 5 interactive lab (profile + radar, similar players, hidden gems, market value) with identical numbers. `pip install -r requirements.txt python api_server.py` # then open `http://localhost:8000` The engine (`scout_engine.py`) is the course engine with its DB credentials removed. All of its database access funnels through `get_connection()`; here we override that to point at an in-memory SQLite database loaded from the CSVs in `./data`, so every command runs unchanged. Provide your own snapshot any time via POST `/upload`.

Imports.

```
import io
import json
import os
import sqlite3
import threading
import pandas as pd
from fastapi import FastAPI, File, Request, Response, UploadFile
```

```
from fastapi.responses import FileResponse, HTMLResponse
import scout_engine as eng
```

``HERE`` (module constant)

```
HERE = os.path.dirname(os.path.abspath(__file__))
```

``DATA_DIR`` (module constant)

```
DATA_DIR = os.path.join(HERE, "data")
```

``_LOCK`` (module constant)

```
_LOCK = threading.Lock()
```

``_db`` (module constant)

```
_db = sqlite3.connect(":memory:", check_same_thread=False)
```

psycopg2-style shim over SQLite: translate %s placeholders to ?

``_Cur`` (class) - A cursor wrapper: rewrites Postgres-style %s placeholders to SQLite ? so the engine's SQL runs unchanged, and exposes fetchall/fetchone/description like psycopg2.

```
class _Cur:
    def __init__(self, cur):
        self._cur = cur

    def execute(self, sql, params=None):
        sql = sql.replace("%s", "?")
        self._cur.execute(sql, list(params) if params else [])
        return self

    def fetchall(self):
        return self._cur.fetchall()

    def fetchone(self):
        return self._cur.fetchone()

    @property
    def description(self):
        return self._cur.description

    def close(self):
        self._cur.close()
```

``_Conn`` (class) - A connection wrapper that behaves like a psycopg2 connection (cursor/commit/rollback/close and with ... support) but is backed by the shared in-memory SQLite database. close() is a no-op so the in-memory data survives between requests.

```
class _Conn:
    def cursor(self):
        return _Cur(_db.cursor())

    def execute(self, sql, params=None):
        return self.cursor().execute(sql, params)

    def commit(self):
        pass
```

```

def rollback(self):
    pass

def close(self):
    # keep the shared in-memory db alive across requests
    pass

def __enter__(self):
    return self

def __exit__(self, *a):
    pass

```

Module code.

```
eng.get_connection = lambda: _Conn()
```

`\_clear\_caches` (def) - Empties the engine's module-level lookup caches whenever the data changes, so stale results can't leak after a reload.

```

def _clear_caches():
    for name in ("_ALL_LEAGUES_CACHE", "_MS_POOL_CACHE"):
        c = getattr(eng, name, None)
        if isinstance(c, dict):
            c.clear()

```

`load\_players\_frame` (def) - Writes the players DataFrame into the SQLite table (replacing what was there) and rebuilds the season / (season,league) / player indexes the engine's queries rely on. Then clears the caches.

```

def load_players_frame(players: pd.DataFrame):
    """Replace just the players table + its indexes, and refresh the engine caches."""
    with _LOCK:
        players.to_sql(
            "league_season_team_player_data", _db, if_exists="replace", index=False
        )
        cur = _db.cursor()
        cur.execute(
            "CREATE INDEX IF NOT EXISTS ix_season "
            "ON league_season_team_player_data(season)"
        )
        cur.execute(
            "CREATE INDEX IF NOT EXISTS ix_season_league "
            "ON league_season_team_player_data(season, league)"
        )
        cur.execute(
            "CREATE INDEX IF NOT EXISTS ix_player "
            "ON league_season_team_player_data(player)"
        )
        _db.commit()
        _clear_caches()

```

`load\_supp\_frame` (def) - Same, for the supplementary table (wages / values / contracts) and its player index. Independent of the players load, so either file can be reloaded on its own.

```

def load_supp_frame(supplementary: pd.DataFrame):
    """Replace just the supplementary table + its index, and refresh the caches."""
    with _LOCK:
        supplementary.to_sql(

```

```

        "player_supplementary_data", _db, if_exists="replace", index=False
    )
    cur = _db.cursor()
    cur.execute(
        "CREATE INDEX IF NOT EXISTS ix_supp_player "
        "ON player_supplementary_data(player)"
    )
    _db.commit()
    _clear_caches()

```

`load\_frames` (def) - Convenience wrapper that loads both tables.

```

def load_frames(players: pd.DataFrame, supplementary: pd.DataFrame):
    """Rebuild both tables from two DataFrames."""
    load_players_frame(players)
    load_supp_frame(supplementary)

```

`\_read\_csv` (def) - Reads an uploaded file into a DataFrame, handling both plain .csv and gzipped .csv.gz.

```

def _read_csv(raw: bytes, name: str) -> pd.DataFrame:
    gz = name.endswith(".gz")
    return pd.read_csv(
        io.BytesIO(raw), compression="gzip" if gz else None, low_memory=False
    )

```

`\_boot` (def) - Runs once at startup: if the data/ files are present it loads them; otherwise it prints a note to upload a snapshot.

```

def _boot():
    players = os.path.join(DATA_DIR, "players.csv.gz")
    supp = os.path.join(DATA_DIR, "supplementary.csv.gz")
    if os.path.exists(players) and os.path.exists(supp):
        p = pd.read_csv(players, compression="gzip", low_memory=False)
        s = pd.read_csv(supp, compression="gzip", low_memory=False)
        load_frames(p, s)
        print(f"Loaded {len(p):,} player-season rows + {len(s):,} supplementary rows.")
    else:
        print(
            "No data/players.csv.gz + data/supplementary.csv.gz found - "
            "upload a snapshot via POST /upload (multipart: players, supplementary)."
        )

```

```

_boot()

```

`app` (module constant)

```

app = FastAPI(title="Data Scout - Day 5 capstone (standalone)")

```

`\_json` (def) - Serialises a result to a JSON response. default=str (the same tolerance the course server uses) lets numpy scalars serialise without crashing.

```

def _json(result) -> Response:
    # Same serialisation as the course server: default=str tolerates numpy scalars.
    return Response(json.dumps(result, default=str), media_type="application/json")

```

`data\_scout` (def) - The main endpoint. The browser POSTs {command, ...}; this looks the command up in the engine's COMMANDS table, runs it inside the lock (one engine call at a time), and returns JSON. A failing command returns {error: ...} instead of crashing the server.

```

@app.post("/api/data-scout")
async def data_scout(request: Request):
    body = await request.json()
    fn = eng.COMMANDS.get(body.get("command", ""))
    if fn is None:
        return _json({"error": f"unknown command: {body.get('command')}"})
    with _LOCK:
        try:
            return _json(fn(body))
        except Exception as e: # mirror the course server's per-command guard
            return _json({"error": str(e)})

```

`upload` (def) - Accepts either file on its own or both (each is optional) and loads whichever is provided, replacing just that table.

```

@app.post("/upload")
async def upload(
    players: UploadFile = File(None), supplementary: UploadFile = File(None)
):
    """Upload either file on its own, or both. Each replaces just its own table."""
    out = {"ok": True}
    if players is not None:
        p = _read_csv(await players.read(), players.filename)
        load_players_frame(p)
        out["player_rows"] = len(p)
    if supplementary is not None:
        s = _read_csv(await supplementary.read(), supplementary.filename)
        load_supp_frame(s)
        out["supplementary_rows"] = len(s)
    if "player_rows" not in out and "supplementary_rows" not in out:
        return _json({"ok": False, "error": "no file provided"})
    return out

```

`app\_js` (def) - Serves the pre-built lab bundle (JavaScript).

```

@app.get("/app.js")
def app_js():
    return FileResponse(os.path.join(HERE, "app.js"), media_type="text/javascript")

```

`app\_css` (def) - Serves the pre-built lab bundle (CSS).

```

@app.get("/app.css")
def app_css():
    return FileResponse(os.path.join(HERE, "app.css"), media_type="text/css")

```

`shortlist\_get` (def) - No-op: the similarity lab stores its shortlist in the browser's localStorage; returning {} (no shortlist key) leaves that copy untouched.

```

@app.get("/api/data-scout/shortlist")
def shortlist_get():
    return {}

```

`shortlist\_post` (def) - No-op accept of a shortlist sync.

```

@app.post("/api/data-scout/shortlist")
async def shortlist_post(request: Request):
    try:
        await request.json()
    except Exception:
        pass
    return {"ok": True}

```

`status` (def) - Reports how many rows are loaded, so the UI panel can show 'loaded - 170,710 player rows'.

```
@app.get("/status")
def status():
    def _count(tbl):
        try:
            with _LOCK:
                return _db.execute(f"SELECT COUNT(*) FROM {tbl}").fetchone()[0]
        except Exception:
            return 0
    return {
        "player_rows": _count("league_season_team_player_data"),
        "supplementary_rows": _count("player_supplementary_data"),
    }
```

`index` (def) - Serves the HTML shell.

```
@app.get("/")
def index():
    idx = os.path.join(HERE, "index.html")
    if os.path.exists(idx):
        return FileResponse(idx)
    return HTMLResponse("<h1>Data Scout API</h1><p>POST /api/data-scout</p>")
```

```
if __name__ == "__main__":
    import uvicorn

    print("\n" + "=" * 60)
    print(" Data Scout is running.")
    print(" -> Open http://localhost:8000 in your browser")
    print("=" * 60 + "\n")
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Part 2 - scout_engine.py (complete, annotated)

The engine is the production code, verbatim, with its database credentials removed. Normally it runs as a stdin/stdout server (main() reads one JSON command per line); in this bundle api_server.py calls its functions directly through the COMMANDS table. The lab uses about a dozen commands; all of them, and every helper they call, are below in source order.

Data Scout analysis server for football player scouting. Communicates via stdin/stdout JSON protocol (same pattern as bop/evolution/halftime). Wraps functions from the legacy tadv15.py Streamlit app.

Imports.

```
import sys
import os
import json
import time
import unicodedata
import numpy as np
import pandas as pd
import psycopg2
from scipy import stats
from sklearn.metrics.pairwise import cosine_similarity, euclidean_distances, manhattan_distances
```

```

from sklearn.preprocessing import StandardScaler
from sklearn.covariance import MinCovDet
from scipy.spatial.distance import mahalanobis as mahalanobis_dist
from datetime import datetime
import warnings

```

```
warnings.filterwarnings('ignore')
```

Configuration

`DB\_CONFIG` (module constant)

```
DB_CONFIG = {"dbname": "", "user": "", "password": "", "host": "", "port": ""}
```

`LEAGUE\_MAP` (module constant)

```

LEAGUE_MAP = {
    'premier-league': {'fbref_id': '9', 'fbref_name': 'Premier-League',
    'understat': 'epl', 'capology': 'uk/premier-league', 'tm': 'GB1'},
    'la-liga': {'fbref_id': '12', 'fbref_name': 'La-Liga',
    'understat': 'la_liga', 'capology': 'es/la-liga', 'tm': 'ES1'},
    'serie-a': {'fbref_id': '11', 'fbref_name': 'Serie-A',
    'understat': 'serie_a', 'capology': 'it/serie-a', 'tm': 'IT1'},
    'bundesliga': {'fbref_id': '20', 'fbref_name': 'Bundesliga',
    'understat': 'bundesliga', 'capology': 'de/1-bundesliga', 'tm': 'L1'},
    'ligue-1': {'fbref_id': '13', 'fbref_name': 'Ligue-1',
    'understat': 'ligue_1', 'capology': 'fr/ligue-1', 'tm': 'FR1'},
    'eredivisie': {'fbref_id': '23', 'fbref_name': 'Eredivisie',
    'understat': None, 'capology': 'nl/eredivisie', 'tm': 'NL1'},
    'efl-championship': {'fbref_id': '10', 'fbref_name': 'Championship',
    'understat': None, 'capology': 'uk/championship', 'tm': 'GB2'},
    'primeira-liga': {'fbref_id': '32', 'fbref_name': 'Primeira-Liga',
    'understat': None, 'capology': 'pt/primeira-liga', 'tm': 'P01'},
    'belgian-pro-league': {'fbref_id': '37', 'fbref_name': 'Belgian-Pro-League',
    'understat': None, 'capology': 'be/first-division-a', 'tm': 'BE1'},
    'serie-b': {'fbref_id': '18', 'fbref_name': 'Serie-B',
    'understat': None, 'capology': 'it/serie-b', 'tm': 'IT2'},
    'major-league': {'fbref_id': '22', 'fbref_name': 'Major-League-Soccer',
    'understat': None, 'capology': 'us/mls', 'tm': 'MLS1'},
    'campeonato-brasileiro-serie-a': {'fbref_id': '24', 'fbref_name': 'Serie-A',
    'understat': None, 'capology': None, 'tm': 'BRA1'},
    'liga-profesional-argentina': {'fbref_id': '21', 'fbref_name': 'Primera-Division',
    'understat': None, 'capology': None, 'tm': 'AR1N'},
    'liga-mx': {'fbref_id': '31', 'fbref_name': 'Liga-MX',
    'understat': None, 'capology': None, 'tm': 'MEX1'},
}

```

Utility Functions

`get\_connection` (def) - The single choke-point for all database access. In this bundle `api_server.py` replaces it with one that returns a SQLite-backed connection, which is why the engine runs unchanged over the local CSV snapshot.

```

def get_connection():
    return psycopg2.connect(**DB_CONFIG)

```

`standardize\_positions` (def) - Maps every raw position string to one of four buckets (GK, DF, MF, FW) in a `primary_position` column. Every comparison is within-position, so this must

be consistent.

```
def standardize_positions(df):
    """Standardize position names and handle multiple positions.
    Ported from legacy tadv15.py lines 360-432.
    Maps various position abbreviations to standard 4 categories: GK, DF, MF, FW.
    Adds primary_position and secondary_position columns.
    """
    position_map = {
        'GK': 'GK', 'GOALKEEPER': 'GK',
        'DF': 'DF', 'DEFENDER': 'DF', 'CB': 'DF', 'LB': 'DF', 'RB': 'DF', 'LWB': 'DF', 'RWB': 'DF',
        'MF': 'MF', 'MIDFIELDER': 'MF', 'CM': 'MF', 'CDM': 'MF', 'CAM': 'MF', 'LM': 'MF', 'RM': 'MF',
        'FW': 'FW', 'ST': 'FW', 'ATTACKER': 'FW', 'STRIKER': 'FW', 'CF': 'FW', 'LW': 'FW', 'RW': 'FW'
    }

    if 'position' not in df.columns:
        for col in ['pos', 'position_played', 'player_position']:
            if col in df.columns:
                df['position'] = df[col]
                break
        else:
            df['position'] = 'UNKNOWN'

    df['position'] = df['position'].astype(str).str.upper().str.strip()

    df['primary_position'] = 'UNKNOWN'
    df['secondary_position'] = None

    for idx, row in df.iterrows():
        pos_str = str(row['position'])

        # Split by common separators
        positions = []
        for sep in [',', '/', '-', ' ']:
            if sep in pos_str:
                positions = [p.strip() for p in pos_str.split(sep)]
                break

        if not positions:
            positions = [pos_str]

        # Map each position and filter valid ones
        mapped_positions = []
        for pos in positions:
            mapped = position_map.get(pos, None)
            if mapped and mapped not in mapped_positions:
                mapped_positions.append(mapped)

        # Assign primary and secondary positions
        if mapped_positions:
            df.at[idx, 'primary_position'] = mapped_positions[0]
            if len(mapped_positions) > 1:
                df.at[idx, 'secondary_position'] = mapped_positions[1]
        else:
            # If no valid mapping found, try to infer from the original string
            if any(gk in pos_str for gk in ['GK', 'GOAL']):
                df.at[idx, 'primary_position'] = 'GK'
            elif any(df_pos in pos_str for df_pos in ['DF', 'DEF', 'CB', 'LB', 'RB', 'WB']):
                df.at[idx, 'primary_position'] = 'DF'
            elif any(mf in pos_str for mf in ['MF', 'MID', 'CM', 'DM', 'AM']):
                df.at[idx, 'primary_position'] = 'MF'
            elif any(fw in pos_str for fw in ['FW', 'ST', 'CF', 'ATTACK', 'STRIKER']):
                df.at[idx, 'primary_position'] = 'FW'
```



```
return df
```

`safe\_float` (def) - Parse a value to float, returning 0.0 on anything unparseable.

```
def safe_float(value):
    if isinstance(value, pd.Series):
        return value.apply(lambda x: safe_float(x))
    if pd.isna(value) or value is None:
        return 0.0
    s = str(value).strip()
    if s in ('', 'nan', 'None', 'null', 'NaN'):
        return 0.0
    try:
        return float(s.replace(',', ''))
    except (ValueError, TypeError):
        return 0.0
```

`parse\_age` (def) - FBref stores age as 'YY-DDD' (years-days); this reads the year part. A plain float parse would misread it.

```
def parse_age(value):
    if pd.isna(value) or value is None:
        return 0.0
    s = str(value).strip()
    if '-' in s:
        parts = s.split('-')
        if len(parts) == 2:
            try:
                years = int(parts[0])
                days = int(parts[1])
                if 10 <= years <= 60 and 0 <= days <= 366:
                    return float(years)
            except (ValueError, TypeError):
                pass
    try:
        return float(s)
    except (ValueError, TypeError):
        return 0.0
```

`r` (def)

```
def r(val, decimals=2):
    if val is None or (isinstance(val, float) and np.isnan(val)):
        return None
    return float(np.round(val, decimals))
```

`\_num\_series` (def) - Vectorised numeric parse tolerant of comma thousand-separators. The minutes column is stored as text like '1,409', which plain `pd.to_numeric` reads as NaN - silently dropping ~40% of players (every regular starter with ≥ 1000 minutes) from minutes filters.

```
def _num_series(col):
    """Vectorised numeric parse tolerant of comma thousand-separators.
    The `minutes` column is stored as text like '1,409', which plain
    pd.to_numeric reads as NaN - silently dropping ~40% of players
    (every regular starter with  $\geq 1000$  minutes) from minutes filters."""
    return pd.to_numeric(col.astype(str).str.replace(',', '', regex=False), errors='coerce')
```

`format\_eur` (def)

```
def format_eur(value):
    if value is None or value <= 0:
        return "N/A"
    if value >= 1_000_000:
        return f"€{value/1_000_000:.1f}M"
    if value >= 1_000:
        return f"€{value/1_000:.0f}K"
    return f"€{value:.0f}"
```

`get\_negative\_metrics` (def) - Lists the metrics where lower is better (goals conceded, errors, cards). These are inverted before scoring so that everywhere higher = better.

```
def get_negative_metrics():
    return [
        'cards_yellow', 'cards_red', 'cards_yellow_red', 'second_yellow', 'straight_red',
        'errors', 'errors_per90', 'own_goals', 'own_goals_per90', 'offsides', 'offsides_per90',
        'fouls', 'fouls_per90', 'miscontrols', 'miscontrols_per90', 'dispossessed',
        'dispossessed_per90',
        'dribbled_past', 'dribbled_past_per90', 'goals_against', 'goals_against_per90',
        'gk_goals_against_per90', 'gk_goals_against', 'goals_against_gk',
        'gk_shots_on_target_against', 'gk_losses', 'gk_own_goals_against',
        'gk_pens_allowed', 'gk_free_kick_goals_against', 'gk_corner_kick_goals_against',
        'on_goals_against', 'on_xg_against',
        'shots_against', 'shots_on_target_against', 'on_goal_against', 'on_goal_against_per90',
        'xg_against', 'xg_against_per90',
        'pk_allow', 'pens_allow', 'pens_missed', 'pens_saved_against', 'pens_conceded',
        'free_kicks_against', 'corners_against', 'crosses_into_box_against',
        'aerials_lost', 'aerials_lost_per90',
        'take_ons_tackled', 'take_ons_tackled_pct',
        'challenges_lost', 'challenges_lost_per90',
        # Shooting from closer is the better finishing signal, so lower avg shot
        # distance -> higher percentile in the Finishing/Shooting categories.
        'average_shot_distance',
    ]
```

`invert\_negative\_metrics` (def)

```
def invert_negative_metrics(value, metric_name):
    if metric_name in get_negative_metrics():
        if value <= 0:
            return 100
        return max(0, 100 / (1 + value))
    return value
```

Playing Style Categories (From Legacy Tadv15.Py Lines 3193-3418)

`RETIRED\_METRICS` (module constant)

```
RETIRED_METRICS = frozenset({
    # shot- & goal-creating actions
    'sca', 'sca_per90', 'sca_passes_live', 'sca_passes_dead', 'sca_take_ons',
    'sca_shots', 'sca_fouled', 'sca_defense',
    'gca', 'gca_per90', 'gca_passes_live', 'gca_passes_dead', 'gca_take_ons',
    'gca_shots', 'gca_fouled', 'gca_defense', 'shot_creating_actions_per90',
    # deep possession - progression
    'progressive_passes', 'progressive_carries', 'progressive_passes_received',
    'progressive_passes_per90', 'progressive_carries_per90',
    'passes_into_final_third', 'passes_into_penalty_area',
    'passes_into_final_third_per90', 'passes_progressive_distance',
    # deep possession - touches by zone
    'touches', 'touches_def_pen_area', 'touches_def_3rd', 'touches_mid_3rd',
```

```

'touches_att_3rd', 'touches_att_pen_area', 'touches_att_pen_area_per90',
'touches_live_ball',
# deep possession - carries
'carries', 'carries_distance', 'carries_progressive_distance',
'carries_into_final_third', 'carries_into_penalty_area',
# discipline counting stats FotMob has no full-season source for (fouls IS
# backfilled from FotMob and kept; these three are not available anywhere free)
'offsides', 'offsides_per90', 'errors', 'errors_per90',
'own_goals', 'own_goals_per90',
})

```

ORPHAN_METRICS (module constant)

```

ORPHAN_METRICS = frozenset({
'aerials_won', 'aerials_lost', 'aerials_won_pct', 'miscontrols', 'dispossessed',
'through_balls', 'tackles_won', 'tackles_def_3rd', 'tackles_mid_3rd', 'tackles_att_3rd',
'challenge_tackles', 'challenge_tackles_pct', 'challenges', 'challenges_lost',
'blocks', 'blocked_shots', 'blocked_passes', 'misc_tackles_won', 'misc_interceptions',
'misc_crosses',
'passes', 'passes_pct', 'passes_short', 'passes_medium', 'passes_long',
'passes_completed_short', 'passes_completed_medium', 'passes_completed_long',
'passes_pct_short', 'passes_pct_medium', 'passes_pct_long', 'passes_total_distance',
'passes_free_kicks', 'passes_switches', 'corner_kicks_in', 'corner_kicks_out',
'corner_kicks_straight', 'crosses_into_penalty_area', 'take_ons', 'take_ons_tackled',
'take_ons_tackled_pct', 'take_ons_won_pct', 'average_shot_distance', 'shots_free_kicks',
'shot_xg', 'shot_npxg', 'fouled', 'passes_received', 'pass_xa', 'pass_xg_assist',
'on_xg_against', 'xg_plus_minus', 'xg_plus_minus_per90',
})

```

ORPHAN_GK_METRICS (module constant)

```

ORPHAN_GK_METRICS = frozenset({
'gk_goal_kicks', 'gk_goal_kick_length_avg', 'gk_pct_goal_kicks_launched',
'gk_passes', 'gk_passes_throws', 'gk_passes_length_avg', 'gk_passes_launched',
'gk_passes_completed_launched', 'gk_pct_passes_launched', 'gk_crosses_stopped',
'gk_crosses_stopped_pct', 'gk_def_actions_outside_pen_area',
'gk_def_actions_outside_pen_area_per90', 'gk_avg_distance_def_actions',
'gk_shots_on_target_against', 'gk_psnpxg_per_shot_on_target_against',
'gk_wins', 'gk_losses', 'gk_ties', 'gk_pens_att', 'gk_pens_allowed',
'gk_pens_saved', 'gk_pens_missed', 'gk_pens_save_pct', 'gk_free_kick_goals_against',
'gk_corner_kick_goals_against', 'points_per_game', 'on_goals_for', 'on_goals_against',
})

```

EXCLUDED_METRICS (module constant)

```

EXCLUDED_METRICS = RETIRED_METRICS | ORPHAN_METRICS | ORPHAN_GK_METRICS

```

get_playing_style_categories (def)

```

def get_playing_style_categories():
    raw = {
        'GK': {
            'Shot Stopping & Saves': ['gk_saves', 'gk_save_pct', 'gk_shots_on_target_against',
'gk_goals_against', 'gk_goals_against_per90', 'gk_psnpxg_per_shot_on_target_against'],
            'Post-Shot xG & Advanced': ['gk_psxg', 'gk_psxg_net', 'gk_psxg_net_per90',
'gk_psnpxg_per_shot_on_target_against'],
            'Distribution & Passing': ['passes_completed', 'passes_pct', 'passes_completed_short',
'passes_pct_short', 'passes_completed_medium', 'passes_pct_medium', 'passes_completed_long',
'passes_pct_long', 'progressive_passes', 'pass_xa', 'gk_passes', 'gk_passes_throws',
'gk_passes_length_avg'],
            'Goal Kicks & Long Distribution': ['gk_goal_kicks', 'gk_goal_kick_length_avg',
'gk_pct_goal_kicks_launched', 'gk_passes_launched', 'gk_passes_completed_launched',
'gk_pct_passes_launched'],

```

```

        'Sweeping & Modern Play': ['gk_crosses_stopped', 'gk_crosses_stopped_pct',
'gk_def_actions_outside_pen_area', 'gk_def_actions_outside_pen_area_per90',
'gk_avg_distance_def_actions', 'touches', 'touches_def_pen_area', 'touches_def_3rd',
'touches_mid_3rd', 'touches_att_3rd', 'touches_att_pen_area', 'touches_live_ball'],
        'Penalties & Set Pieces': ['gk_pens_saved', 'gk_pens_allowed', 'gk_pens_save_pct',
'gk_pens_att', 'gk_pens_missed', 'pens_conceded', 'gk_free_kick_goals_against',
'gk_corner_kick_goals_against'],
        'Expected Goals (xG) Conceded': ['on_xg_against', 'gk_psxg_net', 'xg_plus_minus',
'xg_plus_minus_per90'],
        'Command & Presence': ['gk_clean_sheets', 'gk_clean_sheets_pct', 'gk_games',
'gk_games_starts', 'gk_minutes', 'gk_wins', 'gk_ties', 'gk_losses', 'minutes', 'minutes_per_game',
'points_per_game', 'on_goals_against', 'on_goals_for'],
    },
    'DF': {
        'Defensive Actions & Tackles': ['tackles', 'tackles_won', 'tackles_def_3rd',
'tackles_mid_3rd', 'tackles_att_3rd', 'challenge_tackles', 'challenge_tackles_pct', 'challenges',
'challenges_lost', 'tackles_interceptions', 'misc_tackles_won'],
        'Interceptions & Blocks': ['interceptions', 'blocks', 'blocked_shots', 'blocked_passes',
'clearances', 'misc_interceptions'],
        'Aerial Duels & Physical': ['aerials_won', 'aerials_lost', 'aerials_won_pct', 'fouled'],
        'Ball Playing & Passing': ['passes_completed', 'passes', 'passes_pct',
'passes_completed_short', 'passes_short', 'passes_pct_short', 'passes_completed_medium',
'passes_medium', 'passes_pct_medium', 'passes_completed_long', 'passes_long', 'passes_pct_long',
'progressive_passes', 'pass_xa', 'passes_total_distance', 'passes_progressive_distance'],
        'Progressive Play & Build-Up': ['passes_into_final_third', 'passes_into_penalty_area',
'progressive_carries', 'carries_into_final_third', 'carries_into_penalty_area', 'carries',
'carries_distance', 'carries_progressive_distance', 'progressive_passes_received'],
        'Dribbling & Take-Ons': ['take_ons', 'take_ons_won', 'take_ons_won_pct',
'take_ons_tackled', 'take_ons_tackled_pct', 'miscontrols', 'dispossessed'],
        'Attacking Contribution': ['goals', 'goals_per90', 'assists', 'assists_per90',
'goals_assists', 'goals_assists_per90', 'shots', 'shots_on_target', 'shots_per90',
'shots_on_target_per90', 'shots_on_target_pct', 'sca', 'sca_per90', 'gca', 'gca_per90'],
        'Expected Goals (xG) & xA': ['xg', 'xg_per90', 'xg_assist', 'xg_assist_per90', 'npxg',
'npxg_per90', 'npxg_xg_assist', 'npxg_xg_assist_per90', 'xgot', 'xgot_per90'],
        'Crosses & Set Pieces': ['through_balls', 'crosses_into_penalty_area', 'passes_switches',
'misc_crosses', 'pens_made', 'pens_att', 'pens_won', 'corner_kicks_in', 'corner_kicks_out',
'corner_kicks_straight', 'passes_free_kicks', 'shots_free_kicks'],
        'Touches & Ball Control': ['touches', 'touches_def_pen_area', 'touches_def_3rd',
'touches_mid_3rd', 'touches_att_3rd', 'touches_att_pen_area', 'touches_live_ball', 'ball_recoveries',
'passes_received'],
        'Discipline & Errors': ['fouls', 'cards_yellow', 'cards_red', 'cards_yellow_red',
'errors', 'own_goals', 'offsides', 'pens_conceded'],
    },
    'MF': {
        'Creativity & Chance Creation': ['assists', 'assists_per90', 'through_balls', 'sca',
'sca_per90', 'sca_passes_live', 'sca_passes_dead', 'sca_take_ons', 'sca_shots', 'sca_fouled', 'gca',
'gca_per90', 'gca_passes_live', 'gca_passes_dead'],
        'Expected Assists (xA)': ['xg_assist', 'xg_assist_per90', 'pass_xa', 'assisted_shots',
'pass_xg_assist', 'xg_assist_net'],
        'Passing & Distribution': ['passes_completed', 'passes', 'passes_pct',
'passes_completed_short', 'passes_short', 'passes_pct_short', 'passes_completed_medium',
'passes_medium', 'passes_pct_medium', 'passes_completed_long', 'passes_long', 'passes_pct_long',
'progressive_passes', 'passes_total_distance', 'passes_progressive_distance'],
        'Final Third & Penetration': ['passes_into_final_third', 'passes_into_penalty_area',
'crosses_into_penalty_area', 'passes_switches', 'corner_kicks_in', 'corner_kicks_out',
'corner_kicks_straight', 'misc_crosses'],
        'Ball Carrying & Progressive Play': ['progressive_carries', 'carries_into_final_third',
'carries_into_penalty_area', 'carries', 'carries_distance', 'carries_progressive_distance',
'ball_recoveries', 'progressive_passes_received'],
        'Dribbling & Take-Ons': ['take_ons', 'take_ons_won', 'take_ons_won_pct',
'take_ons_tackled', 'take_ons_tackled_pct', 'miscontrols', 'dispossessed'],
        'Goal Threat & Shooting': ['goals', 'goals_per90', 'shots', 'shots_on_target',
'shots_per90', 'shots_on_target_per90', 'shots_on_target_pct', 'goals_per_shot',

```

```

'goals_per_shot_on_target', 'shots_free_kicks', 'average_shot_distance', 'xgot_net'],
  'Expected Goals (xG)': ['xg', 'xg_per90', 'npxg', 'npxg_per90', 'xg_net',
'npxg_xg_assist', 'npxg_xg_assist_per90', 'npxg_per_shot', 'xg_xg_assist_per90', 'xgot',
'xgot_per90'],
  'Defensive Contribution': ['tackles', 'tackles_won', 'tackles_def_3rd',
'tackles_mid_3rd', 'tackles_att_3rd', 'interceptions', 'blocks', 'blocked_shots', 'blocked_passes',
'challenge_tackles', 'challenges', 'tackles_interceptions'],
  'Aerial & Physical Duels': ['aerials_won', 'aerials_lost', 'aerials_won_pct', 'fouls',
'fouled', 'challenges', 'challenges_lost'],
  'Touches & Positioning': ['touches', 'touches_def_pen_area', 'touches_def_3rd',
'touches_mid_3rd', 'touches_att_3rd', 'touches_att_pen_area', 'touches_live_ball',
'passes_received'],
  'Discipline & Game Management': ['cards_yellow', 'cards_red', 'cards_yellow_red',
'offsides', 'errors', 'own_goals', 'pens_conceded'],
},
  'FW': {
    'Finishing & Clinical': ['goals', 'goals_per90', 'goals_per_shot',
'goals_per_shot_on_target', 'shots', 'shots_on_target', 'shots_per90', 'shots_on_target_per90',
'shots_on_target_pct', 'shots_free_kicks', 'average_shot_distance', 'goals_pens', 'goals_pens_per90',
'xgot_net'],
    'Expected Goals (xG) & Efficiency': ['xg', 'xg_per90', 'npxg', 'npxg_per90', 'xg_net',
'npxg_net', 'npxg_per_shot', 'npxg_xg_assist', 'npxg_xg_assist_per90', 'shot_xg', 'shot_npxg',
'xgot', 'xgot_per90'],
    'Creativity & Assists': ['assists', 'assists_per90', 'through_balls', 'sca', 'sca_per90',
'sca_passes_live', 'sca_passes_dead', 'sca_take_ons', 'gca', 'gca_per90', 'gca_passes_live',
'gca_passes_dead'],
    'Expected Assists (xA)': ['xg_assist', 'xg_assist_per90', 'pass_xa', 'assisted_shots',
'pass_xg_assist', 'xg_assist_net', 'xg_xg_assist_per90'],
    'Dribbling & 1v1 Skills': ['take_ons', 'take_ons_won', 'take_ons_won_pct',
'take_ons_tackled', 'take_ons_tackled_pct'],
    'Ball Control & Touch': ['touches', 'touches_att_pen_area', 'touches_att_3rd',
'touches_live_ball', 'miscontrols', 'dispossessed', 'ball_recoveries', 'passes_received'],
    'Progressive Play & Carries': ['progressive_carries', 'carries_into_penalty_area',
'carries_into_final_third', 'carries', 'carries_distance', 'carries_progressive_distance',
'progressive_passes_received'],
    'Penalties & Set Pieces': ['pens_made', 'pens_att', 'pens_won', 'shots_free_kicks',
'corner_kicks_in', 'corner_kicks_out', 'corner_kicks_straight', 'passes_free_kicks'],
    'Aerial & Heading': ['aerials_won', 'aerials_lost', 'aerials_won_pct', 'fouled'],
    'Link-Up & Passing': ['passes_completed', 'passes', 'passes_pct',
'passes_into_penalty_area', 'passes_into_final_third', 'progressive_passes', 'pass_xa',
'crosses_into_penalty_area', 'passes_total_distance', 'passes_progressive_distance'],
    'Defensive Work': ['tackles', 'tackles_won', 'interceptions', 'blocks',
'challenge_tackles', 'challenges', 'tackles_interceptions'],
    'Discipline': ['cards_yellow', 'cards_red', 'cards_yellow_red', 'fouls', 'offsides',
'errors', 'own_goals', 'pens_conceded'],
  }
}

# Build the radar on ONLY real, available 2025-26 metrics: strip retired +
# 2025-26-unavailable metrics, keep any category that still has >=1 real metric
# (so honest single-metric axes like Distribution=Passes-Completed survive),
# and drop only the fully-empty ones.
result = {
  pos: {cat: kept
    for cat, metrics in cats.items()
    if len(kept := [m for m in metrics if m not in EXCLUDED_METRICS]) >= 1}
  for pos, cats in raw.items()
}

# Drop remnant categories that survive with a metric but are junk:
# DF 'Touches & Ball Control' -> only ball_recoveries+passes_received (misabeled)
# GK 'Expected Goals (xG) Conceded' -> only gk_psxg_net (duplicate of Post-Shot xG)
# GK 'Penalties & Set Pieces' -> only pens_conceded (0 for ~all keepers, meaningless)
# MF 'Aerial & Physical Duels' -> only fouls (misabeled - no aerial data)
for pos, cat in [('DF', 'Touches & Ball Control'),

```

```

        ('GK', 'Expected Goals (xG) Conceded'),
        ('GK', 'Penalties & Set Pieces'),
        ('MF', 'Aerial & Physical Duels'),
        ('MF', 'Ball Carrying & Progressive Play')]: # only ball_recoveries left
    result.get(pos, {}).pop(cat, None)
return result

```

`get\_hidden\_gems\_metrics` (def)

```

def get_hidden_gems_metrics():
    return {
        'GK': {
            'expected_performance': ['gk_save_pct', 'gk_saves', 'gk_clean_sheets_pct'],
            'progression': ['passes_completed', 'gk_clean_sheets_pct', 'minutes'],
            'key_metrics': ['gk_clean_sheets', 'gk_goals_against', 'gk_saves'],
        },
        'DF': {
            'expected_performance': ['xg_per90', 'npxg_per90', 'xg_assist_per90'],
            'progression': ['xg_assist_per90', 'xg_per90'],
            'key_metrics': ['tackles_per90', 'interceptions_per90', 'clearances_per90', 'goals',
'assists'],
        },
        'MF': {
            'expected_performance': ['xg_per90', 'xg_assist_per90', 'npxg_per90'],
            'progression': ['xg_assist_per90', 'xg_per90', 'assists_per90'],
            'key_metrics': ['xg_assist_per90', 'take_ons_won', 'goals', 'assists'],
        },
        'FW': {
            'expected_performance': ['xg_per90', 'npxg_per90', 'xg_assist_per90'],
            'progression': ['take_ons_won', 'xgot_per90', 'shots_on_target_per90'],
            'key_metrics': ['goals', 'assists', 'shots_per90', 'shots_on_target_pct'],
        }
    }

```

Market Value Functions (From Legacy)

`calculate\_player\_market\_value` (def) - The rule-based valuation (chosen by hand, not fitted from fees): league_base(power) * position_mult * age_mult * fitness_mult * performance_mult. Used only when a real market value is absent.

```

def calculate_player_market_value(power_rating, age, position, league, minutes=0, games=0,
performance=None):
    # League base (per power rating). The power rating already encodes league
    # strength (PL=100, Bundesliga=86.3, Serie B=76, ...), so there is NO
    # separate league-name multiplier - that old multiplier was both redundant
    # (double-counting league) and broken (it matched space names like
    # "premier league" but our data uses slugs like "premier-league", so the
    # top leagues silently fell to the 0.3 "others" bucket). Real values, when
    # present, are always used instead of this; this only drives the estimate.
    if power_rating >= 90:
        base = 6_000_000 + (power_rating - 90) * 600_000 # 6-12M
    elif power_rating >= 80:
        base = 3_000_000 + (power_rating - 80) * 300_000 # 3-6M
    elif power_rating >= 70:
        base = 1_500_000 + (power_rating - 70) * 150_000 # 1.5-3M
    elif power_rating >= 60:
        base = 750_000 + (power_rating - 60) * 75_000 # 0.75-1.5M
    elif power_rating >= 50:
        base = 300_000 + (power_rating - 50) * 45_000 # 0.3-0.75M
    else:
        base = 100_000 + max(0, (power_rating - 40)) * 20_000

```

```

pm = {'FW': 1.15, 'MF': 1.05, 'DF': 0.95, 'GK': 0.85}.get(position, 1.0)

if age < 21: am = 1.2
elif age < 24: am = 1.3
elif age <= 26: am = 1.2
elif age <= 28: am = 1.0
elif age <= 30: am = 0.8
elif age <= 32: am = 0.5
else: am = 0.3

fm = 1.0
if games > 0 and minutes > 0:
    avg_min = minutes / games
    if avg_min > 80: fm = 1.1
    elif avg_min < 45: fm = 0.8

# Performance scaling (0-100). The league base reflects roughly a TOP
# player in that league; this scales it down for everyone else. Curve
# peaks at performance 100, so even an 87-percentile player is discounted
# (~0.76) rather than sitting at the base. None -> no scaling.
perf_mult = 1.0
if performance is not None:
    p = max(0.0, min(100.0, float(performance)))
    perf_mult = min(1.1, max(0.04, (p / 100.0) ** 2.0))

return base * pm * am * fm * perf_mult

```

`\_perf\_signal` (def) - Player performance signal (0-100) for scaling the MV/wage estimate. Prefers the league-neutral z-score aggregate, then composite.

```

def _perf_signal(row):
    """Player performance signal (0-100) for scaling the MV/wage estimate.
    Prefers the league-neutral z-score aggregate, then composite. Returns
    None when neither is present so legacy callers keep the old behaviour."""
    for key in ('zscore_comp', 'composite_index'):
        v = row.get(key)
        if v is not None and not (isinstance(v, float) and pd.isna(v)):
            return safe_float(v)
    return None

```

`get\_wage\_value` (def) - Returns a weekly wage: the real wage if present, otherwise estimated from the market value ($mv * 0.004 / 52$, floored at 500). The second return value flags whether it was estimated.

```

def get_wage_value(row):
    wage = safe_float(row.get('weekly_wage_eur', 0))
    if wage > 0:
        return wage, False
    annual = safe_float(row.get('annual_wage_eur', 0))
    if annual > 0:
        return annual / 52, False
    pr = safe_float(row.get('power_rating', 50))
    age = parse_age(row.get('age', 25)) or 25 # age is "YY-DDD" in FBref data; safe_float would read 0
    pos = str(row.get('primary_position', row.get('position', 'MF')))
    league = str(row.get('league', ''))
    mv = calculate_player_market_value(pr, age, pos, league,
                                       safe_float(row.get('minutes', 0)),
                                       safe_float(row.get('games', 0)),
                                       performance=_perf_signal(row))
    estimated = mv * 0.004 / 52

```

```
return max(estimated, 500), True
```

Data-driven market-value models (trained on real market_value_eur)

`MV\_FEATURE\_COLS` (module constant)

```
MV_FEATURE_COLS = ['power_rating', 'age', 'age_sq', 'log_min', 'log_games', 'contract_months',  
                  'goals_per90', 'assists_per90', 'npxg_per90', 'xg_assist_per90', 'sca_per90',  
                  'def_actions_per90', 'aerials_won_pct', 'prog_passes_per90', 'gk_save_pct',  
                  'pos_DF', 'pos_FW', 'pos_GK', 'pos_MF']
```

`MV\_MODEL\_PATH` (module constant)

```
MV_MODEL_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'mv_models.pkl')
```

`\_MV\_MODELS` (module constant)

```
_MV_MODELS = None
```

`\_mv\_features\_dict` (def)

```
def _mv_features_dict(row):  
    age = parse_age(row.get('age', 25)) or 25  
    mins = safe_float(row.get('minutes', 0))  
    games = safe_float(row.get('games', row.get('games_starts', 0)))  
    pos = str(row.get('primary_position', row.get('position', 'MF')))  
  
    def p90(v):  
        return (v / mins * 90.0) if mins > 0 else 0.0  
  
    # Contract length remaining (months) is a real value driver but sparse -  
    # impute a neutral ~30 months (mid-contract) when unknown so it doesn't  
    # push the estimate around for the many players who have no contract on file.  
    cm = contract_months_remaining(row)  
    cm = 30.0 if cm is None else max(0.0, min(60.0, float(cm)))  
  
    # Position-aware performance signals, all POOL-INDEPENDENT raw per-90s /  
    # percentages (no z-score/composite), so training and inference never drift.  
    # These give defenders and keepers real signal instead of goals/assists only:  
    # def_actions_per90 (tackles+interceptions) + aerials for DF, gk_save_pct for GK.  
    return {  
        'power_rating': safe_float(row.get('power_rating', 50)) or 50,  
        'age': age,  
        'age_sq': float(age) * float(age), # value-vs-age is humped, not linear  
        'log_min': float(np.log1p(max(0.0, mins))),  
        'log_games': float(np.log1p(max(0.0, games))),  
        'contract_months': cm,  
        'goals_per90': safe_float(row.get('goals_per90', 0)),  
        'assists_per90': safe_float(row.get('assists_per90', 0)),  
        'npxg_per90': safe_float(row.get('npxg_per90', 0)),  
        'xg_assist_per90': safe_float(row.get('xg_assist_per90', 0)),  
        'sca_per90': safe_float(row.get('sca_per90', 0)),  
        'def_actions_per90': p90(safe_float(row.get('tackles', 0)) +  
safe_float(row.get('interceptions', 0))),  
        'aerials_won_pct': safe_float(row.get('aerials_won_pct', 0)),  
        'prog_passes_per90': p90(safe_float(row.get('progressive_passes', 0))),  
        'gk_save_pct': safe_float(row.get('gk_save_pct', 0)),  
        'pos_DF': 1.0 if pos == 'DF' else 0.0,  
        'pos_FW': 1.0 if pos == 'FW' else 0.0,  
        'pos_GK': 1.0 if pos == 'GK' else 0.0,  
        'pos_MF': 1.0 if pos == 'MF' else 0.0,  
        # not a feature - identity for GroupKFold so the same player can't sit in
```



```

        # both the train and test fold (which inflates the cross-validated R2)
        'player_key': _norm_key(str(row.get('player', ''))) + '|' +
str(int(safe_float(row.get('birth_year', 0)))),
    }

```

`train\_mv\_models` (def) - Fit linear + gradient-boosting models on the REAL market_value_eur values present in the DB, and persist them. Returns evaluation metadata (cross-val R2, n, learned linear coefficients = data-derived multipliers).

```

def train_mv_models(seasons=('2025-2026', '2024-2025', '2023-2024')):
    """Fit linear + gradient-boosting models on the REAL market_value_eur
    values present in the DB, and persist them. Returns evaluation metadata
    (cross-val R2, n, learned linear coefficients = data-derived multipliers)."""
    import pickle
    from sklearn.linear_model import LinearRegression
    from sklearn.ensemble import GradientBoostingRegressor
    from sklearn.model_selection import cross_val_score, GroupKFold

    rows = []
    for s in seasons:
        d = merge_supplementary(load_all_leagues_data(s), s)
        if d.empty or 'market_value_eur' not in d.columns:
            continue
        d = d[pd.to_numeric(d['market_value_eur'], errors='coerce').fillna(0) > 0]
        for _, r in d.iterrows():
            feat = _mv_features_dict(r.to_dict())
            feat['mv'] = safe_float(r.get('market_value_eur'))
            rows.append(feat)
    if len(rows) < 100:
        return {'error': f'not enough labeled market values ({len(rows)}')}

    train_df = pd.DataFrame(rows)
    X = train_df[MV_FEATURE_COLS].values
    y = np.log1p(train_df['mv'].values)
    groups = train_df['player_key'].values
    lin = LinearRegression().fit(X, y)
    gbm = GradientBoostingRegressor(n_estimators=200, max_depth=3, learning_rate=0.05).fit(X, y)
    # GroupKFold by player identity: the SAME player recurs across the training
    # seasons with near-identical features and value, so plain K-fold would put
    # one season in train and another in test and "predict" a player it has
    # already seen - inflating R2. Grouping by player gives the honest estimate.
    n_groups = len(set(groups))
    cv = GroupKFold(n_splits=min(5, max(2, n_groups)))
    lin_r2 = float(cross_val_score(LinearRegression(), X, y, cv=cv, groups=groups,
    scoring='r2').mean())
    gbm_r2 = float(cross_val_score(GradientBoostingRegressor(n_estimators=200, max_depth=3,
    learning_rate=0.05),
                                X, y, cv=cv, groups=groups, scoring='r2').mean())

    meta = {
        'n': len(train_df),
        'n_players': int(n_groups),
        'seasons': list(seasons),
        'cv': 'GroupKFold(player)',
        'linear_r2': round(lin_r2, 3),
        'gbm_r2': round(gbm_r2, 3),
        'linear_coefficients': {c: round(float(v), 3) for c, v in zip(MV_FEATURE_COLS, lin.coef_)},
        'linear_intercept': round(float(lin.intercept_), 3),
        'trained_at': datetime.now().isoformat(timespec='seconds'),
    }

    with open(MV_MODEL_PATH, 'wb') as fh:
        pickle.dump({'linear': lin, 'gbm': gbm, 'feature_cols': MV_FEATURE_COLS, 'meta': meta}, fh)
    # JSON sidecar - lets the linear method predict from coefficients without
    # unpickling sklearn, so it survives a scikit-learn version mismatch.

```

```

with open(MV_MODEL_PATH.replace('.pkl', '.json'), 'w') as fh:
    json.dump({'feature_cols': MV_FEATURE_COLS, 'meta': meta}, fh, indent=2)
global _MV_MODELS
_MV_MODELS = None
return meta

```

`\_read\_mv\_files` (def)

```

def _read_mv_files():
    out = {}
    # JSON sidecar first (version-proof; powers the linear method by coefficients)
    try:
        with open(MV_MODEL_PATH.replace('.pkl', '.json')) as fh:
            out.update(json.load(fh))
    except Exception:
        pass
    # Pickle (sklearn models; powers GBM, and linear if version-compatible)
    try:
        import pickle
        with open(MV_MODEL_PATH, 'rb') as fh:
            out.update(pickle.load(fh))
    except Exception:
        pass
    return out

```

`\_load\_mv\_models` (def)

```

def _load_mv_models():
    global _MV_MODELS
    if _MV_MODELS is not None:
        return _MV_MODELS
    out = _read_mv_files()
    # Auto-(re)train if there is no model yet, or the persisted one was fitted on
    # a different feature set (i.e. after a model upgrade) - so new features take
    # effect on deploy without a manual retrain. Guarded: if training can't run
    # (no data), keep whatever loaded and fall back to the heuristic.
    stored = out.get('feature_cols')
    if (not out) or (stored is not None and list(stored) != list(MV_FEATURE_COLS)):
        try:
            meta = train_mv_models()
            if isinstance(meta, dict) and 'error' not in meta:
                out = _read_mv_files()
        except Exception as e:
            print(f"[mv] auto-retrain skipped: {e}", file=sys.stderr, flush=True)
    _MV_MODELS = out
    return out

```

`\_model\_predict\_mv` (def)

```

def _model_predict_mv(row, which):
    models = _load_mv_models()
    if not models:
        return None
    cols = models.get('feature_cols', MV_FEATURE_COLS)
    feat = _mv_features_dict(row)
    pred = None
    model = models.get(which)
    if model is not None and hasattr(model, 'predict'):
        try:
            pred = float(np.expml(model.predict([[feat[c] for c in cols]])[0]))
        except Exception:
            pred = None
    # Linear fallback: compute from stored coefficients (no sklearn needed)

```

```

if pred is None and which == 'linear':
    meta = models.get('meta', {})
    coefs = meta.get('linear_coefficients')
    inter = meta.get('linear_intercept')
    if coefs and inter is not None:
        pred = float(np.expml(inter + sum(coefs.get(c, 0) * feat.get(c, 0) for c in cols)))
if pred is None:
    return None
return max(50_000.0, min(250_000_000.0, pred))

```

`get\_market\_value` (def)

```

def get_market_value(row, method='heuristic'):
    mv = safe_float(row.get('market_value_eur', 0))
    if mv > 0:
        return mv, False
    pr = safe_float(row.get('power_rating', 50))
    age = parse_age(row.get('age', 25)) or 25 # age is "YY-DDD" in FBref data; safe_float would read 0
    pos = str(row.get('primary_position', row.get('position', 'MF')))
    league = str(row.get('league', ''))
    est = calculate_player_market_value(pr, age, pos, league,
                                       safe_float(row.get('minutes', 0)),
                                       safe_float(row.get('games', 0)),
                                       performance=_perf_signal(row))

    return est, True

```

`predict\_mv\_from\_performance` (def) - The model's performance-predicted value, IGNORING any real price (unlike `get_market_value`, which returns the real price when it exists). Used to compute the value `RESIDUAL = actual - predicted` (negative = market underpays = a gem).

```

def predict_mv_from_performance(row, method='heuristic'):
    """The model's performance-predicted value, IGNORING any real price (unlike
    get_market_value, which returns the real price when it exists). Used to compute
    the value RESIDUAL = actual - predicted (negative = market underpays = a gem)."""
    pr = safe_float(row.get('power_rating', 50))
    age = parse_age(row.get('age', 25)) or 25
    pos = str(row.get('primary_position', row.get('position', 'MF')))
    league = str(row.get('league', ''))
    return calculate_player_market_value(pr, age, pos, league,
                                       safe_float(row.get('minutes', 0)),
                                       safe_float(row.get('games', 0)),
                                       performance=_perf_signal(row))

```

`contract\_months\_remaining` (def)

```

def contract_months_remaining(row):
    expiry = row.get('contract_expiry', None)
    if expiry is None or pd.isna(expiry):
        return None
    try:
        if isinstance(expiry, str):
            for fmt in ('%Y-%m-%d', '%d/%m/%Y', '%Y-%m', '%Y'):
                try:
                    exp_dt = datetime.strptime(expiry.strip(), fmt)
                    break
                except ValueError:
                    continue
            else:
                return None
        else:
            return None
    except:
        return None

```

```

        exp_dt = pd.Timestamp(expiry).to_pydatetime()
        now = datetime.now()
        months = (exp_dt.year - now.year) * 12 + (exp_dt.month - now.month)
        return max(0, months)
    except Exception:
        return None

```

`contract\_opportunity\_breakdown` (def) - Splits the contract-opportunity score into urgency (less time left = more leverage) + release-clause discount (a clause below market value is a cheap exit), capped 0-100.

```

def contract_opportunity_breakdown(row):
    """Contract-opportunity score split into its two parts so the lab can show
    the working: urgency (less time left = more leverage) + release-clause
    discount (a clause below market value = a bargain exit). Capped 0-100."""
    months = contract_months_remaining(row)
    mv, _ = get_market_value(row)
    rc = safe_float(row.get('release_clause_eur', 0))
    if months is None: urgency = 20
    elif months <= 0: urgency = 60
    elif months <= 6: urgency = 55
    elif months <= 12: urgency = 45
    elif months <= 18: urgency = 30
    elif months <= 24: urgency = 15
    else: urgency = 5
    clause_score = 0
    if rc > 0 and mv > 0:
        ratio = rc / mv
        if ratio < 0.5: clause_score = 40
        elif ratio < 0.75: clause_score = 30
        elif ratio < 1.0: clause_score = 20
        elif ratio < 1.25: clause_score = 10
    return {
        'months': months,
        'urgency': urgency,
        'clause': clause_score,
        'total': round(min(100, urgency + clause_score), 1),
    }

```

`calculate\_contract\_opportunity\_score` (def)

```

def calculate_contract_opportunity_score(row):
    return contract_opportunity_breakdown(row)['total']

```

`calculate\_moneyball` (def) - The blend the lab shows: performance * 0.5 + value_efficiency * 0.3 + contract_opportunity * 0.2.

```

def calculate_moneyball(perf, val_eff, contract):
    return round(perf * 0.5 + val_eff * 0.3 + contract * 0.2, 1)

```

Data Loading

`load\_meta` (def)

```

def load_meta():
    with get_connection() as conn:
        df = pd.read_sql("""
            SELECT DISTINCT season, league, team
            FROM league_season_team_player_data
            ORDER BY season DESC, league, team

```

```

        """ , conn)
    return df

```

`load\_players` (def) - Loads one team's players for a season from the database (here, the SQLite snapshot).

```

def load_players(season, league, team):
    with get_connection() as conn:
        df = pd.read_sql("""
            SELECT * FROM league_season_team_player_data
            WHERE season = %s AND league = %s AND team = %s
        """, conn, params=[season, league, team])
    if not df.empty:
        df = standardize_positions(df)
    return df

```

`load\_league\_data` (def) - Loads a whole league-season pool.

```

def load_league_data(season, league):
    with get_connection() as conn:
        df = pd.read_sql("""
            SELECT * FROM league_season_team_player_data
            WHERE season = %s AND league = %s
        """, conn, params=[season, league])
    if not df.empty:
        df = standardize_positions(df)
    return df

```

`\_ALL\_LEAGUES\_CACHE` (module constant)

```

_ALL_LEAGUES_CACHE = {}

```

`load\_all\_leagues\_data` (def) - Loads every league for a season (cached).

```

def load_all_leagues_data(season):
    # Cached per season: multi-season matching loads several seasons and the
    # gem tagging re-loads the same season, so the full-table query + position
    # standardisation would otherwise run many times per request. A defensive
    # copy is returned so callers can mutate freely without corrupting the
    # cache (process-lifetime; cleared by restarting the server after an
    # update, same as the other caches here).
    if season in _ALL_LEAGUES_CACHE:
        return _ALL_LEAGUES_CACHE[season].copy()
    with get_connection() as conn:
        df = pd.read_sql("""
            SELECT * FROM league_season_team_player_data
            WHERE season = %s
        """, conn, params=[season])
    if not df.empty:
        df = standardize_positions(df)
    _ALL_LEAGUES_CACHE[season] = df
    return df.copy()

```

`load\_supplementary` (def) - Loads the supplementary wages/values/contracts table for a season.

```

def load_supplementary(season):
    """Load supplementary data (wages, market values, contracts).
    Falls back to the latest available season if no data for requested season."""
    try:
        with get_connection() as conn:
            df = pd.read_sql("""

```

```

        SELECT * FROM player_supplementary_data WHERE season = %s
        """ , conn, params=[season])
    if df.empty:
        # Fallback: use the latest available season's supplementary data
        df = pd.read_sql("""
            SELECT * FROM player_supplementary_data
            WHERE season = (SELECT MAX(season) FROM player_supplementary_data)
            """ , conn)
    return df
except Exception:
    return pd.DataFrame()

```

`SUPP\_COLS` (module constant)

```

SUPP_COLS = ['contract_expiry', 'release_clause_eur', 'weekly_wage_eur', 'annual_wage_eur',
'market_value_eur']

```

`\_norm\_key` (def) - Normalise a name/team/league for matching: strip accents, lowercase, collapse whitespace. So 'Ádám Nagy' and 'Adam Nagy' match.

```

def _norm_key(s):
    """Normalise a name/team/league for matching: strip accents, lowercase,
    collapse whitespace. So 'Ádám Nagy' and 'Adam Nagy' match."""
    if not isinstance(s, str):
        s = '' if s is None else str(s)
    s = unicodedata.normalize('NFKD', s)
    s = ''.join(c for c in s if not unicodedata.combining(c))
    return ' '.join(s.lower().split())

```

`merge\_supplementary` (def) - Joins wages / market values / contract dates onto the player rows, matched by a normalised name key.

```

def merge_supplementary(df, season):
    """Merge supplementary data (wages, market values, contracts) into a
    player dataframe. Matching is accent-insensitive. First tries an exact
    match on player+team+league; for unmatched rows it falls back to
    player+league (recovers team-name format differences / mid-season
    transfers) WITHOUT crossing leagues - a Liga MX player must never inherit
    a same-named player's wage from another league."""
    supp = load_supplementary(season)
    if supp.empty:
        return df
    supp_cols = [c for c in SUPP_COLS if c in supp.columns]
    supp_dedup = supp[['player', 'team', 'league'] + supp_cols].drop_duplicates().copy()

    df = df.copy()
    df['_pkey'] = df['player'].map(_norm_key)
    df['_tkey'] = df['team'].map(_norm_key)
    df['_lkey'] = df['league'].map(_norm_key)
    supp_dedup['_pkey'] = supp_dedup['player'].map(_norm_key)
    supp_dedup['_tkey'] = supp_dedup['team'].map(_norm_key)
    supp_dedup['_lkey'] = supp_dedup['league'].map(_norm_key)

    # Step 1: strict merge on normalised player + team + league
    supp_strict = supp_dedup[['_pkey', '_tkey', '_lkey'] + supp_cols].drop_duplicates(
        subset=['_pkey', '_tkey', '_lkey'], keep='first')
    merged = df.merge(supp_strict, on=['_pkey', '_tkey', '_lkey'], how='left')

    # Step 2: fallback on normalised player + league only (same league),
    # filling any field still missing. Never matches across leagues.
    supp_by_pl = supp_dedup.drop_duplicates(subset=['_pkey', '_lkey'], keep='first')
    supp_by_pl = supp_by_pl[['_pkey', '_lkey'] + supp_cols].rename(

```

```

        columns={c: c + '_fallback' for c in supp_cols})
merged = merged.merge(supp_by_pl, on=['_pkey', '_lkey'], how='left')
for col in supp_cols:
    fb = col + '_fallback'
    if fb in merged.columns:
        merged[col] = merged[col].fillna(merged[fb])
        merged.drop(columns=[fb], inplace=True)

merged.drop(columns=['_pkey', '_tkey', '_lkey'], inplace=True, errors='ignore')
return merged

```

`load\_player\_history` (def)

```

def load_player_history(player_name):
    with get_connection() as conn:
        df = pd.read_sql("""
            SELECT * FROM league_season_team_player_data
            WHERE player = %s ORDER BY season ASC, league, team
        """, conn, params=[player_name])
    if df.empty:
        df = pd.read_sql("""
            SELECT * FROM league_season_team_player_data
            WHERE LOWER(player) LIKE LOWER(%s)
            ORDER BY season ASC, league, team
        """, conn, params=[f'%{player_name}%'])
    if not df.empty:
        df = standardize_positions(df)
    return df

```

`load\_player\_supplementary\_history` (def) - Every supplementary row (verified market value / wage / contract) for a player across ALL seasons, matched by exact name then accent-insensitive LIKE. Unlike `load_supplementary(season)`, this NEVER falls back to another season - so a recent verified value can't leak onto an older trajectory point.

```

def load_player_supplementary_history(player_name):
    """Every supplementary row (verified market value / wage / contract) for a
    player across ALL seasons, matched by exact name then accent-insensitive
    LIKE. Unlike load_supplementary(season), this NEVER falls back to another
    season - so a recent verified value can't leak onto an older trajectory
    point. Returns the raw frame (season, team, market_value_eur, ...)."""
    try:
        with get_connection() as conn:
            df = pd.read_sql("""
                SELECT * FROM player_supplementary_data WHERE player = %s
            """, conn, params=[player_name])
            if df.empty:
                df = pd.read_sql("""
                    SELECT * FROM player_supplementary_data
                    WHERE LOWER(player) LIKE LOWER(%s)
                """, conn, params=[f'%{player_name}%'])
            return df
    except Exception:
        return pd.DataFrame()

```

Percentile Scoring

`calculate\_percentile\_score` (def)

```

def calculate_percentile_score(value, comparison_values):
    if len(comparison_values) == 0 or pd.isna(value):
        return 0

```

```

clean = [v for v in comparison_values if not pd.isna(v)]
if len(clean) == 0:
    return 0
return stats.percentileofscore(clean, value, kind='rank')

```

`calculate\_category\_scores` (def) - Per-style-category percentile (or normalized) scores for one player against a comparison pool. Powers the profile radar.

```

def calculate_category_scores(player_data, comparison_data, style_categories, position,
method='percentile', empty_as_none=False):
    """empty_as_none=True marks a category with NO measurable metrics (all
    structurally N/A in this league) as None instead of 0, so the profile radar
    and breakdown can show 'no data' rather than a misleading zero. Off by default
    so other callers (gems, similarity, market value) keep numeric scores."""
    if position not in style_categories:
        return {}
    empty_val = None if empty_as_none else 0
    category_scores = {}
    neg_metrics = get_negative_metrics()
    for cat_name, metrics in style_categories[position].items():
        available = [m for m in metrics if m in comparison_data.columns]
        if not available:
            category_scores[cat_name] = empty_val
            continue
        if method == 'percentile':
            percentiles = []
            for metric in available:
                pv = safe_float(player_data.get(metric, 0)) if isinstance(player_data, dict) else
                safe_float(player_data[metric]) if metric in player_data.index else 0
                comp = comparison_data[metric].apply(safe_float).dropna()
                # Skip columns with no signal (all identical / structurally N/A,
                # e.g. average_shot_distance that is 0 for a whole season) so they
                # don't inject a flat ~50 into the category score.
                if len(comp) > 0 and comp.min() != comp.max():
                    pctl = calculate_percentile_score(pv, comp.tolist())
                    if metric in neg_metrics:
                        pctl = 100 - pctl
                    percentiles.append(pctl)
            category_scores[cat_name] = float(np.mean(percentiles)) if percentiles else empty_val
        elif method == 'normalized':
            norms = []
            for metric in available:
                pv = safe_float(player_data.get(metric, 0)) if isinstance(player_data, dict) else
                safe_float(player_data[metric]) if metric in player_data.index else 0
                comp = comparison_data[metric].apply(safe_float).dropna()
                if len(comp) > 0:
                    mn, mx = comp.min(), comp.max()
                    if mx > mn:
                        n = ((pv - mn) / (mx - mn)) * 100
                        n = max(0, min(100, n))
                        if metric in neg_metrics:
                            n = 100 - n
                        norms.append(n)
            category_scores[cat_name] = float(np.mean(norms)) if norms else empty_val
    return category_scores

```

Composite Index (Legacy _Calculate_Composite_Index From Tadv15.Py)

`\_get\_position\_metrics\_for\_composite` (def) - The per-90 / key metrics used in the composite z-score, chosen per position (only metrics actually available in the data).


```
def _get_position_metrics_for_composite(pos):
    """Per-90 / key metrics used in composite z-score by position."""
    if pos == 'GK':
        # only 2025-26-available keeper metrics (crosses/sweeping/passes_pct unavailable)
        return ['gk_save_pct', 'gk_clean_sheets_pct', 'gk_psxg_net_per90',
                'gk_goals_against_per90', 'gk_saves', 'gk_clean_sheets']
    elif pos == 'DF':
        # only 2025-26-available metrics (blocks/aerials/passes_pct/etc. unavailable)
        return ['tackles', 'interceptions', 'clearances', 'tackles_interceptions',
                'ball_recoveries', 'xg_assist_per90', 'xg_per90', 'goals_per90', 'assists_per90']
    elif pos == 'MF':
        return ['xg_per90', 'npxg_per90', 'xg_assist_per90', 'shots_on_target_per90',
                'goals_per90', 'assists_per90', 'xgot_per90', 'take_ons_won']
    else: # FW
        return ['goals_per90', 'assists_per90', 'xg_per90', 'npxg_per90', 'shots_per90',
                'shots_on_target_pct', 'xg_assist_per90', 'npxg_per_shot',
                'xgot_per90', 'goals_per_shot', 'take_ons_won']
```

`calculate\_composite\_index` (def) - The core score. $\text{composite} = 0.40 * \text{z-score aggregate (percentile-ranked)} + 0.30 * \text{style-category percentile average (coverage-aware)} + 0.30 * \text{league power rating}$. Adds `zscore_comp` / `style_pctile_avg` / `power_norm` / `composite_index` columns to the pool.

```
def calculate_composite_index(pos_df, pos, style_categories):
    """Calculate composite index for every player in pos_df.

    Composite = 40% z-score aggregate + 30% style-category percentile avg + 30% power rating
    (normalised).
    Negative metrics inverted so higher = better.
    Returns pos_df with extra columns: zscore_comp, style_pctile_avg, power_norm, composite_index.
    """
    neg_metrics = get_negative_metrics()
    metrics = _get_position_metrics_for_composite(pos)

    # --- 1. Z-score aggregation (40%) ---
    available = [m for m in metrics if m in pos_df.columns]
    zscore_sums = pd.Series(0.0, index=pos_df.index)
    valid_count = pd.Series(0, index=pos_df.index)

    for m in available:
        col = _num_series(pos_df[m])
        non_null = ~col.isna()
        mean_val = col.mean()
        std_val = col.std()
        if std_val > 0:
            z = (col - mean_val) / std_val
            if m in neg_metrics:
                z = -z
            zscore_sums += z.fillna(0)
            valid_count += non_null.astype(int)

    avg_z = zscore_sums / valid_count.replace(0, 1)
    # Percentile rank, not min-max: min-max is squashed by small-sample
    # outliers (a freak per-90 stretches the scale and crushes everyone else,
    # so even elite players landed near the bottom). Percentile rank is
    # outlier-immune, uniform 0-100, and comparable across seasons.
    if len(avg_z) > 1:
        zscore_norm = avg_z.rank(pct=True) * 100
    else:
        zscore_norm = pd.Series(50.0, index=pos_df.index)

    # --- 2. Style-category percentile average (30%) ---
```

```

# Coverage-aware: only count categories where the player has data for
# >= 50% of the metrics in that category. Avoids low-minutes players
# getting credited with median (50) for every missing stat, which used
# to inflate composites for players with tiny samples.
position_cats = style_categories.get(pos, {})
cat_sums = pd.Series(0.0, index=pos_df.index)
cat_valid_per_player = pd.Series(0, index=pos_df.index)

for cat_name, cat_metrics in position_cats.items():
    avail_cat = [m for m in cat_metrics if m in pos_df.columns]
    if not avail_cat:
        continue
    cat_pctile_sum = pd.Series(0.0, index=pos_df.index)
    cat_pctile_present = pd.Series(0, index=pos_df.index)
    for m in avail_cat:
        col = _num_series(pos_df[m])
        pctile = col.rank(pct=True, na_option='keep') * 100
        if m in neg_metrics:
            pctile = 100 - pctile
        non_null = ~pctile.isna()
        cat_pctile_sum += pctile.fillna(0)
        cat_pctile_present += non_null.astype(int)
    coverage_threshold = max(1, len(avail_cat) // 2)
    sufficient = cat_pctile_present >= coverage_threshold
    cat_avg = cat_pctile_sum / cat_pctile_present.replace(0, 1)
    cat_sums += cat_avg.where(sufficient, 0)
    cat_valid_per_player += sufficient.astype(int)

style_avg = cat_sums / cat_valid_per_player.replace(0, 1)
style_avg = style_avg.where(cat_valid_per_player > 0, 50)
style_avg = style_avg.clip(0, 100)

# --- 3. Power rating (30%) ---
# Each league's quality is already on a 0-100 scale (PL=100,
# Bundesliga=86.3, ..., Serie B=76) - use it directly. The
# min-max normalisation only made sense over a multi-league pool;
# the lab restricts the comparison cohort to one league + one
# position, so min == max == league_power, and the normalisation
# collapsed to 50 for everyone - capping every composite at 85
# and making the Elite (80+) / World Class (90+) tiers from the
# Day 1 course content unreachable.
if 'power_rating' not in pos_df.columns:
    pos_df = pos_df.copy()
    pos_df['power_rating'] = 0
pr = pos_df['power_rating'].apply(safe_float)
power_norm = pr.clip(0, 100)

# --- Composite ---
composite = zscore_norm * 0.40 + style_avg * 0.30 + power_norm * 0.30

pos_df = pos_df.copy()
pos_df['zscore_comp'] = zscore_norm.round(1)
pos_df['style_pctile_avg'] = style_avg.round(1)
pos_df['power_norm'] = power_norm.round(1)
pos_df['composite_index'] = composite.round(1)
return pos_df

```

`get\_power\_rating` (def) - Get league-level power rating from player data.

```

def get_power_rating(player_data):
    """Get league-level power rating from player data."""
    if isinstance(player_data, dict):
        return safe_float(player_data.get('power_rating', 0))

```

```

if 'power_rating' in player_data.index:
    return safe_float(player_data['power_rating'])
return 0.0

```

`composite\_description` (def) - Turns a composite into a tier label: ≥ 90 World Class, ≥ 80 Elite, ≥ 70 Excellent, ≥ 60 Good, ≥ 50 Average, ≥ 40 Below Average, else Poor.

```

def composite_description(score):
    """Describe composite index score."""
    if score >= 90: return "World Class"
    if score >= 80: return "Elite"
    if score >= 70: return "Excellent"
    if score >= 60: return "Good"
    if score >= 50: return "Average"
    if score >= 40: return "Below Average"
    return "Poor"

```

`\_season\_start\_year` (def)

```

def _season_start_year(season):
    try:
        return int(str(season).split('-')[0])
    except (ValueError, AttributeError):
        return 0

```

`\_MULTISEASON\_CACHE` (module constant)

```

_MULTISEASON_CACHE = {}

```

`compute\_multiseason\_features` (def) - Identifies a player across seasons (by name + birth year) and derives trajectory / consistency features that power the Riser gem signal and the trajectory label.

```

def compute_multiseason_features(season, league, position, lookback=3, min_minutes=270):
    """Build a per-player performance time series over [season-lookback, season]
    and derive trajectory/consistency features. Players are identified across
    seasons by normalised name + birth_year (birth_year ~99.6% populated).

    Uses the league-neutral z-score component (zscore_comp), NOT full composite,
    so a player moving to a stronger league doesn't show a fake trajectory bump
    from league power. Returns {(_pkey, birth_year): {...features...}}.
    """
    cache_key = (season, league or '_all_', position, lookback, min_minutes)
    if cache_key in _MULTISEASON_CACHE:
        return _MULTISEASON_CACHE[cache_key]

    start = _season_start_year(season)
    window = [f"{y}-{y + 1}" for y in range(start - lookback, start + 1)]
    style_cats = get_playing_style_categories()

    per_player = {}
    for s in window:
        d = load_league_data(s, league) if league else load_all_leagues_data(s)
        if d.empty:
            continue
        pos_d = d[d['primary_position'] == position].copy() if 'primary_position' in d.columns else
d.copy()
        if 'minutes' in pos_d.columns:
            pos_d = pos_d[_num_series(pos_d['minutes']).fillna(0) >= min_minutes]
        if pos_d.empty:
            continue
        pos_d = calculate_composite_index(pos_d, position, style_cats)

```

```

si = _season_start_year(s)
for _, r in pos_d.iterrows():
    by = int(safe_float(r.get('birth_year', 0)))
    if by <= 0:
        continue
    key = (_norm_key(str(r.get('player', ''))), by)
    per_player.setdefault(key, []).append({
        'season': s, 'idx': si,
        'name': str(r.get('player', '')),
        'score': safe_float(r.get('zscore_comp', 0)),
        'minutes': safe_float(r.get('minutes', 0)),
    })

feats = {}
for key, hist in per_player.items():
    hist = sorted(hist, key=lambda x: x['idx'])
    scores = [h['score'] for h in hist]
    idxs = [h['idx'] for h in hist]
    mins = [h['minutes'] for h in hist]
    n = len(hist)
    slope = float(np.polyfit(idxs, scores, 1)[0]) if n >= 2 else 0.0
    min_slope = float(np.polyfit(idxs, mins, 1)[0]) if n >= 2 else 0.0
    consistency = float(np.std(scores)) if n >= 2 else 0.0
    latest = scores[-1]
    prior_avg = (sum(scores[:-1]) / (n - 1)) if n >= 2 else latest
    feats[key] = {
        'player': hist[-1]['name'],
        'seasons_tracked': n,
        'series': [{ 'season': h['season'], 'score': round(h['score'], 1), 'minutes':
int(round(h['minutes']))} for h in hist],
        'trajectory_slope': round(slope, 2),
        'consistency_std': round(consistency, 2),
        'peak': round(max(scores), 1),
        'latest': round(latest, 1),
        'breakout': bool(n >= 2 and (latest - prior_avg) >= 8),
        'minutes_slope': round(min_slope, 1),
    }
    _MULTISEASON_CACHE[cache_key] = feats
return feats

```

`\_VOLUME\_METRICS` (module constant)

```

_VOLUME_METRICS = {
    'minutes', 'gk_minutes', 'games', 'games_starts', 'gk_games', 'gk_games_starts',
}

```

`\_is\_rate\_metric` (def) - A metric already expressed as a rate/ratio (per-90, percentage, average, per-game). These are blended with a minutes-weighted average; everything else is a counting total and gets converted to a true per-90.

```

def _is_rate_metric(m):
    """A metric already expressed as a rate/ratio (per-90, percentage, average,
    per-game). These are blended with a minutes-weighted average; everything
    else is a counting total and gets converted to a true per-90."""
    return (m.endswith('_per90') or m.endswith('_pct') or '_avg' in m
            or 'per_game' in m or m == 'points_per_game'
            or m.endswith('_per_shot_on_target_against'))

```

`\_MS\_POOL\_CACHE` (module constant)

```

_MS_POOL_CACHE = {}

```

``_MS_SERIES_CACHE`` (module constant)

```
_MS_SERIES_CACHE = {}
```

``_multiseason_pool`` (def) - Aggregate each player's stats over the last window seasons into one minutes-weighted row per player identity (normalised name + birth year). Powers the multi-season 'average level' similarity mode: a player's numbers are blended across seasons (weighted by how much they played each season), so the match reflects a sustained profile, not one season.

```
def _multiseason_pool(season, position, window):
    """Aggregate each player's stats over the last `window` seasons into one
    minutes-weighted row per player identity (normalised name + birth year).
    Powers the multi-season 'average level' similarity mode: a player's numbers
    are blended across seasons (weighted by how much they played each season),
    so the match reflects a sustained profile, not one season. Returns a frame
    shaped like a single-season position pool, with `minutes` = total across the
    window and `seasons_count` = how many seasons the player appears in.

    Cached per (season, position, window) so the Similar Players list and every
    head-to-head comparison reuse one aggregation instead of rebuilding it."""
    window = max(2, min(5, int(window)))
    ck = (season, position, window)
    if ck in _MS_POOL_CACHE:
        return _MS_POOL_CACHE[ck].copy()
    start = _season_start_year(season)
    win = [f"{y}-{y + 1}" for y in range(start - (window - 1), start + 1)]
    style = get_playing_style_categories()
    mset = set()
    for ms in style.get(position, {}).values():
        mset.update(ms)
    frames = []
    for s in win:
        d = load_all_leagues_data(s)
        if d.empty:
            continue
        d = position_pool(d, position)
        if d.empty:
            continue
        d = d.copy()
        d['_idx'] = _season_start_year(s)
        d['_season'] = s
        frames.append(d)
    if not frames:
        return None
    alld = pd.concat(frames, ignore_index=True, sort=False)
    metric_cols = [m for m in mset if m in alld.columns]
    if not metric_cols:
        return None
    alld['_pkey'] = alld['player'].astype(str).map(_norm_key)
    alld['_by'] = (_num_series(alld['birth_year']).fillna(0).astype(int)
                  if 'birth_year' in alld.columns else 0)
    alld['_min'] = (_num_series(alld['minutes']).fillna(0)
                   if 'minutes' in alld.columns else 0.0)
    nummat = {m: _num_series(alld[m]) for m in metric_cols}
    rows = []
    series_map = {}
    for (pk, by), g in alld.groupby(['_pkey', '_by']):
        idx = g.index
        w = alld.loc[idx, '_min'].values
        # actual minutes per season for this player (the seasons that fed the blend)
        smin = {}
```

```

for s_, mn_ in zip(g['_season'].values, w):
    smin[s_] = smin.get(s_, 0.0) + float(mn_)
series_map[(pk, by)] = smin
rec = {}
for m in metric_cols:
    vals = nummat[m].loc[idx]
    mask = vals.notna().values
    if not mask.any():
        rec[m] = np.nan
        continue
    vv = vals.values[mask]
    ww = w[mask]
    if m in _VOLUME_METRICS:
        rec[m] = float(vv.sum()) # total (volume)
    elif _is_rate_metric(m):
        # already a rate -> minutes-weighted average across seasons
        rec[m] = float(np.average(vv, weights=ww)) if ww.sum() > 0 else float(vv.mean())
    else:
        # counting total -> true per-90 (sum of the stat / sum of 90s),
        # so the blended number reflects rate, not how much he played
        nineties = ww.sum() / 90.0
        rec[m] = float(vv.sum() / nineties) if nineties > 0 else float(vv.mean())
latest = g.sort_values('_idx').iloc[-1]
rec['player'] = latest.get('player', '')
rec['team'] = latest.get('team', '')
rec['league'] = latest.get('league', '')
rec['age'] = latest.get('age', 0)
rec['primary_position'] = latest.get('primary_position', position)
rec['secondary_position'] = latest.get('secondary_position', '')
rec['birth_year'] = by
rec['minutes'] = float(w.sum())
rec['seasons_count'] = int(g['_season'].nunique())
rec['_pkey'] = pk
rec['_by'] = by
rows.append(rec)
out = pd.DataFrame(rows)
_MS_POOL_CACHE[ck] = out
_MS_SERIES_CACHE[ck] = series_map
return out.copy()

```

`\_GEM\_KEYSET\_CACHE` (module constant)

```
_GEM_KEYSET_CACHE = {}
```

`\_gem\_keyset` (def) - Set of (normalised name, team) for players flagged as hidden gems in the given scope. Cached per (season, scope, position) - the gem computation is heavy, and the similar-players list only needs a membership test.

```

def _gem_keyset(season, position, league=None):
    """Set of (normalised name, team) for players flagged as hidden gems in the
    given scope. Cached per (season, scope, position) - the gem computation is
    heavy, and the similar-players list only needs a membership test."""
    ck = (season, league or '_all', position)
    if ck in _GEM_KEYSET_CACHE:
        return _GEM_KEYSET_CACHE[ck]
    req = {'season': season, 'position': position}
    if league:
        req['league'] = league
    try:
        res = cmd_get_hidden_gems(req)
        ks = {(_norm_key(str(g.get('player', ''))), str(g.get('team', '')))
              for g in res.get('gems', [])}
    
```

```

except Exception:
    ks = set()
    _GEM_KEYSET_CACHE[ck] = ks
return ks

```

`\_AVAIL\_CACHE` (module constant)

```
_AVAIL_CACHE = {}
```

`\_availability\_index` (def) - Supplementary-merged rows for a season, indexed by (name, team, league) so each similar player can be tagged with his contract/wage/value. Cached.

```

def _availability_index(season):
    """Supplementary-merged rows for a season, indexed by (name, team, league)
    so each similar player can be tagged with his contract/wage/value. Cached."""
    if season in _AVAIL_CACHE:
        return _AVAIL_CACHE[season]
    try:
        df = merge_supplementary(load_all_leagues_data(season), season)
        idx = {}
        for rec in df.to_dict('records'):
            k = (_norm_key(str(rec.get('player', ''))),
                 _norm_key(str(rec.get('team', ''))),
                 _norm_key(str(rec.get('league', ''))))
            idx[k] = rec
        _AVAIL_CACHE[season] = idx
    except Exception:
        idx = {}
        _AVAIL_CACHE[season] = idx
    return idx

```

`\_tag\_availability` (def) - Attach contract months, wage, market value, release clause and an 'opportunity' type (free / expiring / clause) to one similar-player dict.

```

def _tag_availability(sp, season, avail):
    """Attach contract months, wage, market value, release clause and an
    'opportunity' type (free / expiring / clause) to one similar-player dict."""
    rec = avail.get((_norm_key(sp['player']), _norm_key(sp['team']), _norm_key(sp['league'])))
    months = wage = mv = rc = None
    opp = None
    if rec is not None:
        months = contract_months_remaining(rec)
        try:
            w, _est = get_wage_value(rec)
            wage = r(w, 0) if w else None
        except Exception:
            wage = None
        mv = safe_float(rec.get('market_value_eur', 0)) or None
        rc = safe_float(rec.get('release_clause_eur', 0)) or None
        if months is not None and months <= 6:
            opp = 'free'
        elif months is not None and months <= 12:
            opp = 'expiring'
        elif rc and rc > 0:
            opp = 'clause'
        sp['contract_months'] = months
        sp['wage'] = wage
        sp['wage_label'] = (format_eur(wage) + '/wk') if wage else None
        sp['market_value'] = r(mv, 0) if mv else None
        sp['market_value_label'] = format_eur(mv) if mv else None
        sp['release_clause'] = r(rc, 0) if rc else None
        sp['release_clause_label'] = format_eur(rc) if rc else None

```

```
sp['opportunity'] = opp
```

`\_trajectory\_label` (def) - Human-readable trajectory classification + one-line summary.

```
def _trajectory_label(f):
    """Human-readable trajectory classification + one-line summary."""
    n = f['seasons_tracked']
    if n < 2:
        return 'Limited history', 'Only one tracked season - no trend yet.'
    series = f['series']
    span = f"z {series[0]['score']:.0f} → {series[-1]['score']:.0f}"
    if f['consistency_std'] > 10:
        return 'Volatile', f'Inconsistent across {n} seasons ({span}) - high variance, treat the
latest season with caution.'
    if f['trajectory_slope'] >= 2:
        extra = ' Breakout - latest season well above prior form.' if f['breakout'] else ''
        return 'Rising', f'Improving across {n} seasons ({span}).{extra}'
    if f['trajectory_slope'] <= -2:
        return 'Declining', f'Trending down across {n} seasons ({span}) - possible age/role decline.'
    return 'Stable', f'Consistent across {n} seasons ({span}) - reliable, predictable level.'
```

`cmd\_backtest\_gems` (def) - Validate the detector: flag gems in season, then measure how those players' performance (league-neutral z-score) and minutes changed horizon seasons later, vs a baseline of all other tracked players. If flagged gems out-improve the baseline, the detector has predictive value.

```
def cmd_backtest_gems(req):
    """Validate the detector: flag gems in `season`, then measure how those
players' performance (league-neutral z-score) and minutes changed `horizon`
seasons later, vs a baseline of all other tracked players. If flagged gems
out-improve the baseline, the detector has predictive value. Heavy (loads
multiple seasons) - a deliberate study action, not part of browsing."""
    flag_season = req.get('season')
    position = req.get('position', 'FW')
    league = req.get('league')
    horizon = int(req.get('horizon', 1))
    if not flag_season:
        return {'error': 'season required'}
    start = _season_start_year(flag_season)
    outcome_season = f"{start + horizon}-{start + horizon + 1}"

    gem_req = {'season': flag_season, 'position': position}
    if league:
        gem_req['league'] = league
    gem_res = cmd_get_hidden_gems(gem_req)
    gems = gem_res.get('gems', [])
    if not gems:
        return {'error': 'no gems flagged in that season/position', 'flag_season': flag_season}
    # Split flagged gems by whether the Riser signal fired - the key
    # comparison for validating Method 7.
    riser_keys = {_norm_key(g['player']) for g in gems if g.get('methods', {}).get('riser')}
    other_keys = {_norm_key(g['player']) for g in gems if not g.get('methods', {}).get('riser')}

    feats = compute_multiseason_features(outcome_season, league, position, lookback=horizon)
    # group -> {'score': [], 'min': [], 'stayed_good': []}
    groups = {'riser': {'s': [], 'm': [], 'g': []},
              'flagged_non_riser': {'s': [], 'm': [], 'g': []},
              'baseline': {'s': [], 'm': [], 'g': []}}
    GOOD = 60.0 # outcome z-score considered "still good"
    for (pkey, _by), f in feats.items():
        by_season = {p['season']: p for p in f['series']}
        if flag_season not in by_season or outcome_season not in by_season:
```



```

        continue
    ds = by_season[outcome_season]['score'] - by_season[flag_season]['score']
    dm = by_season[outcome_season]['minutes'] - by_season[flag_season]['minutes']
    stayed = by_season[outcome_season]['score'] >= GOOD
    g = 'riser' if pkey in riser_keys else 'flagged_non_riser' if pkey in other_keys else
'baseline'
    groups[g]['s'].append(ds); groups[g]['m'].append(dm); groups[g]['g'].append(stayed)

def summ(d):
    s = d['s']
    if not s:
        return {'n': 0}
    return {
        'n': len(s),
        'median_score_delta': round(float(np.median(s)), 1),
        'pct_improved': round(100.0 * float(np.mean([x > 0 for x in s])),),
        'pct_stayed_good': round(100.0 * float(np.mean(d['g']))),
        'median_minutes_delta': int(round(float(np.median(d['m'])))),
    }

return {
    'flag_season': flag_season,
    'outcome_season': outcome_season,
    'position': position,
    'league': league or 'all',
    'good_threshold_z': GOOD,
    'n_flagged_total': len(gems),
    'riser': summ(groups['riser']),
    'flagged_non_riser': summ(groups['flagged_non_riser']),
    'baseline': summ(groups['baseline']),
}

```

`cmd\_train\_mv\_models` (def) - Train + persist the linear/GBM market-value models on real values.

```

def cmd_train_mv_models(req):
    """Train + persist the linear/GBM market-value models on real values."""
    seasons = req.get('seasons')
    return train_mv_models(tuple(seasons)) if seasons else train_mv_models()

```

`cmd\_get\_mv\_model\_info` (def) - Return metadata about the trained MV models (R2, n, learned coefficients).

```

def cmd_get_mv_model_info(req):
    """Return metadata about the trained MV models (R2, n, learned coefficients)."""
    models = _load_mv_models()
    if not models or 'meta' not in models:
        return {'trained': False}
    return {'trained': True, **models['meta']}

```

`cmd\_get\_gem\_trajectory` (def) - On-demand multi-season enrichment for the gems list. Returns a map keyed by player display name -> trajectory features + label.

```

def cmd_get_gem_trajectory(req):
    """On-demand multi-season enrichment for the gems list. Returns a map
    keyed by player display name -> trajectory features + label. Heavy
    (loads several seasons), so it's a separate call from the main gem query
    and cached per (season, league, position)."""
    season = req.get('season')
    position = req.get('position', 'MF')
    league = req.get('league')
    if not season:

```

```

        return {'error': 'season required'}
    lookback = int(req.get('lookback', 3))
    feats = compute_multiseason_features(season, league, position, lookback=lookback)
    out = {}
    for f in feats.values():
        label, summary = _trajectory_label(f)
        out[f['player']] = {
            'seasons_tracked': f['seasons_tracked'],
            'series': f['series'],
            'trajectory_slope': f['trajectory_slope'],
            'consistency_std': f['consistency_std'],
            'breakout': f['breakout'],
            'minutes_slope': f['minutes_slope'],
            'label': label,
            'summary': summary,
        }
    return {'trajectory': out, 'season': season, 'position': position}

```

Command Handlers

`cmd\_get\_leagues` (def)

```

def cmd_get_leagues(req):
    result = {}
    for key, cfg in LEAGUE_MAP.items():
        label = key.replace('-', ' ').title()
        result[key] = {'label': label, 'fbref_id': cfg['fbref_id']}
    return {'leagues': result}

```

`cmd\_get\_teams` (def)

```

def cmd_get_teams(req):
    season = req.get('season')
    league = req.get('league')
    if not season or not league:
        return {'error': 'season and league required'}
    with get_connection() as conn:
        df = pd.read_sql("""
            SELECT DISTINCT team FROM league_season_team_player_data
            WHERE season = %s AND league = %s ORDER BY team
            """, conn, params=[season, league])
    return {'teams': df['team'].tolist()}

```

`cmd\_get\_players` (def)

```

def cmd_get_players(req):
    season = req.get('season')
    league = req.get('league')
    team = req.get('team')
    if not all([season, league, team]):
        return {'error': 'season, league, and team required'}
    df = load_players(season, league, team)
    players = []
    for _, row in df.iterrows():
        sec = row.get('secondary_position')
        players.append({
            'player': row.get('player', ''),
            'position': str(row.get('primary_position', row.get('position', ''))),
            'secondary_position': str(sec) if sec is not None and str(sec) not in ('', 'nan', 'None')
        })
    else ' ',
        'age': parse_age(row.get('age', 0)),
        'minutes': safe_float(row.get('minutes', 0)),

```

```

        'goals': safe_float(row.get('goals', 0)),
        'assists': safe_float(row.get('assists', 0)),
    })
    return {'players': players}

```

`cmd\_get\_player\_profile` (def) - The profile behind the radar: composite index, the z/style/power split, and the per-category scores.

```

def cmd_get_player_profile(req):
    season = req.get('season')
    league = req.get('league')
    team = req.get('team')
    player_name = req.get('player')
    if not all([season, league, team, player_name]):
        return {'error': 'season, league, team, player required'}

    df = load_players(season, league, team)
    player_rows = df[df['player'] == player_name]
    if player_rows.empty:
        return {'error': f'Player {player_name} not found'}

    player = player_rows.iloc[0]
    position = str(player.get('primary_position', player.get('position', 'MF')))
    if position not in ['GK', 'DF', 'MF', 'FW']:
        position = 'MF'

    # Load league comparison data for percentiles
    league_df = load_league_data(season, league)
    pos_df = league_df[league_df['primary_position'] == position] if 'primary_position' in
league_df.columns else league_df

    style_cats = get_playing_style_categories()
    scores = calculate_category_scores(player, pos_df, style_cats, position, method='percentile',
empty_as_none=True)

    # Calculate composite index (legacy formula)
    pos_df_ci = calculate_composite_index(pos_df, position, style_cats)
    # Match the specific club row, not just the name - a player with a
    # mid-season transfer has two rows in this season+league, and a name-only
    # .iloc[0] would hand both stints the same composite.
    player_ci_rows = pos_df_ci[(pos_df_ci['player'] == player_name) & (pos_df_ci['team'] == team)]
    if player_ci_rows.empty:
        player_ci_rows = pos_df_ci[pos_df_ci['player'] == player_name]
    if not player_ci_rows.empty:
        ci_row = player_ci_rows.iloc[0]
        composite = r(safe_float(ci_row.get('composite_index', 0)), 1)
        zscore_comp = r(safe_float(ci_row.get('zscore_comp', 0)), 1)
        style_pctile_avg = r(safe_float(ci_row.get('style_pctile_avg', 0)), 1)
        power_norm = r(safe_float(ci_row.get('power_norm', 0)), 1)
    else:
        _valid = [v for v in scores.values() if v is not None]
        composite = r(float(np.mean(_valid)), 1) if _valid else 0
        zscore_comp = 0
        style_pctile_avg = 0
        power_norm = 0

    # Radar data - omit categories with no measurable data in this league so the
    # radar only plots axes we can actually score (no misleading zeros).
    radar = [{'category': k, 'score': r(v, 1)} for k, v in scores.items() if v is not None]

    # Basic stats
    basic = {
        'player': player_name,

```

```

        'team': team,
        'league': league,
        'season': season,
        'position': position,
        'age': parse_age(player.get('age', 0)),
        'minutes': safe_float(player.get('minutes', 0)),
        'games': safe_float(player.get('games', safe_float(player.get('games_starts', 0)))),
        'goals': safe_float(player.get('goals', 0)),
        'assists': safe_float(player.get('assists', 0)),
        'xg': r(safe_float(player.get('xg', 0)), 2),
        'xg_assist': r(safe_float(player.get('xg_assist', 0)), 2),
    }

    return {
        'profile': basic,
        'radar': radar,
        'composite_index': composite,
        'composite_description': composite_description(composite),
        'zscore_comp': zscore_comp,
        'style_pctile_avg': style_pctile_avg,
        'power_norm': power_norm,
        'category_scores': {k: (r(v, 1) if v is not None else None) for k, v in scores.items()},
    }

```

`POSITION\_LEADER\_METRICS` (module constant)

```

POSITION_LEADER_METRICS = {
    'FW': [('goals_per90', 'Goals /90', False), ('npxg_per90', 'npxG /90', False),
('xg_assist_per90', 'xA /90', False), ('shots_on_target_per90', 'Shots on Target /90', False),
('xg_per90', 'xG /90', False), ('xgot_per90', 'xGOT /90', False)],
    'MF': [('assists_per90', 'Assists /90', False), ('xg_assist_per90', 'xA /90', False),
('xg_per90', 'xG /90', False), ('npxg_per90', 'npxG /90', False), ('shots_on_target_per90', 'Shots on
Target /90', False), ('goals_per90', 'Goals /90', False)],
    'DF': [('tackles', 'Tackles /90', True), ('interceptions', 'Interceptions /90', True),
('tackles_interceptions', 'Tackles + Int /90', True), ('clearances', 'Clearances /90', True),
('ball_recoveries', 'Ball Recoveries /90', True), ('xg_assist_per90', 'xA /90', False)],
    'GK': [('gk_save_pct', 'Save %', False), ('gk_saves', 'Saves /90', True), ('gk_clean_sheets_pct',
'Clean Sheet %', False), ('gk_psxg_net_per90', 'PSxG +/- /90', False), ('gk_clean_sheets', 'Clean
Sheets /90', True), ('gk_goals_against_per90', 'Goals Against /90', False)],
}

```

`\_metric\_leaders` (def) - Per-metric league leaders: for each headline metric, the best value + who holds it, plus the selected player's own value. Leaders must have $\geq$ min_minutes so a cameo can't top a per-90 rate.

```

def _metric_leaders(pos_df, position, player_row, min_minutes=500):
    """Per-metric league leaders: for each headline metric, the best value + who
    holds it, plus the selected player's own value. Leaders must have  $\geq$  min_minutes
    so a cameo can't top a per-90 rate."""
    specs = POSITION_LEADER_METRICS.get(position, [])
    if not specs or pos_df is None or pos_df.empty:
        return []
    mins = pos_df['minutes'].apply(safe_float) if 'minutes' in pos_df.columns else None
    p_min = safe_float(player_row.get('minutes', 0)) if player_row is not None else 0
    out = []
    for col, label, per90 in specs:
        if col not in pos_df.columns:
            continue
        vals = pos_df[col].apply(safe_float)
        series = (vals / mins.replace(0, np.nan) * 90.0) if (per90 and mins is not None) else vals
        if mins is not None:
            series = series.where(mins  $\geq$  min_minutes)

```

```

clean = series.dropna()
if clean.empty:
    continue
best_i = clean.idxmax()
row = {
    'metric': label,
    'best_value': r(float(clean.loc[best_i]), 2),
    'best_player': str(pos_df.loc[best_i, 'player']) if 'player' in pos_df.columns else '',
}
if player_row is not None:
    pv = safe_float(player_row.get(col, 0))
    if per90 and p_min > 0:
        pv = pv / p_min * 90.0
    row['player_value'] = r(pv, 2)
out.append(row)
return out

```

`\_category\_leaders` (def) - Per-category 'league best' envelope: for EACH style category, the highest category score in the pool and the player who owns it (often a different player per category). Percentiles are computed against the full position pool - the same basis as the selected player's radar - but only genuine regulars ($\geq$ min_minutes) can hold a category, so a small-sample outlier can't win one.

```

def _category_leaders(pos_df, position, style_cats, min_minutes=900):
    """Per-category 'league best' envelope: for EACH style category, the highest
    category score in the pool and the player who owns it (often a different
    player per category). Percentiles are computed against the full position pool
    - the same basis as the selected player's radar - but only genuine regulars
    ( $\geq$  min_minutes) can hold a category, so a small-sample outlier can't win one."""
    if pos_df is None or pos_df.empty:
        return []
    elig = pos_df
    if 'minutes' in pos_df.columns:
        f = pos_df[pos_df['minutes'].apply(safe_float) >= min_minutes]
        if len(f) >= 5:
            elig = f
    elig = elig.reset_index(drop=True)
    best = {} # category -> (score, player, team)
    for _, row in elig.iterrows():
        scores = calculate_category_scores(
            row, pos_df, style_cats, position, method='percentile', empty_as_none=True)
        for cat, v in scores.items():
            if v is None:
                continue
            if cat not in best or v > best[cat][0]:
                best[cat] = (v, str(row.get('player', '')), str(row.get('team', '')))
    # preserve the category order the profile radar uses
    out = []
    for cat in style_cats.get(position, {}).keys():
        if cat in best:
            sc, pl, tm = best[cat]
            out.append({'category': cat, 'score': r(sc, 1), 'player': pl, 'team': tm})
    return out

```

`cmd\_get\_position\_benchmark` (def) - Per-metric league leaders + the flat 50 average ring - powers the Day-1 Player Profile radar overlay and the 'vs the league's best' table. Position is taken from the reference player (season/league/team/player) if given, else from an explicit position.

```

def cmd_get_position_benchmark(req):
    """Per-metric league leaders + the flat 50 average ring - powers the Day-1

```

```

Player Profile radar overlay and the 'vs the league's best' table. Position is
taken from the reference player (season/league/team/player) if given, else
from an explicit `position`.""
season = req.get('season')
league = req.get('league')
if not all([season, league]):
    return {'error': 'season, league required'}

position = req.get('position')
team = req.get('team')
player_name = req.get('player')
player_row = None
if team and player_name:
    try:
        df = load_players(season, league, team)
        prows = df[df['player'] == player_name]
        if not prows.empty:
            player_row = prows.iloc[0]
            if not position:
                position = str(player_row.get('primary_position', player_row.get('position',
'MF'))))
    except Exception:
        player_row = None
if position not in ['GK', 'DF', 'MF', 'FW']:
    position = 'MF'

league_df = load_league_data(season, league)
pos_df = league_df[league_df['primary_position'] == position].copy() if 'primary_position' in
league_df.columns else league_df.copy()
if pos_df is None or pos_df.empty:
    return {'error': f'no {position} players for {league} {season}'}
pos_df = pos_df.reset_index(drop=True)

# League best (for the radar overlay). Picked by the z-score aggregate = raw
# per-90 OUTPUT, NOT composite - composite rewards well-roundedness and can
# crown a tidy squad player over an elite scorer. This gives the genuine
# standout performer, with their radar aligned to the profile's categories.
style_cats = get_playing_style_categories()
best_block = None
try:
    pos_df_ci = calculate_composite_index(pos_df.copy(), position, style_cats)
    if 'zscore_comp' in pos_df_ci.columns and not pos_df_ci.empty:
        # Only genuine regulars (>= 900 min ~ 10 full games) can be "the best"
        # so a small-sample per-90 outlier off the bench can't win it.
        elig = pos_df_ci
        if 'minutes' in pos_df_ci.columns:
            filt = pos_df_ci[pos_df_ci['minutes'].apply(safe_float) >= 900]
            if not filt.empty:
                elig = filt
        elig = elig.reset_index(drop=True)
        bi = int(elig['zscore_comp'].fillna(-1).values.argmax())
        brow = elig.iloc[bi]
        bscores = calculate_category_scores(brow, pos_df, style_cats, position,
method='percentile', empty_as_none=True)
        best_block = {
            'player': str(brow.get('player', '')),
            'team': str(brow.get('team', '')),
            'zscore_comp': r(safe_float(brow.get('zscore_comp', 0)), 1),
            'radar': [{ 'category': k, 'score': r(v, 1)} for k, v in bscores.items() if v is not
None],
        }
    except Exception as e:
        print(f"[get_position_benchmark] best failed: {e}", file=sys.stderr, flush=True)

```

```

return {
    'position': position,
    'league': league,
    'season': season,
    'sample_size': int(len(pos_df)),
    'league_average': 50.0, # percentile midpoint (measured ~50.2-50.4 across the pool)
    'best': best_block,
    'category_leaders': _category_leaders(pos_df, position, style_cats),
    'metric_leaders': _metric_leaders(pos_df, position, player_row),
}

```

`cmd\_get\_playing\_style` (def)

```

def cmd_get_playing_style(req):
    season = req.get('season')
    league = req.get('league')
    team = req.get('team')
    player_name = req.get('player')
    if not all([season, league, team, player_name]):
        return {'error': 'season, league, team, player required'}

    df = load_players(season, league, team)
    player_rows = df[df['player'] == player_name]
    if player_rows.empty:
        return {'error': f'Player {player_name} not found'}

    player = player_rows.iloc[0]
    position = str(player.get('primary_position', player.get('position', 'MF')))
    if position not in ['GK', 'DF', 'MF', 'FW']:
        position = 'MF'

    league_df = load_league_data(season, league)
    pos_df = league_df[league_df['primary_position'] == position] if 'primary_position' in
league_df.columns else league_df

    style_cats = get_playing_style_categories()

    # Percentile scores
    pctl_scores = calculate_category_scores(player, pos_df, style_cats, position,
method='percentile', empty_as_none=True)
    # Normalized scores
    norm_scores = calculate_category_scores(player, pos_df, style_cats, position,
method='normalized', empty_as_none=True)

    categories = []
    for cat_name in style_cats.get(position, {}).keys():
        available_metrics = [m for m in style_cats[position][cat_name] if m in pos_df.columns]
        metric_details = []
        for m in available_metrics:
            pv = safe_float(player.get(m, 0)) if isinstance(player, dict) else safe_float(player[m])
            if m in player.index else 0
            comp = pos_df[m].apply(safe_float).dropna()
            # Skip structurally-N/A columns (no variance in the pool, e.g. a stat
            # that is 0 for a whole season) so the breakdown never shows a
            # misleading "0.00" row that carries no information.
            if len(comp) == 0 or comp.min() == comp.max():
                continue
            pctl = calculate_percentile_score(pv, comp.tolist())
            if m in get_negative_metrics():
                pctl = 100 - pctl
            metric_details.append({
                'metric': m,

```

```

        'value': r(pv, 2),
        'percentile': r(pctl, 1),
    })
    ps = pctl_scores.get(cat_name)
    ns = norm_scores.get(cat_name)
    no_data = (ps is None) or (len(metric_details) == 0)
    categories.append({
        'name': cat_name,
        'percentile_score': None if no_data else r(ps, 1),
        'normalized_score': None if (ns is None) else r(ns, 1),
        'metrics': metric_details,
        'no_data': no_data,
    })

return {
    'position': position,
    'categories': categories,
}

```

`\_gem\_display\_stats` (def) - Position-specific stat line for a hidden gem. A keeper is judged on clean sheets and save %, not goals.

```

def _gem_display_stats(row, position):
    """Position-specific stat line for a hidden gem.

    A keeper is judged on clean sheets and save %, not goals. Each
    position gets the stats that actually matter for it, so the UI
    evidence matches the position-specific detection.
    """
    def _missing(key):
        raw = row.get(key)
        return raw is None or (isinstance(raw, float) and pd.isna(raw)) or (
            isinstance(raw, str) and raw.strip().lower() in ('', 'nan', 'none', 'null'))

    def count(key):
        if _missing(key):
            return 'N/A'
        return str(int(round(safe_float(row.get(key, 0)))))

    def dec(key):
        if _missing(key):
            return 'N/A'
        return str(round(safe_float(row.get(key, 0)), 1))

    def pct(key):
        if _missing(key):
            return 'N/A'
        return f"{round(safe_float(row.get(key, 0)), 1)}%"

    games = int(round(safe_float(row.get('games', row.get('games_starts', 0)))))
    minutes = int(round(safe_float(row.get('minutes', 0))))
    tail = [
        {'label': 'Games', 'value': str(games)},
        {'label': 'Minutes', 'value': f"{minutes:,}"},
    ]

    if position == 'GK':
        return [
            {'label': 'Clean Sheets', 'value': str(count('gk_clean_sheets'))},
            {'label': 'Save %', 'value': pct('gk_save_pct')},
            {'label': 'Goals Against', 'value': str(count('gk_goals_against'))},
        ] + tail

    if position == 'DF':
        return [

```



```

        {'label': 'Tackles', 'value': str(count('tackles'))},
        {'label': 'Interceptions', 'value': str(count('interceptions'))},
        {'label': 'Aerials Won %', 'value': pct('aerials_won_pct')},
        {'label': 'Prog. Passes', 'value': str(count('progressive_passes'))},
    ] + tail
if position == 'MF':
    return [
        {'label': 'Key Passes', 'value': str(count('assisted_shots'))},
        {'label': 'xA', 'value': str(dec('xg_assist'))},
        {'label': 'Prog. Passes', 'value': str(count('progressive_passes'))},
        {'label': 'Tackles', 'value': str(count('tackles'))},
    ] + tail
# FW
return [
    {'label': 'Goals', 'value': str(count('goals'))},
    {'label': 'Assists', 'value': str(count('assists'))},
    {'label': 'xG', 'value': str(dec('xg'))},
    {'label': 'Shots', 'value': str(count('shots'))},
] + tail

```

`METRIC\_LABEL` (module constant)

```

METRIC_LABEL = {
    'cards_yellow': 'yellow cards', 'cards_red': 'red cards', 'cards_yellow_red': 'second-yellow
reds',
    'fouls': 'fouls committed', 'fouled': 'times fouled', 'errors': 'errors leading to a shot',
    'own_goals': 'own goals', 'offsides': 'offsides', 'pens_conceded': 'penalties conceded',
    'pens_made': 'penalties scored', 'pens_att': 'penalties taken', 'pens_missed': 'penalties
missed',
    'pens_won': 'penalties won', 'miscontrols': 'miscontrols', 'dispossessed': 'times dispossessed',
    'goals': 'goals', 'goals_per90': 'goals/90', 'goals_pens': 'non-penalty goals',
    'assists': 'assists', 'assists_per90': 'assists/90', 'goals_assists': 'goals + assists',
    'xg': 'xG', 'xg_per90': 'xG/90', 'npxg': 'non-penalty xG', 'npxg_per90': 'non-penalty xG/90',
    'xg_assist': 'xA', 'xg_assist_per90': 'xA/90', 'npxg_per_shot': 'non-penalty xG per shot',
    'shots': 'shots', 'shots_per90': 'shots/90', 'shots_on_target': 'shots on target',
    'shots_on_target_pct': 'shot accuracy', 'goals_per_shot': 'goals per shot',
    'average_shot_distance': 'avg shot distance', 'sca': 'shot-creating actions',
    'sca_per90': 'shot-creating actions/90', 'gca': 'goal-creating actions',
    'gca_per90': 'goal-creating actions/90', 'assisted_shots': 'key passes',
    'through_balls': 'through balls', 'progressive_passes': 'progressive passes',
    'progressive_carries': 'progressive carries', 'progressive_passes_received': 'progressive passes
received',
    'passes_into_final_third': 'passes into the final third', 'passes_into_penalty_area': 'passes
into the box',
    'carries_into_final_third': 'carries into the final third', 'carries_into_penalty_area': 'carries
into the box',
    'tackles': 'tackles', 'tackles_won': 'tackles won', 'interceptions': 'interceptions',
    'blocks': 'blocks', 'blocked_shots': 'shots blocked', 'blocked_passes': 'passes blocked',
    'clearances': 'clearances', 'aerials_won': 'aerial duels won', 'aerials_lost': 'aerial duels
lost',
    'aerials_won_pct': 'aerial win %', 'take_ons': 'take-ons attempted', 'take_ons_won': 'take-ons
completed',
    'take_ons_won_pct': 'dribble success %', 'passes_pct': 'pass completion %',
    'passes_completed': 'passes completed', 'crosses_into_penalty_area': 'crosses into the box',
    'touches_att_pen_area': 'touches in the box', 'ball_recoveries': 'ball recoveries',
    'gk_saves': 'saves', 'gk_save_pct': 'save %', 'gk_clean_sheets': 'clean sheets',
    'gk_clean_sheets_pct': 'clean sheet %', 'gk_goals_against': 'goals conceded',
    'gk_goals_against_per90': 'goals conceded/90', 'gk_psxg_net': 'post-shot xG +/-',
    'gk_pens_saved': 'penalties saved', 'gk_crosses_stopped_pct': 'crosses claimed %',
    # Corner / free-kick / pass-type metrics - these are TYPES, not outcomes.
    # ("corner_kicks_in" = inswinging corners, NOT corners that went in.)
    'corner_kicks': 'corners taken', 'corner_kicks_in': 'inswinging corners',
    'corner_kicks_out': 'outswinging corners', 'corner_kicks_straight': 'straight corners',

```

```

    'passes_free_kicks': 'free-kick passes', 'shots_free_kicks': 'free-kick shots',
    'passes_switches': 'switches of play', 'crosses': 'crosses', 'misc_crosses': 'crosses',
    'passes_dead': 'dead-ball passes', 'passes_live': 'open-play passes',
    'sca_passes_live': 'open-play shot-creating passes', 'sca_passes_dead': 'dead-ball shot-creating
passes',
    'sca_take_ons': 'take-ons leading to a shot', 'sca_shots': 'shots leading to another shot',
    'sca_fouled': 'fouls won leading to a shot', 'gca_passes_live': 'open-play goal-creating passes',
    'gca_passes_dead': 'dead-ball goal-creating passes', 'pass_xa': 'pass-based xA',
    'passes_total_distance': 'total pass distance', 'passes_progressive_distance': 'progressive pass
distance',
    'carries': 'carries', 'carries_distance': 'carry distance', 'carries_progressive_distance':
'progressive carry distance',
}

```

``DESCRIPTOR_METRICS`` (module constant)

```

DESCRIPTOR_METRICS = {
    'corner_kicks', 'corner_kicks_in', 'corner_kicks_out', 'corner_kicks_straight',
    'passes_free_kicks', 'shots_free_kicks', 'passes_switches', 'passes_dead', 'passes_live',
    'crosses', 'misc_crosses', 'passes_total_distance', 'passes_progressive_distance',
    'carries', 'carries_distance', 'carries_progressive_distance', 'touches',
    'touches_def_pen_area', 'touches_def_3rd', 'touches_mid_3rd', 'touches_att_3rd',
    'touches_live_ball', 'passes_received', 'gk_passes', 'gk_passes_throws',
    'gk_passes_launched', 'gk_goal_kicks', 'gk_passes_completed_launched',
    # availability / volume / team-outcome - not individual quality signals
    'minutes', 'gk_minutes', 'minutes_per_game', 'games', 'games_starts',
    'gk_games', 'gk_games_starts', 'gk_wins', 'gk_ties', 'gk_losses',
    'points_per_game', 'on_goals_against', 'on_goals_for',
}

```

``LEAGUE_LABEL`` (module constant)

```

LEAGUE_LABEL = {
    'efl-championship': 'the EFL Championship', 'serie-b': 'Serie B',
    'eredivisie': 'the Eredivisie', 'belgian-pro-league': 'the Belgian Pro League',
    'primeira-liga': 'the Primeira Liga', 'brasileirao': 'the Brasileirão',
    'liga-profesional-argentina': 'Liga Profesional Argentina', 'liga-mx': 'Liga MX',
    'major-league-soccer': 'MLS', 'premier-league': 'the Premier League',
    'la-liga': 'La Liga', 'serie-a': 'Serie A', 'bundesliga': 'the Bundesliga', 'ligue-1': 'Ligue 1',
}

```

``_metric_label`` (def)

```

def _metric_label(m):
    if m in METRIC_LABEL:
        return METRIC_LABEL[m]
    label = m[3:] if m.startswith('gk_') else m
    label = label.replace('_per90', '/90').replace('_pct', ' %').replace('_', ' ')
    return label.strip()

```

``_format_metric_value`` (def)

```

def _format_metric_value(m, raw, per90=False):
    if raw is None or (isinstance(raw, float) and pd.isna(raw)) or (isinstance(raw, str) and
raw.strip().lower() in ('', 'nan', 'none')):
        return None
    try:
        v = float(str(raw).replace(',', ''))
    except (ValueError, TypeError):
        return None
    if m.endswith('_pct') or m == 'aerials_won_pct':
        return f"{v:.1f}%"
    if m.endswith('_per90'):

```

```

        return f"{v:.2f}/90"
# In multi-season mode counting stats are stored as a per-90 rate (see
# _multiseason_pool), so label them as such instead of as a raw count.
if per90 and not _is_rate_metric(m) and m not in _VOLUME_METRICS:
    return f"{v:.2f}/90"
if abs(v - round(v)) < 1e-6:
    return str(int(round(v)))
return f"{v:.1f}"

```

``_league_label` (def)`

```

def _league_label(slug):
    return LEAGUE_LABEL.get(slug, slug.replace('-', ' ').title())

```

``position_pool` (def)` - Rows for players who play OR can play position - primary position match, plus secondary position so versatile players are included.

```

def position_pool(df, position):
    """Rows for players who play OR can play `position` - primary position
    match, plus secondary position so versatile players are included."""
    if 'primary_position' not in df.columns:
        return df.copy()
    mask = df['primary_position'] == position
    if 'secondary_position' in df.columns:
        mask = mask | (df['secondary_position'] == position)
    return df[mask].copy()

```

``SIMILAR_STAT_KEYS` (module constant)`

```

SIMILAR_STAT_KEYS = {
    'GK': [('Clean Sheets', 'gk_clean_sheets'), ('Save %', 'gk_save_pct'), ('Goals Against',
'gk_goals_against')],
    'DF': [('Tackles', 'tackles'), ('Interceptions', 'interceptions'), ('Aerials %',
'aerials_won_pct')],
    'MF': [('Goals', 'goals'), ('xA', 'xg_assist'), ('xG', 'xg')],
    'FW': [('Goals', 'goals'), ('Assists', 'assists'), ('xG', 'xg')],
}

```

``_similar_stats` (def)`

```

def _similar_stats(row, position, per90=False):
    keys = SIMILAR_STAT_KEYS.get(position, SIMILAR_STAT_KEYS['FW'])
    out = []
    for label, k in keys:
        val = _format_metric_value(k, row.get(k), per90=per90)
        out.append({'label': label, 'value': val if val is not None else 'N/A'})
    return out

```

``_all_similar_metric_keys` (def)` - Ordered unique list of a position's style-category metrics present in the data - the full set the Similar Players 'metric' dropdown offers.

```

def _all_similar_metric_keys(position, columns):
    """Ordered unique list of a position's style-category metrics present in the
    data - the full set the Similar Players 'metric' dropdown offers."""
    cats = get_playing_style_categories().get(position, {})
    keys = []
    for metrics in cats.values():
        for m in metrics:
            if m in columns and m not in keys:
                keys.append(m)
    return keys

```

`\_all\_similar\_stats` (def) - Formatted value for every metric, keyed by column name.

```
def _all_similar_stats(row, keys, per90=False):
    """Formatted value for every metric, keyed by column name."""
    out = {}
    for k in keys:
        v = _format_metric_value(k, row.get(k), per90=per90)
        out[k] = v if v is not None else 'N/A'
    return out
```

`\_fmt\_diff` (def) - Signed, tidily-rounded difference for the comparison popup.

```
def _fmt_diff(d):
    """Signed, tidily-rounded difference for the comparison popup."""
    ad = abs(d)
    s = f'{d:.0f}' if ad >= 100 else (f'{d:.1f}' if ad >= 10 else f'{d:.2f}')
    if '.' in s:
        s = s.rstrip('0').rstrip('.')
    return ('+' + s) if d >= 0 and not s.startswith('-') else s
```

`\_compare\_metrics` (def) - Per-metric candidate value, target value, their difference and WHO is better on that metric ('player'=candidate / 'target' / 'tie' / None). "Better" respects metric direction - for negative metrics (lower is better, e.g.

```
def _compare_metrics(cand_row, target_row, keys, per90=False):
    """Per-metric candidate value, target value, their difference and WHO is
    better on that metric ('player'=candidate / 'target' / 'tie' / None). "Better"
    respects metric direction - for negative metrics (lower is better, e.g. avg
    shot distance, disposessions) the sign is flipped. Metrics the candidate has
    no data for come back 'N/A' so the UI can hide them (imputed to the pool mean
    in the model, so no signal)."""
    neg = get_negative_metrics()
    out = {}
    for k in keys:
        c_raw = cand_row.get(k)
        t_raw = target_row.get(k) if target_row is not None else None
        cv = _format_metric_value(k, c_raw, per90=per90)
        tv = _format_metric_value(k, t_raw, per90=per90)
        cnum = _num_series(pd.Series([c_raw]))[0]
        tnum = _num_series(pd.Series([t_raw]))[0]
        diff = None
        better = None
        if not pd.isna(cnum) and not pd.isna(tnum):
            d = float(cnum) - float(tnum)
            diff = _fmt_diff(d)
            if d == 0:
                better = 'tie'
            else:
                cand_better = (d < 0) if k in neg else (d > 0)
                better = 'player' if cand_better else 'target'
        out[k] = {
            'value': cv if cv is not None else 'N/A',
            'target': tv if tv is not None else 'N/A',
            'diff': diff,
            'better': better,
        }
    return out
```

`\_all\_similar\_metric\_groups` (def) - The position's style categories with the metrics present in the data - used to lay out the per-player 'all data points' popup by category.

```
def _all_similar_metric_groups(position, columns):
```

```

"""The position's style categories with the metrics present in the data -
used to lay out the per-player 'all data points' popup by category."""
cats = get_playing_style_categories().get(position, {})
groups = []
for cat, metrics in cats.items():
    present = [m for m in metrics if m in columns]
    if present:
        groups.append({'category': cat, 'metrics': present})
return groups

```

`CATEGORY\_PLAIN` (module constant)

```

CATEGORY_PLAIN = {
    # GK
    'Shot Stopping & Saves': 'shot-stopping',
    'Post-Shot xG & Advanced': 'beating the quality of shots faced (post-shot xG)',
    'Distribution & Passing': 'distribution and passing',
    'Goal Kicks & Long Distribution': 'long distribution',
    'Sweeping & Modern Play': 'sweeping behind the defence',
    'Penalties & Set Pieces': 'penalty and set-piece situations',
    'Expected Goals (xG) Conceded': 'limiting the quality of chances conceded',
    'Command & Presence': 'command of his area and availability',
    # DF
    'Defensive Actions & Tackles': 'tackling',
    'Interceptions & Blocks': 'interceptions and blocks',
    'Aerial Duels & Physical': 'aerial duels',
    'Ball Playing & Passing': 'ball-playing and passing',
    'Progressive Play & Build-Up': 'ball progression from the back',
    'Dribbling & Take-Ons': 'dribbling',
    'Attacking Contribution': 'attacking output (goals and assists)',
    'Expected Goals (xG) & xA': 'chance quality created and taken',
    'Crosses & Set Pieces': 'crossing and set pieces',
    'Touches & Ball Control': 'ball retention',
    'Discipline & Errors': 'discipline (avoiding errors)',
    # MF
    'Creativity & Chance Creation': 'chance creation',
    'Expected Assists (xA)': 'high-quality chance creation (xA)',
    'Passing & Distribution': 'passing and distribution',
    'Final Third & Penetration': 'penetrating final-third passing',
    'Ball Carrying & Progressive Play': 'ball carrying and progression',
    'Goal Threat & Shooting': 'goal threat and shooting',
    'Expected Goals (xG)': 'getting into shooting positions (xG)',
    'Defensive Contribution': 'defensive contribution',
    'Aerial & Physical Duels': 'aerial and physical duels',
    'Touches & Positioning': 'involvement and positioning',
    'Discipline & Game Management': 'discipline and game management',
    # FW
    'Finishing & Clinical': 'clinical finishing',
    'Expected Goals (xG) & Efficiency': 'getting into scoring positions (xG)',
    'Creativity & Assists': 'creativity and assists',
    'Dribbling & 1v1 Skills': '1v1 dribbling',
    'Ball Control & Touch': 'ball control and link play',
    'Progressive Play & Carries': 'carrying the ball into dangerous areas',
    'Aerial & Heading': 'aerial duels and heading',
    'Link-Up & Passing': 'link-up play and passing',
    'Defensive Work': 'pressing and defensive work',
    'Discipline': 'discipline',
}

```

`POSITION\_WORD` (module constant)

```

POSITION_WORD = {'GK': 'goalkeeper', 'DF': 'defender', 'MF': 'midfielder', 'FW': 'forward'}

```

`\_ordinal` (def)

```
def _ordinal(n):
    n = int(round(n))
    if 10 <= (n % 100) <= 20:
        suffix = 'th'
    else:
        suffix = {1: 'st', 2: 'nd', 3: 'rd'}.get(n % 10, 'th')
    return f"{n}{suffix}"
```

`\_season\_short` (def)

```
def _season_short(season):
    s = str(season)
    if '-' in s:
        a, b = s.split('-', 1)
        return f"{a}-{b[-2:]}" if len(b) >= 2 else s
    return s
```

`build\_verdict` (def) - Plain-English scouting verdict assembled from the computed signals.

```
def build_verdict(player, position, age, team, season, league, methods_triggered,
                  top_cats, bottom_cats, value_ratio, wage_pctl,
                  mv_label, mv_estimated, contract_months, anomaly,
                  present_cats, total_cats, goals, xg_missing, shots_missing, minutes,
                  traj=None, release_clause=0.0, assists=0.0):
    """Plain-English scouting verdict assembled from the computed signals."""
    pos_word = POSITION_WORD.get(position, 'player')

    if methods_triggered >= 5:
        tier, headline = 'Strong Signal', 'A priority target - act quickly.'
    elif methods_triggered == 4:
        tier, headline = 'Good Signal', 'Worth a closer look.'
    elif methods_triggered == 3:
        tier, headline = 'Worth Monitoring', 'Keep on the watchlist.'
    else:
        tier, headline = 'Weak Signal', 'Track, but no rush.'

    summary = (f"{player} is a {age}-year-old {pos_word} at {team} ({_season_short(season)}), "
              f"flagged by {methods_triggered} of 7 detection methods.")

    def driver_str(c):
        drivers = c.get('drivers') or []
        if not drivers:
            return ''
        return ' - ' + ', '.join(f"{d['label']}: {d['value']}" for d in drivers)

    def strength_text(c):
        p = c['percentile']
        plain = CATEGORY_PLAIN.get(c['category'], c['category'].lower())
        band = 'Elite' if p >= 85 else 'Strong' if p >= 70 else 'Solid' if p >= 55 else 'Around
average'
        return f"{band} at {plain} ({_ordinal(p)} percentile){driver_str(c)}."

    def weakness_text(c):
        p = c['percentile']
        plain = CATEGORY_PLAIN.get(c['category'], c['category'].lower())
        band = 'Weak' if p <= 25 else 'Below par' if p <= 40 else 'Room to improve'
        return f"{band} at {plain} ({_ordinal(p)} percentile){driver_str(c)}."

    strengths = [{'category': c['category'], 'percentile': c['percentile'], 'drivers':
c.get('drivers', []), 'text': strength_text(c)} for c in top_cats]
```

```

weaknesses = [{'category': c['category'], 'percentile': c['percentile'], 'drivers':
c.get('drivers', [])}, {'text': weakness_text(c)} for c in bottom_cats]

# Data-coverage honesty: when most style categories have no data
# (e.g. a forward in a league with no shot/xG data), say so plainly
# and temper the headline rather than calling a data-starved player elite.
coverage = (present_cats / total_cats) if total_cats else 0.0
data_note = None
if coverage < 0.5:
    missing_bits = []
    if xg_missing:
        missing_bits.append('xG')
    if shots_missing:
        missing_bits.append('shots')
    missing_str = ' and '.join(missing_bits) if missing_bits else 'several advanced metrics'
    note = (f"Heads up: {missing_str} read zero here because {_league_label(league)} doesn't
carry "
           f"advanced shot data - those aren't real zeros. ")
    if goals and goals > 0:
        note += f"He still scored {int(goals)} in {int(minutes)} minutes, so the flag rests on
basic output plus league strength, "
    else:
        note += "so the flag rests on basic output plus league strength, "
        note += (f"with only {present_cats} of {total_cats} style categories carrying enough data to
score. "
                f"Lean on video over the model here, and treat the headline tier with caution.")
    data_note = note
    headline = 'Promising on basic stats - but limited data to confirm.'

wage_band = 'bottom 30%' if wage_pctl <= 30 else 'lower half' if wage_pctl <= 50 else 'upper
half'
if value_ratio > 120:
    assess = 'exceptional value for money'
elif value_ratio > 80:
    assess = 'strong value'
elif value_ratio > 60:
    assess = 'good value'
else:
    assess = 'fair value'
value_note = (f"Estimated {mv_label}, with a wage in the {wage_band} for his position"
              f" - value ratio {value_ratio:.0f}, {assess}.")

contract_note = None
if contract_months is not None:
    if contract_months <= 0:
        cn = 'is out of contract - available as a free agent.'
    elif contract_months <= 6:
        cn = 'has under 6 months left - could be signed cheaply or pre-agreed.'
    elif contract_months <= 12:
        cn = 'is in the final year of his contract - negotiate before he can leave on a free.'
    elif contract_months <= 18:
        cn = 'has 18 months or less remaining - a clear window to sign before the price climbs.'
    elif contract_months <= 24:
        cn = 'has around two years left - still time to negotiate from strength.'
    else:
        cn = 'has a long contract remaining - any deal would command a premium.'
    contract_note = f"{age} years old and {cn}"

# Age horizon + profile type - a hidden gem can be a development
# prospect (young, future resale) or a short-term value play (a cheap
# veteran still performing). Make the value window explicit by age.
if age <= 20:
    age_horizon = 'Long runway - primary value is development and future resale.'

```

```

        profile_type = 'Development Prospect'
    elif age <= 23:
        age_horizon = 'Prime potential - peak upside and resale value (the classic hidden-gem age).'
        profile_type = 'Development Prospect'
    elif age <= 26:
        age_horizon = 'Approaching prime - immediate quality with solid resale value.'
        profile_type = 'Prime Performer'
    elif age <= 29:
        age_horizon = 'In his prime - buy for impact; resale window still decent.'
        profile_type = 'Prime Performer'
    elif age <= 32:
        age_horizon = 'Experienced - immediate impact, but a 2-3 year window and fading resale.'
        profile_type = 'Short-Term Value'
    else:
        age_horizon = 'Veteran - short-term squad value only; minimal resale, expect 1-2 useful
seasons.'
        profile_type = 'Short-Term Value'

    veteran = age >= 30
    weak_area = CATEGORY_PLAIN.get(bottom_cats[0]['category'], 'his weaker areas') if bottom_cats
else 'his weaker areas'
    if methods_triggered >= 4:
        if veteran:
            recommendation = (f"Request video footage focused on {weak_area}. A low-cost, ready-now
option for "
                                f"immediate squad depth rather than a long-term project - pursue only
if the short-term value fits the plan.")
        else:
            recommendation = (f"Request video footage focused on {weak_area}, and schedule a live
scout "
                                f"for his next 3 matches.")
    elif methods_triggered == 3:
        if veteran:
            recommendation = 'Shortlist as a short-term value / squad-depth option; review recent
highlights before committing.'
        else:
            recommendation = 'Add to the shortlist and review highlights over the next few matches.'
    else:
        recommendation = 'Log for tracking; revisit if his minutes or output rise.'

# Output framing (data-derived): goals/assists over the minutes played.
output_note = None
if minutes > 0:
    per90 = 90.0 / minutes
    ga = (goals + assists)
    output_note = (f"Output: {int(goals)} goals, {int(assists)} assists in {int(minutes)} minutes
"
                    f"({ga * per90:.2f} goal involvements per 90).")

# Trajectory signal (multi-season), if we have it.
trajectory_note = None
if traj and traj.get('seasons_tracked', 0) >= 2:
    _lab, _summ = _trajectory_label(traj)
    trajectory_note = f"Trajectory signal: {_summ}"
    if traj.get('minutes_slope', 0) > 50:
        trajectory_note += ' Minutes are trending up - the club is leaning on him more.'
    elif traj.get('minutes_slope', 0) < -50:
        trajectory_note += ' Minutes are trending down.'

# Release-clause framing - what the parent club thinks vs the open market.
clause_note = None
if release_clause and release_clause > 0:
    clause_note = (f"Release clause {format_eur(release_clause)} - the parent club's ceiling, "

```



```

        f"against an estimated market value of {mv_label}. A wide gap is itself a
signal.")

    caveats = []
    if mv_estimated:
        caveats.append('Market value is estimated, not an actual transfer figure.')
    if minutes and minutes < 900:
        caveats.append(f'Small sample ({int(minutes)} min) - per-90 rates are noisy; weight career
history and video.')
    if anomaly:
        caveats.append('Elite across multiple categories - strong fit for a specialist role.')

    return {
        'tier': tier,
        'headline': headline,
        'summary': summary,
        'profile_type': profile_type,
        'age_horizon': age_horizon,
        'output_note': output_note,
        'trajectory_note': trajectory_note,
        'clause_note': clause_note,
        'strengths': strengths,
        'weaknesses': weaknesses,
        'value_note': value_note,
        'contract_note': contract_note,
        'recommendation': recommendation,
        'caveats': caveats,
        'data_note': data_note,
        'coverage': round(coverage, 2),
    }

```

`cmd\_get\_hidden\_gems` (def) - Screens a whole position: minimum-minutes filter, composite, then seven detection signals per player (percentile outlier, z-score, value ratio, age-weighted potential, moneyball, statistical anomaly, riser). A player is a gem only if ≥ 2 signals fire, at least one is a value/upside signal, and they are not already expensive.

```

def cmd_get_hidden_gems(req):
    season = req.get('season')
    league = req.get('league')
    leagues = req.get('leagues') # optional list of league slugs (multi-select)
    position = req.get('position', 'MF')
    use_trajectory = req.get('use_trajectory', True)
    mv_method = 'heuristic' # heuristic-only valuation (single, readable formula)
    # Analyst-set ceiling: any player whose market value (real OR estimated via
    # the selected method) is at/above this can't be a "hidden gem". Default 40M.
    try:
        mv_ceiling = float(req.get('mv_ceiling', 40_000_000)) or 40_000_000
    except (TypeError, ValueError):
        mv_ceiling = 40_000_000
    if not season:
        return {'error': 'season required'}

    if leagues:
        df = load_all_leagues_data(season)
        if not df.empty and 'league' in df.columns:
            df = df[df['league'].isin(leagues)].copy()
    elif league:
        df = load_league_data(season, league)
    else:
        df = load_all_leagues_data(season)

    if df.empty:

```

```

        return {'gems': [], 'total': 0}

# Merge supplementary data (wages, market values, contracts)
df = merge_supplementary(df, season)

pos_df = df[df['primary_position'] == position].copy() if 'primary_position' in df.columns else
df.copy()
if pos_df.empty:
    return {'gems': [], 'total': 0}

# Minimum-minutes filter - small samples produce noise (composite
# inflated by NaN handling, z-scores blown up by tiny std, value
# ratios exploded by floor-level wage estimates). Drop them before
# any percentile/z math runs so the comparison pool is meaningful.
min_minutes_for_gem = {'GK': 270, 'DF': 450, 'MF': 450, 'FW': 450}.get(position, 450)
if 'minutes' in pos_df.columns:
    mins_col = _num_series(pos_df['minutes']).fillna(0)
    pos_df = pos_df[mins_col >= min_minutes_for_gem].copy()
if pos_df.empty:
    return {'gems': [], 'total': 0, 'min_minutes': min_minutes_for_gem}

style_cats = get_playing_style_categories()
neg_metrics = get_negative_metrics()

# Calculate composite index vectorized (legacy formula)
pos_df = calculate_composite_index(pos_df, position, style_cats)

# Multi-season trajectory (cached). Powers Method 7 (Riser). Uses only
# seasons up to `season`, so it's leak-free for backtesting. For a
# multi-league subset we track across all leagues (identity is by name +
# birth_year, so it spans leagues anyway).
traj_league = None if leagues else league
traj_feats = compute_multiseason_features(season, traj_league, position) if use_trajectory else
{}

# Pre-compute wages for percentile
wages = pos_df.apply(lambda r2: get_wage_value(r2.to_dict())[0], axis=1)

# Composite z-score stats (for z-score detection method)
ci_mean = pos_df['composite_index'].mean()
ci_std = pos_df['composite_index'].std()
position_pool_size = int(len(pos_df))

# Pre-compute per-player category percentile ranks for Method 6 (Statistical Anomaly)
# Coverage-aware: players missing >50% of a category's metrics get
# NaN for that category, so they can't trigger an anomaly on hollow data.
position_cats = style_cats.get(position, {})
cat_pctile_cols = {}
cat_metric_pctiles = {} # {category: {metric: directional pctile Series}} - for driver callouts
total_cats = 0
for cat_name, cat_metrics in position_cats.items():
    avail = [m for m in cat_metrics if m in pos_df.columns]
    if not avail:
        continue
    total_cats += 1
    cat_pctile_sum = pd.Series(0.0, index=pos_df.index)
    cat_pctile_present = pd.Series(0, index=pos_df.index)
    metric_pcts = {}
    for m in avail:
        col = _num_series(pos_df[m])
        pctile = col.rank(pct=True, na_option='keep') * 100
        if m in neg_metrics:
            pctile = 100 - pctile

```

```

        non_null = ~pctile.isna()
        cat_pctile_sum += pctile.fillna(0)
        cat_pctile_present += non_null.astype(int)
        metric_pcts[m] = pctile
    cat_metric_pctiles[cat_name] = metric_pcts
    coverage_threshold = max(1, len(avail) // 2)
    cat_avg = cat_pctile_sum / cat_pctile_present.replace(0, 1)
    cat_pctile_cols[cat_name] = cat_avg.where(cat_pctile_present >= coverage_threshold, np.nan)

```

```

def _category_drivers(cat_name, idx, want_low):
    """Top 2 metrics driving a category up (strength) or down (weakness)."""
    metric_pcts = cat_metric_pctiles.get(cat_name, {})
    items = []
    for m, pser in metric_pcts.items():
        if m in DESCRIPTOR_METRICS:
            continue
        v = pser.get(idx, np.nan)
        if pd.notna(v):
            items.append((m, float(v)))
    items.sort(key=lambda x: x[1], reverse=not want_low)
    drivers = []
    seen_labels = set()
    for m, _pct in items:
        if len(drivers) >= 2:
            break
        label = _metric_label(m)
        if label in seen_labels:
            continue
        val = _format_metric_value(m, pos_df.at[idx, m])
        if val is not None:
            seen_labels.add(label)
            drivers.append({'label': label, 'value': val})
    return drivers

```

```

results = []
for idx, row in pos_df.iterrows():
    composite = safe_float(row.get('composite_index', 0))
    age = parse_age(row.get('age', 25))
    mv, mv_est = get_market_value(row.to_dict(), method=mv_method)
    wage, wage_est = get_wage_value(row.to_dict())

```

6 detection methods

1. Percentile Outlier: composite > 80 (top ~10%, Elite tier).

Recalibrated up from 70 after the z-norm switched to percentile rank

(composites are no longer squashed, so 70 now caught ~30% of players).

pctl_outlier = composite > 80

2. Z-Score: composite z-score > 1.5

z_score = (composite - ci_mean) / ci_std if ci_std > 0 else 0

z_outlier = bool(z_score > 1.5)

3. Value Ratio: composite / wage_percentile.

Only valid on REAL wage data - an estimated wage hits the floor,

which would falsely flag every no-data player as a value gem and

dominate the list. Don't fire (and don't reward in Moneyball) when

the wage is estimated; we simply don't know if they're underpaid.

wage_pctl = stats.percentileofscore(wages.dropna(), wage, kind='rank') if len(wages) > 1 else

value_ratio = (composite / max(wage_pctl, 1)) * 50

value_gem = bool(value_ratio > 60) and not wage_est

4. Age-Weighted Potential: young + high composite

age_potential = composite * (1.3 if age < 23 else 1.1 if age < 26 else 0.9 if age > 30 else

age_gem = age < 24 and age_potential > 85

5. Composite Score: all factors combined

50

1.0)

```

contract_score = calculate_contract_opportunity_score(row.to_dict())
value_eff = min(100, value_ratio) if not wage_est else 50.0
moneyball = calculate_moneyball(composite, value_eff, contract_score)
composite_gem = moneyball > 65
# 6. Statistical Anomaly: 2+ style categories above 90th percentile
top_categories = sum(1 for cn, cs in cat_pctile_cols.items() if cs.get(idx, 0) > 90)
anomaly = top_categories >= 2
# 7. Riser: improving across seasons (forward-looking). Backtesting
# showed the other 6 methods favour peak players who regress to the
# mean; the Riser signal surfaces players still on the way up.
_by = int(safe_float(row.get('birth_year', 0)))
_tf = traj_feats.get((_norm_key(str(row.get('player', ''))), _by)) if _by > 0 else None
riser = bool(_tf and _trajectory_label(_tf)[0] == 'Rising')

gem_methods = sum([pctl_outlier, z_outlier, value_gem, age_gem, composite_gem, anomaly,
riser])

# A hidden gem must be UNDERVALUED, not merely elite. The performance
# methods (percentile, z-score, anomaly, riser) fire for any top player
# - including superstars like Mbappé/Haaland who are maximally valued.
# Require at least one value/upside signal (cheap relative to output,
# young with upside, or strong moneyball), and exclude anyone already
# carrying a high REAL market value - they can't be "hidden".
value_signal = value_gem or age_gem or composite_gem
# mv here is the real value if we have one, otherwise the heuristic
# estimate. The ceiling applies to both.
already_expensive = (mv >= mv_ceiling) or ((not wage_est) and wage_pctl >= 85)

if gem_methods >= 2 and value_signal and not already_expensive:
    # Per-player category percentiles (skip NaN = insufficient coverage)
    player_cats = []
    for cn, cs in cat_pctile_cols.items():
        v = cs.get(idx, np.nan)
        if pd.notna(v):
            player_cats.append({'category': cn, 'percentile': int(round(float(v)))})

    # Coverage gate: a player we have no usable performance data on
    # cannot be evaluated as a gem. With zero scoring categories the
    # composite is propped up only by league power + a neutral fallback,
    # which is meaningless - exclude them outright rather than flag noise.
    if not player_cats:
        continue

    player_cats.sort(key=lambda c: c['percentile'], reverse=True)
    top_cats_list = player_cats[:3]
    bottom_cats_list = list(reversed(player_cats[-3:])) if len(player_cats) >= 3 else
list(reversed(player_cats))
    anomaly_cats_list = [c for c in player_cats if c['percentile'] > 90]
    for c in top_cats_list:
        c['drivers'] = _category_drivers(c['category'], idx, want_low=False)
    for c in bottom_cats_list:
        c['drivers'] = _category_drivers(c['category'], idx, want_low=True)

    present_cats = len(player_cats)
    goals_val = safe_float(row.get('goals', 0))

    def _raw_missing(key):
        raw = row.get(key)
        return raw is None or (isinstance(raw, float) and pd.isna(raw))
    xg_missing = _raw_missing('xg')
    shots_missing = _raw_missing('shots')

    contract_months = contract_months_remaining(row.to_dict())

```

```

verdict = build_verdict(
    str(row.get('player', '')), position, age, str(row.get('team', '')),
    season, str(row.get('league', '')), gem_methods,
    top_cats_list, bottom_cats_list, value_ratio, wage_pctl,
    format_eur(mv), mv_est, contract_months, anomaly,
    present_cats, total_cats, goals_val, xg_missing, shots_missing,
    safe_float(row.get('minutes', 0)),
    traj=_tf, release_clause=safe_float(row.get('release_clause_eur', 0)),
    assists=safe_float(row.get('assists', 0)),
)

# Output-per-EUR-M: production (xG+xA) per million of market value - the
# model-free 'value residual' quick check (higher = more output per euro).
_out = (safe_float(row.get('xg', 0)) or 0.0) + (safe_float(row.get('xg_assist', 0)) or
0.0)

_mvm = (mv or 0) / 1e6
output_per_mv = round(_out / _mvm, 2) if (_mvm >= 0.5 and _out > 0) else None
# Model's performance-predicted price + the value residual (actual -
# predicted, EUR M). Residual only meaningful against a REAL price;
# negative = market underpays for his output = a gem.
_pred = predict_mv_from_performance(row.to_dict(), mv_method)
predicted_value = r(_pred, 0) if (_pred and _pred > 0) else None
predicted_value_label = format_eur(_pred) if (_pred and _pred > 0) else None
value_residual = None
if not mv_est and mv and mv > 0 and _pred and _pred > 0:
    value_residual = round((mv - _pred) / 1e6, 1)
results.append({
    'output_per_mv': output_per_mv,
    'value_residual': value_residual,
    'predicted_value': predicted_value,
    'predicted_value_label': predicted_value_label,
    'player': row.get('player', ''),
    'team': str(row.get('team', '')),
    'league': str(row.get('league', '')),
    'age': age,
    'position': position,
    'composite': r(composite, 1),
    'market_value': r(mv, 0),
    'market_value_label': format_eur(mv),
    'mv_estimated': mv_est,
    'wage': r(wage, 0),
    'wage_label': format_eur(wage) + '/wk',
    'wage_estimated': wage_est,
    'moneyball_score': r(moneyball, 1),
    'z_score': r(z_score, 2),
    'value_ratio': r(value_ratio, 1),
    'age_potential': r(age_potential, 1),
    'methods_triggered': gem_methods,
    'methods': {
        'percentile_outlier': pctl_outlier,
        'z_score_outlier': z_outlier,
        'value_ratio': value_gem,
        'age_weighted': age_gem,
        'composite_score': composite_gem,
        'statistical_anomaly': anomaly,
        'riser': riser,
    },
    'goals': safe_float(row.get('goals', 0)),
    'assists': safe_float(row.get('assists', 0)),
    'minutes': safe_float(row.get('minutes', 0)),
    'games': int(round(safe_float(row.get('games', row.get('games_starts', 0))))),
    'games_starts': int(round(safe_float(row.get('games_starts', 0)))),
    'display_stats': _gem_display_stats(row, position),

```

```

        'composite_components': {
            'z_aggregate': safe_float(row.get('zscore_comp', 0)),
            'style_pctile': safe_float(row.get('style_pctile_avg', 0)),
            'league_power': safe_float(row.get('power_norm', 0)),
        },
        'top_categories': top_cats_list,
        'bottom_categories': bottom_cats_list,
        'anomaly_categories': anomaly_cats_list,
        'position_pool_size': position_pool_size,
        'position_mean_composite': r(ci_mean, 1),
        'position_std_composite': r(ci_std, 2),
        'wage_percentile': r(wage_pctl, 1),
        'verdict': verdict,
    })

results.sort(key=lambda x: x['moneyball_score'], reverse=True)
return {'gems': results[:50], 'total': len(results), 'min_minutes': min_minutes_for_gem,
        'season': season, 'mv_method': mv_method}

```

Multi-criteria decision analysis: AHP weights + TOPSIS ranking + sensitivity

`\_AHP\_PRIORITY` (module constant)

```

_AHP_PRIORITY = {
    'GK': {'Shot Stopping & Saves': 9, 'Post-Shot xG & Advanced': 6, 'Command & Presence': 4,
           'Distribution & Passing': 2},
    'DF': {'Defensive Actions & Tackles': 9, 'Interceptions & Blocks': 8, 'Ball Playing & Passing':
6,
           'Discipline & Errors': 5, 'Attacking Contribution': 3, 'Expected Goals (xG) & xA': 3,
           'Dribbling & Take-Ons': 2, 'Crosses & Set Pieces': 2},
    'MF': {'Passing & Distribution': 8, 'Creativity & Chance Creation': 8, 'Expected Assists (xA)':
7,
           'Defensive Contribution': 6, 'Goal Threat & Shooting': 4, 'Expected Goals (xG)': 4,
           'Dribbling & Take-Ons': 4, 'Discipline & Game Management': 3},
    'FW': {'Finishing & Clinical': 9, 'Expected Goals (xG) & Efficiency': 8, 'Creativity & Assists':
5,
           'Expected Assists (xA)': 5, 'Dribbling & 1v1 Skills': 5, 'Link-Up & Passing': 4,
           'Ball Control & Touch': 4, 'Penalties & Set Pieces': 3, 'Defensive Work': 2, 'Discipline':
3},
}

```

`\_AHP\_RI` (module constant)

```

_AHP_RI = {1: 0.0, 2: 0.0, 3: 0.58, 4: 0.90, 5: 1.12, 6: 1.24, 7: 1.32, 8: 1.41,
           9: 1.45, 10: 1.49, 11: 1.51, 12: 1.54}

```

`\_saaty\_round` (def) - Snaps a priority ratio to Saaty's 1-9 integer scale (or its reciprocal) for the AHP pairwise matrix.

```

def _saaty_round(r):
    """Snap a priority ratio to the Saaty 1-9 integer scale (or its reciprocal)."""
    return float(min(9, max(1, round(r)))) if r >= 1 else 1.0 / min(9, max(1, round(1.0 / r)))

```

`\_ahp\_weights` (def) - AHP: turns a position's category priorities into a consistent weight vector via the pairwise matrix eigenvector, and reports the consistency ratio (CR = CI/RI; CR < 0.1 means the priorities aren't self-contradictory).

```

def _ahp_weights(categories, position):
    """AHP eigenvector weights + consistency ratio for a position's criteria.
    Returns (weights, CR, meta) where meta carries the consistency workings."""
    pr = [_AHP_PRIORITY.get(position, {}).get(c, 3) for c in categories]

```

```

n = len(pr)
if n == 0:
    return np.array([]), 0.0, {'lambda_max': 0.0, 'ci': 0.0, 'ri': 0.0, 'n': 0}
P = np.ones((n, n))
for i in range(n):
    for j in range(n):
        if i != j:
            P[i, j] = _saaty_round(pr[i] / pr[j])
w = (P / P.sum(0)).mean(1)
w = w / w.sum()
ri = _AHP_RI.get(n, 1.54)
if n > 2:
    lmax = float((P @ w / w).mean())
    ci = (lmax - n) / (n - 1)
    CR = ci / (ri or 1.0)
else:
    lmax, ci, CR = float(n), 0.0, 0.0
return w, CR, {'lambda_max': round(lmax, 3), 'ci': round(ci, 3),
              'ri': round(float(ri), 3), 'n': n}

```

`\_topsis` (def) - TOPSIS: weighted, vector-normalised criteria; closeness = D-minus / (D-plus + D-minus), where D-plus/D-minus are distances to the ideal / anti-ideal. 1 = at the ideal.

```

def _topsis(X, w):
    """TOPSIS closeness coefficient per alternative (row); all criteria benefit-type."""
    X = np.asarray(X, float)
    denom = np.sqrt((X ** 2).sum(0))
    denom[denom == 0] = 1.0
    V = (X / denom) * w
    Ap, An = V.max(0), V.min(0)
    Dp = np.sqrt(((V - Ap) ** 2).sum(1))
    Dn = np.sqrt(((V - An) ** 2).sum(1))
    tot = Dp + Dn
    tot[tot == 0] = 1.0
    return Dn / tot

```

`\_mcdm\_ranks` (def) - Rank alternatives by descending closeness.

```

def _mcdm_ranks(C):
    order = np.argsort(-C)
    rank = np.empty(len(C), dtype=int)
    for p, i in enumerate(order):
        rank[i] = p + 1
    return rank

```

`\_mcdm\_sensitivity` (def) - Re-runs TOPSIS with each criterion weight boosted +50% in turn, recording each alternative's best and worst rank, so the lab can say whether a match is robust.

```

def _mcdm_sensitivity(X, w):
    """Rank low/high for each alternative as each criterion weight is boosted +50%."""
    lo = hi = _mcdm_ranks(_topsis(X, w))
    for k in range(len(w)):
        w2 = w.copy()
        w2[k] *= 1.5
        w2 = w2 / w2.sum()
        rk = _mcdm_ranks(_topsis(X, w2))
        lo = np.minimum(lo, rk)
        hi = np.maximum(hi, rk)
    return lo, hi

```

`cmd\_get\_similar\_players` (def) - Builds the similar-players shortlist (distance-based), then runs the AHP + TOPSIS + sensitivity analysis over it and returns the closeness / rank / robustness for the target and every candidate.

```
def cmd_get_similar_players(req):
    season = req.get('season')
    league = req.get('league')
    team = req.get('team')
    player_name = req.get('player')
    method = req.get('method', 'cosine')
    top_n = req.get('top_n', 10)
    if not all([season, league, team, player_name]):
        return {'error': 'season, league, team, player required'}

    df = load_players(season, league, team)
    player_rows = df[df['player'] == player_name]
    if player_rows.empty:
        return {'error': f'Player {player_name} not found'}

    player = player_rows.iloc[0]
    position = str(player.get('primary_position', player.get('position', 'MF')))
    if position not in ['GK', 'DF', 'MF', 'FW']:
        position = 'MF'

    # Minutes filter: an unfiltered pool is full of cup cameos and end-of-bench
    # players whose per-90s are noise and whose counting stats are 0/null. They
    # show up as spurious "perfect" matches (a 40-minute CB has a flat, empty
    # profile cosine loves). Require a real sample so comparisons are credible.
    min_minutes = req.get('min_minutes', 600)
    # window > 1 -> multi-season "average level" matching; 1 -> single season.
    window = max(1, min(5, int(req.get('window', 1) or 1)))

    if window > 1:
        # Multi-season: blend each player's last `window` seasons into one
        # minutes-weighted row, then run the identical similarity pipeline.
        agg = _multiseason_pool(season, position, window)
        if agg is None or agg.empty:
            return {'similar': [], 'error': 'No multi-season data available'}
        tkey = _norm_key(player_name)
        tby = int(safe_float(player.get('birth_year', 0)))
        tmask = (agg['_pkey'] == tkey) & (agg['_by'] == tby)
        if not tmask.any():
            tmask = agg['_pkey'] == tkey # fall back to name-only identity
        if not tmask.any():
            return {'similar': [], 'error': 'Target not found across these seasons'}
        player_ref = agg[tmask].iloc[0]
        pos_df = agg[~tmask.values].copy()
        if 'minutes' in pos_df.columns and min_minutes:
            filtered = pos_df[pos_df['minutes'] >= min_minutes]
            if len(filtered) >= 10:
                pos_df = filtered
        pos_df = pos_df.reset_index(drop=True)
    else:
        # Single season. Pool = players who play OR can play this position.
        all_df = load_all_leagues_data(season)
        pos_df = position_pool(all_df, position)
        pos_df = pos_df[~((pos_df['player'] == player_name) & (pos_df['team'] == team))]
        if pos_df.empty:
            return {'similar': []}
        if 'minutes' in pos_df.columns and min_minutes:
            mins = _num_series(pos_df['minutes']).fillna(0)
            filtered = pos_df[mins >= min_minutes]
            if len(filtered) >= 10:
```



```

        pos_df = filtered
    pos_df = pos_df.reset_index(drop=True)
    player_ref = player

style_cats = get_playing_style_categories()
all_metrics = []
for cat_metrics in style_cats.get(position, {}).values():
    all_metrics.extend(cat_metrics)
all_metrics = list(set(all_metrics))

available = [m for m in all_metrics if m in pos_df.columns and m in player_ref.index]
if len(available) < 3:
    return {'similar': [], 'error': 'Not enough metrics available'}

# Numeric matrix with real NaNs (safe_float would hide missingness as 0).
num = pos_df[available].apply(_num_series)

# Drop metrics with NO data anywhere in the pool (e.g. xGOT in seasons before
# it was collected - all-NaN in 2023-24 and earlier). An all-NaN column has a
# NaN mean, which poisons the target's imputed vector and crashes the distance
# step with "Input contains NaN". Comparing on a metric no one has is meaningless.
nonempty = [mtr for mtr in available if bool(num[mtr].notna().any())]
if len(nonempty) < 3:
    return {'similar': [], 'error': 'Not enough metrics with data for this season'}
if len(nonempty) < len(available):
    available = nonempty
    num = num[available]

# Coverage guard: drop players missing more than half their metrics - their
# profile is too thin to compare and would be dominated by imputed values.
coverage = num.notna().mean(axis=1)
keep = coverage >= 0.5
if keep.sum() >= 10:
    pos_df = pos_df[keep.values].reset_index(drop=True)
    num = num[keep.values].reset_index(drop=True)

# Impute the remaining gaps with the pool mean (neutral - lands at 0 after
# scaling) instead of 0 (which scaling turns into an artificial extreme).
col_means = num.mean()
comp_matrix = num.fillna(col_means).fillna(0)

# Player values - impute the target's own gaps with the same pool means.
player_vals = []
for m in available:
    tv = _num_series(pd.Series([player_ref[m]]))[0]
    player_vals.append(float(tv) if not pd.isna(tv) else float(col_means.get(m, 0)))

# Invert negative metrics
neg = get_negative_metrics()
for i, m in enumerate(available):
    if m in neg:
        max_val = max(comp_matrix[m].max(), player_vals[i], 1)
        comp_matrix[m] = max_val - comp_matrix[m]
        player_vals[i] = max_val - player_vals[i]

# Belt-and-suspenders: no NaN/inf may reach the distance functions, whatever
# imputation left behind (a still-empty target metric, an inf from inversion).
player_arr = np.nan_to_num(np.array(player_vals, dtype=float).reshape(1, -1),
                           nan=0.0, posinf=0.0, neginf=0.0)
comp_arr = np.nan_to_num(comp_matrix.values.astype(float),
                           nan=0.0, posinf=0.0, neginf=0.0)

# Scale

```

```

try:
    scaler = StandardScaler()
    comp_scaled = scaler.fit_transform(comp_arr)
    player_scaled = scaler.transform(player_arr)
except:
    comp_scaled = comp_arr
    player_scaled = player_arr

# Calculate similarity
if method == 'cosine':
    sims = cosine_similarity(player_scaled, comp_scaled)[0]
    sims = (sims + 1) / 2 # 0-1
elif method == 'euclidean':
    dists = euclidean_distances(player_scaled, comp_scaled)[0]
    sims = 1 / (1 + dists)
elif method == 'manhattan':
    dists = manhattan_distances(player_scaled, comp_scaled)[0]
    sims = 1 / (1 + dists)
elif method == 'mahalanobis':
    try:
        n_samples, n_feat = comp_scaled.shape
        # Covariance needs more samples than features to be full-rank; below
        # that, MinCovDet returns a near-singular matrix and the distances
        # blow up to nonsense (the 1e-6 ridge hides the error instead of
        # surfacing it). Guard on both the sample count and the condition
        # number, and fall back to a true distance (Euclidean) rather than
        # silently returning garbage.
        if n_samples <= n_feat + 1:
            raise ValueError('pool too small for a stable covariance estimate')
        all_data = np.vstack([player_scaled[0], comp_scaled])
        cov_est = MinCovDet(support_fraction=0.75)
        cov_est.fit(all_data)
        cov_matrix = cov_est.covariance_ + np.eye(all_data.shape[1]) * 1e-6
        if np.linalg.cond(cov_matrix) > 1e6:
            raise ValueError('covariance ill-conditioned')
        inv_cov = np.linalg.inv(cov_matrix)
        dists = []
        for row_feat in comp_scaled:
            try:
                d = mahalanobis_dist(player_scaled[0], row_feat, inv_cov)
            except:
                d = np.linalg.norm(player_scaled[0] - row_feat)
            dists.append(d)
        dists = np.array(dists)
        sims = np.exp(-dists / (np.mean(dists) + 1e-6))
    except Exception:
        dists = euclidean_distances(player_scaled, comp_scaled)[0]
        sims = 1 / (1 + dists)
else:
    sims = cosine_similarity(player_scaled, comp_scaled)[0]
    sims = (sims + 1) / 2

# Pool-relative similarity: rank each candidate against the whole pool so
# the score is "closer than X% of comparison players". This keeps all four
# methods on one comparable 0-100 scale - the raw 1/(1+d) conversion
# collapses toward 0 once the feature vector has 100+ metrics, which looks
# broken even though it is mathematically faithful.
sim_pct = pd.Series(sims).rank(pct=True).values * 100

# Magnitude-preserving match score (0-100): min-max of the raw method score
# across the pool. The percentile alone clusters every top match at ~99-100%
# and reads identically for all four methods (it is purely rank-based); this
# keeps the real spread and the differences between methods visible. The

```

```

# closest player is 100, the farthest 0.
s_min, s_max = float(np.min(sims)), float(np.max(sims))
match = (sims - s_min) / (s_max - s_min) * 100 if s_max > s_min else np.full(len(sims), 100.0)
pool_size = int(len(sims))
# 1-based rank, 1 = closest
order = np.argsort(sims)[::-1]
rank_of = np.empty(len(sims), dtype=int)
for pos_rank, idx_ in enumerate(order):
    rank_of[idx_] = pos_rank + 1

# Calculate composite index for comparison pool
style_cats = get_playing_style_categories()
pos_df = calculate_composite_index(pos_df, position, style_cats)

# Top N
top_idx = np.argsort(sims)[::-1][:top_n]
_cols = set(pos_df.columns)
metric_keys = _all_similar_metric_keys(position, _cols)
metric_groups = _all_similar_metric_groups(position, _cols)
similar = []
cand_rows = []
for i in top_idx:
    row = pos_df.iloc[i]
    cand_rows.append(row)
    similar.append({
        'player': str(row.get('player', '')),
        'team': str(row.get('team', '')),
        'league': str(row.get('league', '')),
        'age': parse_age(row.get('age', 0)),
        'position': position,
        'primary_position': str(row.get('primary_position', position) or position),
        'similarity': r(float(match[i]), 1),
        'percentile': r(float(sim_pct[i]), 1),
        'rank': int(rank_of[i]),
        'pool_size': pool_size,
        'goals': safe_float(row.get('goals', 0)),
        'assists': safe_float(row.get('assists', 0)),
        'minutes': safe_float(row.get('minutes', 0)),
        'seasons': int(safe_float(row.get('seasons_count', 1))) if window > 1 else 1,
        'composite': r(safe_float(row.get('composite_index', 0)), 1),
        'stats': _similar_stats(row, position, per90=(window > 1)),
        'metrics': _compare_metrics(row, player_ref, metric_keys, per90=(window > 1)),
    })

# Hidden-gem tag: flag a similar player if they are flagged as a gem EITHER
# across the whole position pool (all leagues) OR within their own league.
try:
    pool_gems = _gem_keyset(season, position, None)
    for sp in similar:
        ident = (_norm_key(sp['player']), sp['team'])
        sp['is_gem'] = ident in pool_gems or ident in _gem_keyset(season, position, sp['league'])
except Exception:
    for sp in similar:
        sp['is_gem'] = False

# Contract/availability tag per match (free / expiring / release-clause) so
# the front end can surface the signable players among the matches.
try:
    avail = _availability_index(season)
    for sp in similar:
        _tag_availability(sp, season, avail)
except Exception:
    pass

```

```

target_summary = {
    'player': player_name,
    'team': team,
    'position': position,
    'age': parse_age(player_ref.get('age', 0)),
    'minutes': safe_float(player_ref.get('minutes', 0)),
    'seasons': int(safe_float(player_ref.get('seasons_count', 1))) if window > 1 else 1,
}

# ---- Advanced analysis: AHP-weighted TOPSIS + sensitivity over the shortlist ----
# Decision matrix = target + candidates (rows) x style-category percentiles
# (columns). AHP sets the weights, TOPSIS ranks by closeness to the ideal,
# sensitivity checks the ranking against +/-50% weight shifts. League data only.
mcdm_summary = None
try:
    cats = list(style_cats.get(position, {}).keys())
    if cats and cand_rows:
        w, CR, ahp_meta = _ahp_weights(cats, position)

    def _catvec(r):
        sc = calculate_category_scores(r, pos_df, style_cats, position,
                                       method='percentile', empty_as_none=True)
        return [float(sc.get(c)) if sc.get(c) is not None else 50.0 for c in cats]

    X = np.array([_catvec(player_ref)] + [_catvec(r) for r in cand_rows], float)
    # Weighted normalized matrix - shared by the closeness score AND the
    # per-criterion gaps ("where he loses ground" = biggest weighted gaps
    # below the ideal Ap).
    denom = np.sqrt((X ** 2).sum(0))
    denom[denom == 0] = 1.0
    V = (X / denom) * w
    Ap, An = V.max(0), V.min(0)
    Dp = np.sqrt(((V - Ap) ** 2).sum(1))
    Dn = np.sqrt(((V - An) ** 2).sum(1))
    tot = Dp + Dn
    tot[tot == 0] = 1.0
    C = Dn / tot
    rank = _mcdm_ranks(C)
    N = int(len(C))
    # Sensitivity: boost each criterion weight +50% in turn and re-rank,
    # keeping the full rank matrix so we can name the driving criterion.
    n_crit = len(w)
    Rk = np.empty((n_crit, N), dtype=int)
    for k in range(n_crit):
        w2 = w.copy()
        w2[k] *= 1.5
        w2 = w2 / w2.sum()
        Rk[k] = _mcdm_ranks(_topsis(X, w2))
    lo = np.minimum(Rk.min(0), rank)
    hi = np.maximum(Rk.max(0), rank)
    priorities = [int(_AHP_PRIORITY.get(position, {}).get(c, 3)) for c in cats]
    ptotal = int(sum(priorities))
    t_rank, t_C = int(rank[0]), float(C[0])
    for j, sp in enumerate(similar):
        ci = j + 1
        c_rank, c_C = int(rank[ci]), float(C[ci])
        r_lo, r_hi = int(lo[ci]), int(hi[ci])
        robust = (r_hi - r_lo) <= 1
        gaps = Ap - V[ci]
        loses = [cats[k] for k in np.argsort(-gaps)[:2] if gaps[k] > 1e-9]
        # the criterion whose +50% weight most hurts (worst) / helps (best)
        col = Rk[:, ci]

```

N)

```

        wk, bk = int(np.argmax(col)), int(np.argmin(col))
        sens_worst = ({'criterion': cats[wk], 'rank': int(col[wk])}
                        if int(col[wk]) > c_rank else None)
        sens_best = ({'criterion': cats[bk], 'rank': int(col[bk])}
                      if int(col[bk]) < c_rank else None)
        rankword = 'the strongest option' if c_rank == 1 else 'ranked #%d of %d' % (c_rank,
N)

        rel = 'above' if c_rank < t_rank else 'below' if c_rank > t_rank else 'level with'
        stab = ('robust to +/-50% weight shifts' if robust
                 else 'weight-sensitive (rank swings #%d-#%d)' % (r_lo, r_hi))
        sp['mcdm'] = {
            'closeness': round(c_C, 3),
            'rank': c_rank,
            'n_alternatives': N,
            'target_rank': t_rank,
            'rank_low': r_lo,
            'rank_high': r_hi,
            'robust': robust,
            'loses_ground': loses,
            'd_best': round(float(Dp[ci]), 3),
            'd_worst': round(float(Dn[ci]), 3),
            'sens_worst': sens_worst,
            'sens_best': sens_best,
            # Detailed-math internals: this candidate's weighted-normalized
            # value per criterion, and its rank under every +50% scenario.
            'weighted': [round(float(v), 4) for v in V[ci]],
            'sens_scenarios': [{'criterion': cats[k], 'rank': int(Rk[k, ci])}
                               for k in range(n_crit)],
            'verdict': ('AHP-weighted TOPSIS puts him %s, %s the target (%#d) - %s.'
                        % (rankword, rel, t_rank, stab)),
        }
        mcdm_summary = {
            'criteria': [{'category': c, 'weight': round(float(w[k]), 3),
                        'priority': priorities[k]}
                         for k, c in enumerate(cats)],
            'priority_total': ptotal,
            'consistency_ratio': round(float(CR), 3),
            'ahp': ahp_meta,
            'ideal': [round(float(v), 4) for v in Ap],
            'anti_ideal': [round(float(v), 4) for v in An],
            'n_alternatives': N,
            'target_closeness': round(t_C, 3),
            'target_rank': t_rank,
        }
    except Exception:
        mcdm_summary = None
    for sp in similar:
        sp.setdefault('mcdm', None)

    return {
        'similar': similar,
        'method': method,
        'metrics_used': len(available),
        'metric_keys': metric_keys,
        'metric_groups': metric_groups,
        'window': window,
        'min_minutes': min_minutes,
        'target': target_summary,
        'mcdm': mcdm_summary,
    }

```

`cmd\_compare\_players` (def) - Head-to-head between a target player and one candidate. Returns the four distance-method similarity scores for the exact pair (scaled identically to

get_similar_players so the cosine score matches the table), plus a metric-by-metric breakdown with a per-metric and per-category winner judged on shared all-leagues percentiles.

```
def cmd_compare_players(req):
    """Head-to-head between a target player and one candidate.

    Returns the four distance-method similarity scores for the exact pair
    (scaled identically to get_similar_players so the cosine score matches
    the table), plus a metric-by-metric breakdown with a per-metric and
    per-category winner judged on shared all-leagues percentiles."""
    season = req.get('season')
    league = req.get('league')
    team = req.get('team')
    player_name = req.get('player')
    cand_league = req.get('cand_league')
    cand_team = req.get('cand_team')
    cand_player = req.get('cand_player')
    if not all([season, league, team, player_name, cand_league, cand_team, cand_player]):
        return {'error': 'season, league, team, player, cand_league, cand_team, cand_player
required'}

    all_df = load_all_leagues_data(season)
    if all_df.empty:
        return {'error': 'No data for season'}

    trows = all_df[(all_df['player'] == player_name) & (all_df['team'] == team)]
    if trows.empty:
        return {'error': f'Player {player_name} not found'}
    target = trows.iloc[0]
    # The candidate may not appear in the CURRENT season (multi-season mode shows
    # his latest-in-window team, which can be a season/club he has since left), so
    # this lookup is best-effort - the multi-season branch finds him in the
    # aggregate, and single-season mode hard-checks it below.
    crows = all_df[(all_df['player'] == cand_player) & (all_df['team'] == cand_team)]
    cand = crows.iloc[0] if not crows.empty else None

    position = str(target.get('primary_position', target.get('position', 'MF')))
    if position not in ['GK', 'DF', 'MF', 'FW']:
        position = 'MF'

    min_minutes = req.get('min_minutes', 600)
    window = max(1, min(5, int(req.get('window', 1) or 1)))

    # Multi-season: re-point target, candidate and pool at the minutes-weighted
    # aggregate so the head-to-head matches the (multi-season) Similar list. The
    # cached _multiseason_pool means this reuses the list's aggregation.
    if window > 1:
        agg = _multiseason_pool(season, position, window)
        if agg is not None and not agg.empty:
            t_by = int(safe_float(target.get('birth_year', 0)))
            tk, ck = _norm_key(player_name), _norm_key(cand_player)
            tmask = (agg['_pkey'] == tk) & (agg['_by'] == t_by)
            if not tmask.any():
                tmask = agg['_pkey'] == tk
            # Locate the candidate in the aggregate by the same name+team the
            # Similar list showed (his latest-in-window club), not by current
            # season - so a candidate who has since moved still resolves.
            cmask = (agg['player'] == cand_player) & (agg['team'] == cand_team)
            if not cmask.any():
                cmask = agg['_pkey'] == ck_
            if tmask.any() and cmask.any():
                target = agg[tmask].iloc[0]
```

```

        cand = agg[cmask].iloc[0]
        c_by = int(safe_float(cand.get('_by', cand.get('birth_year', 0))))
        pool = agg[~tmask.values].copy()
        if 'minutes' in pool.columns and min_minutes:
            is_cand = (pool['_pkey'] == ck_) & (pool['_by'] == c_by)
            filtered = pool[(pool['minutes'] >= min_minutes) | is_cand]
            if len(filtered) >= 10:
                pool = filtered
            pool = pool.reset_index(drop=True)
        else:
            window = 1 # identity not found across seasons -> single season
    else:
        window = 1

if window == 1:
    if cand is None:
        return {'error': f'Candidate {cand_player} not found'}
    # Single season. Pool = players who play/can play this position, all
    # leagues, target excluded (candidate kept). Minutes-filtered to match
    # the Similar Players list so the percentile baselines line up.
    pos_df = position_pool(all_df, position)
    pool = pos_df[~((pos_df['player'] == player_name) & (pos_df['team'] ==
team))].copy().reset_index(drop=True)
    if 'minutes' in pool.columns and min_minutes:
        mins = _num_series(pool['minutes']).fillna(0)
        is_cand = (pool['player'] == cand_player) & (pool['team'] == cand_team)
        filtered = pool[(mins >= min_minutes) | is_cand]
        if len(filtered) >= 10:
            pool = filtered.reset_index(drop=True)

style_cats = get_playing_style_categories()
neg = get_negative_metrics()

# ---- Four-method similarity, reported as a pool-relative percentile ----
# For each method we score the target against EVERY pool member, then read
# off where the candidate ranks ("closer than X% of comparison players").
# This matches get_similar_players exactly and keeps all four methods on one
# comparable 0-100 scale instead of the deflated raw 1/(1+d) values.
all_metrics = []
for cat_metrics in style_cats.get(position, {}).values():
    all_metrics.extend(cat_metrics)
all_metrics = list(set(all_metrics))
available = [m for m in all_metrics if m in pool.columns and m in target.index and m in
cand.index]

cand_locs = np.where(
    ((pool['player'] == cand_player) & (pool['team'] == cand_team)).values
)[0]
cand_i = int(cand_locs[0]) if len(cand_locs) else None

def _pct_of(sims_arr, i):
    return float(pd.Series(sims_arr).rank(pct=True).values[i] * 100)

def _match_of(sims_arr, i):
    # magnitude-preserving 0-100 score: min-max of the raw method scores so
    # the panel shows real spread and the methods differ from each other,
    # instead of every method reading the same rank-based percentile.
    mn, mx = float(np.min(sims_arr)), float(np.max(sims_arr))
    return float((sims_arr[i] - mn) / (mx - mn) * 100) if mx > mn else 100.0

methods = {}
methods_raw = {}
if len(available) >= 3 and not pool.empty and cand_i is not None:

```

```

num = pool[available].apply(_num_series)
col_means = num.mean()
comp_matrix = num.fillna(col_means).fillna(0)
t_vals = []
for m in available:
    tv = _num_series(pd.Series([target[m]]))[0]
    t_vals.append(float(tv) if not pd.isna(tv) else float(col_means.get(m, 0)))
for i, m in enumerate(available):
    if m in neg:
        max_val = max(comp_matrix[m].max(), t_vals[i], 1)
        comp_matrix[m] = max_val - comp_matrix[m]
        t_vals[i] = max_val - t_vals[i]
t_arr = np.array(t_vals).reshape(1, -1)
comp_arr = comp_matrix.values
try:
    scaler = StandardScaler()
    comp_scaled = scaler.fit_transform(comp_arr)
    t_s = scaler.transform(t_arr)
except Exception:
    comp_scaled, t_s = comp_arr, t_arr

cos_sims = (cosine_similarity(t_s, comp_scaled)[0] + 1) / 2
methods['cosine'] = r(_pct_of(cos_sims, cand_i), 1)
methods_raw['cosine'] = r(_match_of(cos_sims, cand_i), 1)
euc_sims = 1 / (1 + euclidean_distances(t_s, comp_scaled)[0])
methods['euclidean'] = r(_pct_of(euc_sims, cand_i), 1)
methods_raw['euclidean'] = r(_match_of(euc_sims, cand_i), 1)
man_sims = 1 / (1 + manhattan_distances(t_s, comp_scaled)[0])
methods['manhattan'] = r(_pct_of(man_sims, cand_i), 1)
methods_raw['manhattan'] = r(_match_of(man_sims, cand_i), 1)
try:
    n_samples, n_feat = comp_scaled.shape
    if n_samples <= n_feat + 1:
        raise ValueError('pool too small for a stable covariance estimate')
    all_data = np.vstack([t_s[0], comp_scaled])
    cov_est = MinCovDet(support_fraction=0.75)
    cov_est.fit(all_data)
    cov_matrix = cov_est.covariance_ + np.eye(all_data.shape[1]) * 1e-6
    if np.linalg.cond(cov_matrix) > 1e6:
        raise ValueError('covariance ill-conditioned')
    inv_cov = np.linalg.inv(cov_matrix)
    mah_dists = []
    for row_feat in comp_scaled:
        try:
            mah_dists.append(mahalanobis_dist(t_s[0], row_feat, inv_cov))
        except Exception:
            mah_dists.append(np.linalg.norm(t_s[0] - row_feat))
    mah_dists = np.array(mah_dists)
    mah_sims = np.exp(-mah_dists / (np.mean(mah_dists) + 1e-6))
    methods['mahalanobis'] = r(_pct_of(mah_sims, cand_i), 1)
    methods_raw['mahalanobis'] = r(_match_of(mah_sims, cand_i), 1)
except Exception:
    # ill-conditioned covariance -> fall back to Euclidean (a true
    # distance), not cosine, so the score still means "output distance".
    methods['mahalanobis'] = methods['euclidean']
    methods_raw['mahalanobis'] = methods_raw['euclidean']

# ---- Metric-by-metric comparison on shared all-leagues percentiles ----
categories = []
t_wins = c_wins = ties = 0
for cat_name, metrics in style_cats.get(position, {}).items():
    avail = [m for m in metrics if m in pool.columns and m in target.index and m in cand.index]
    metric_rows = []

```



```

t_pctls = []
c_pctls = []
for m in avail:
    if m in DESCRIPTOR_METRICS:
        continue
    comp = pool[m].apply(safe_float).dropna().tolist()
    if not comp:
        continue
    tv = safe_float(target[m])
    cv = safe_float(cand[m])
    tp = calculate_percentile_score(tv, comp)
    cp = calculate_percentile_score(cv, comp)
    if m in neg:
        tp = 100 - tp
        cp = 100 - cp
    t_pctls.append(tp)
    c_pctls.append(cp)
    if abs(tp - cp) < 0.5:
        w = 'tie'; ties += 1
    elif tp > cp:
        w = 'target'; t_wins += 1
    else:
        w = 'candidate'; c_wins += 1
    metric_rows.append({
        'metric': m,
        'label': _metric_label(m),
        'target_value': _format_metric_value(m, target.get(m), per90=(window > 1)),
        'cand_value': _format_metric_value(m, cand.get(m), per90=(window > 1)),
        'target_pctile': r(tp, 0),
        'cand_pctile': r(cp, 0),
        'winner': w,
        'negative': m in neg,
    })
if not metric_rows:
    continue
t_cat = float(np.mean(t_pctls)) if t_pctls else 0
c_cat = float(np.mean(c_pctls)) if c_pctls else 0
categories.append({
    'name': cat_name,
    'target_score': r(t_cat, 0),
    'cand_score': r(c_cat, 0),
    'winner': 'target' if t_cat > c_cat + 0.5 else 'candidate' if c_cat > t_cat + 0.5 else
'tie',
    'metrics': metric_rows,
})

# Season-by-season series so the user can VERIFY a multi-season match.
# In multi-season mode this reflects the SAME seasons that fed the blend
# (minutes from _multiseason_pool's position pool, no 270-min floor); the
# 'level' z comes from compute_multiseason_features where it qualifies. A
# window season with no minutes simply isn't in the dataset for that player.
season_series = {'target': [], 'candidate': [], 'metric_label': 'Level (league-neutral z)'}
try:
    lookback = max(2, window - 1)
    feats = compute_multiseason_features(season, None, position, lookback=lookback)
    tk = _norm_key(player_name)
    ck2 = _norm_key(cand_player)
    t_by = int(safe_float(target.get('birth_year', 0)))
    c_by = int(safe_float(cand.get('birth_year', 0)))
    tfeat = feats.get((tk, t_by), {})
    cfeat = feats.get((ck2, c_by), {})
    season_series['target_slope'] = tfeat.get('trajectory_slope')
    season_series['candidate_slope'] = cfeat.get('trajectory_slope')

```

```

if window > 1:
    start = _season_start_year(season)
    win_seasons = [f"{y}-{y + 1}" for y in range(start - (window - 1), start + 1)]
    smap = _MS_SERIES_CACHE.get((season, position, window), {})
    tmin = smap.get((tk, t_by), {})
    cmin = smap.get((ck2, c_by), {})
    tlev = {p['season']: p['score'] for p in tfeat.get('series', [])}
    clef = {p['season']: p['score'] for p in cfeat.get('series', [])}
    season_series['window_seasons'] = win_seasons
    season_series['target'] = [
        {'season': s, 'score': tlev.get(s), 'minutes': tmin.get(s)} for s in win_seasons
    ]
    season_series['candidate'] = [
        {'season': s, 'score': clef.get(s), 'minutes': cmin.get(s)} for s in win_seasons
    ]
else:
    season_series['target'] = tfeat.get('series', [])
    season_series['candidate'] = cfeat.get('series', [])
except Exception:
    pass

# Wages (real where available, else estimated) so the verdict can weigh cost.
avail_idx = _availability_index(season)

def basic_stats(row, pl, tm, lg):
    wage = wage_label = None
    wage_est = False
    rec = avail_idx.get((_norm_key(pl), _norm_key(tm), _norm_key(lg)))
    if rec is not None:
        try:
            w, wage_est = get_wage_value(rec)
            if w:
                wage = r(w, 0)
                wage_label = format_eur(w) + '/wk'
        except Exception:
            pass
    return {
        'player': pl, 'team': tm, 'league': lg, 'position': position,
        'age': parse_age(row.get('age', 0)),
        'minutes': safe_float(row.get('minutes', 0)),
        'games': safe_float(row.get('games', row.get('games_starts', 0))),
        'goals': safe_float(row.get('goals', 0)),
        'assists': safe_float(row.get('assists', 0)),
        'xg': r(safe_float(row.get('xg', 0)), 2),
        'xg_assist': r(safe_float(row.get('xg_assist', 0)), 2),
        'per90': window > 1,
        'wage': wage,
        'wage_label': wage_label,
        'wage_estimated': wage_est,
    }

return {
    'target': basic_stats(target, player_name, team, league),
    'candidate': basic_stats(cand, cand_player, cand_team, cand_league),
    'methods': methods,
    'methods_raw': methods_raw,
    'metrics_used': len(available),
    'window': window,
    'season_series': season_series,
    'categories': categories,
    'summary': {'target_wins': t_wins, 'candidate_wins': c_wins, 'ties': ties},
}

```

`cmd\_get\_career\_history` (def) - A player's composite per season - the line the lab draws in the career-trajectory chart.

```
def cmd_get_career_history(req):
    player_name = req.get('player')
    if not player_name:
        return {'error': 'player required'}

    df = load_player_history(player_name)
    if df.empty:
        return {'history': [], 'player': player_name}

    style_cats = get_playing_style_categories()

    # Group by season+league to compute composite_index per context
    history = []
    for _, row in df.iterrows():
        season = str(row.get('season', ''))
        league = str(row.get('league', ''))
        position = str(row.get('primary_position', row.get('position', 'MF')))
        if position not in ['GK', 'DF', 'MF', 'FW']:
            position = 'MF'

        # Try to compute composite_index for this season/league
        composite = 0
        try:
            league_df = load_league_data(season, league)
            pos_pool = league_df[league_df['primary_position'] == position].copy() if
'primary_position' in league_df.columns else league_df.copy()
            if len(pos_pool) > 1:
                pos_pool = calculate_composite_index(pos_pool, position, style_cats)
                # Match the SPECIFIC club row, not just the player name - a
                # player can have two rows in one season+league after a
                # mid-season transfer (e.g. Coventry + Swansea in 2020-21),
                # and .iloc[0] would otherwise give both stints the same score.
                team = str(row.get('team', ''))
                p_rows = pos_pool[(pos_pool['player'] == player_name) & (pos_pool['team'] == team)]
                if p_rows.empty:
                    p_rows = pos_pool[pos_pool['player'] == player_name]
                if not p_rows.empty:
                    composite = safe_float(p_rows.iloc[0].get('composite_index', 0))
        except:
            pass

        history.append({
            'season': season,
            'team': str(row.get('team', '')),
            'league': league,
            'position': position,
            'age': parse_age(row.get('age', 0)),
            'games': safe_float(row.get('games', row.get('games_starts', 0))),
            'minutes': safe_float(row.get('minutes', 0)),
            'goals': safe_float(row.get('goals', 0)),
            'assists': safe_float(row.get('assists', 0)),
            'xg': r(safe_float(row.get('xg', 0)), 2),
            'xg_assist': r(safe_float(row.get('xg_assist', 0)), 2),
            'composite': r(composite, 1),
        })

    return {'history': history, 'player': player_name}
```

`cmd\_get\_contract\_opportunities` (def)

```

def cmd_get_contract_opportunities(req):
    season = req.get('season')
    league = req.get('league')
    position = req.get('position', 'MF')
    months_threshold = req.get('months_threshold', 12)
    if not season:
        return {'error': 'season required'}

    if league:
        df = load_league_data(season, league)
    else:
        df = load_all_leagues_data(season)

    # Merge supplementary data for contract info
    df = merge_supplementary(df, season)

    pos_df = df[df['primary_position'] == position].copy() if 'primary_position' in df.columns else df.copy()
    if pos_df.empty:
        return {'free_targets': [], 'expiring_stars': [], 'clause_steals': []}

    # Calculate composite for sorting
    style_cats = get_playing_style_categories()
    composites = []
    for _, row in pos_df.iterrows():
        scores = calculate_category_scores(row, pos_df, style_cats, position, method='percentile')
        composites.append(float(np.mean(list(scores.values())) if scores else 0)
    pos_df['composite_index'] = composites

    # Contract months
    pos_df['contract_months'] = pos_df.apply(lambda row: contract_months_remaining(row.to_dict()),
axis=1)

    # Market value
    pos_df['market_val'] = pos_df.apply(lambda row: get_market_value(row.to_dict())[0], axis=1)
    pos_df['mv_estimated'] = pos_df.apply(lambda row: get_market_value(row.to_dict())[1], axis=1)

    has_contract = pos_df['contract_months'].notna()

    def to_result(sub_df, limit=15):
        out = []
        for _, row in sub_df.head(limit).iterrows():
            out.append({
                'player': str(row.get('player', '')),
                'team': str(row.get('team', '')),
                'league': str(row.get('league', '')),
                'age': parse_age(row.get('age', 0)),
                'composite': r(row.get('composite_index', 0), 1),
                'contract_months': int(row['contract_months']) if
pd.notna(row.get('contract_months')) else None,
                'market_value': r(row.get('market_val', 0), 0),
                'market_value_label': format_eur(row.get('market_val', 0)),
                'mv_estimated': bool(row.get('mv_estimated', True)),
            })
        return out

    # Free targets: contract expired or <= 6 months
    free = pos_df[has_contract & (pos_df['contract_months'] <= 6)].sort_values('composite_index',
ascending=False)

    # Expiring stars: 6-threshold months, above median composite
    median_comp = pos_df['composite_index'].median() if len(pos_df) > 5 else 50
    expiring = pos_df[has_contract & (pos_df['contract_months'] > 6) & (pos_df['contract_months'] <=

```

```

months_threshold) & (pos_df['composite_index'] >= median_comp)].sort_values('composite_index',
ascending=False)

# Clause steals
clause_steals_list = []
if 'release_clause_eur' in pos_df.columns:
    pos_df['_rc'] = pos_df['release_clause_eur'].apply(safe_float)
    clause_mask = (pos_df['_rc'] > 0) & (pos_df['market_val'] > 0) & (pos_df['_rc'] <
pos_df['market_val'] * 0.75)
    clause_df = pos_df[clause_mask].copy()
    if not clause_df.empty:
        clause_df['discount'] = ((1 - clause_df['_rc'] / clause_df['market_val']) * 100).round(1)
        clause_df = clause_df.sort_values('discount', ascending=False)
        for _, row in clause_df.head(15).iterrows():
            clause_steals_list.append({
                'player': str(row.get('player', '')),
                'team': str(row.get('team', '')),
                'league': str(row.get('league', '')),
                'release_clause': format_eur(row.get('_rc', 0)),
                'market_value_label': format_eur(row.get('market_val', 0)),
                'discount': r(row.get('discount', 0), 1),
                'composite': r(row.get('composite_index', 0), 1),
            })

return {
    'free_targets': to_result(free),
    'expiring_stars': to_result(expiring),
    'clause_steals': clause_steals_list,
}

```

`cmd\_get\_wage\_benchmark` (def)

```

def cmd_get_wage_benchmark(req):
    season = req.get('season')
    league = req.get('league')
    team = req.get('team')
    player_name = req.get('player')
    if not all([season, league, team, player_name]):
        return {'error': 'season, league, team, player required'}

    df = load_players(season, league, team)
    player_rows = df[df['player'] == player_name]
    if player_rows.empty:
        return {'error': f'Player {player_name} not found'}
    player = player_rows.iloc[0]
    position = str(player.get('primary_position', player.get('position', 'MF')))

    # Merge supplementary (smart: tries player+team+league, then falls back to player name only)
    supp = load_supplementary(season)
    player_dict = player.to_dict()
    if not supp.empty:
        # Try exact match first, then fallback to player name only
        match = supp[(supp['player'] == player_name) & (supp['team'] == team) & (supp['league'] ==
league)]
        if match.empty:
            match = supp[supp['player'] == player_name]
        if not match.empty:
            for col in ['weekly_wage_eur', 'annual_wage_eur', 'market_value_eur', 'contract_expiry']:
                if col in match.columns:
                    player_dict[col] = match.iloc[0].get(col)

    wage, wage_est = get_wage_value(player_dict)
    league_df = load_league_data(season, league)

```

```

all_df = load_all_leagues_data(season)

pos_league = league_df[league_df['primary_position'] == position] if 'primary_position' in
league_df.columns else league_df
pos_all = all_df[all_df['primary_position'] == position] if 'primary_position' in all_df.columns
else all_df

# Merge supp for wages (smart merge)
pos_league = merge_supplementary(pos_league.copy(), season)
pos_all = merge_supplementary(pos_all.copy(), season)

def get_wages_series(src_df):
    return src_df.apply(lambda row: get_wage_value(row.to_dict())[0], axis=1).dropna()

league_wages = get_wages_series(pos_league)
all_wages = get_wages_series(pos_all)

league_wage_pct = r(stats.percentileofscore(league_wages, wage, kind='rank'), 1) if
len(league_wages) > 0 else 50
global_wage_pct = r(stats.percentileofscore(all_wages, wage, kind='rank'), 1) if len(all_wages) >
0 else 50

# Composite/performance percentile
style_cats = get_playing_style_categories()
scores = calculate_category_scores(player, pos_league, style_cats, position, method='percentile')
composite = float(np.mean(list(scores.values()))) if scores else 50

perf_pct = r(composite, 1)

# Quadrant
if composite >= 60 and league_wage_pct <= 40:
    quadrant = 'Underpaid Star'
elif composite >= 50 and league_wage_pct >= 50:
    quadrant = 'Fair Value'
elif composite < 50 and league_wage_pct >= 60:
    quadrant = 'Overpaid'
elif composite < 40 and league_wage_pct < 40:
    quadrant = 'Developing'
else:
    quadrant = 'Fair Value'

return {
    'player': player_name,
    'position': position,
    'wage': r(wage, 0),
    'wage_label': format_eur(wage) + '/wk',
    'wage_estimated': wage_est,
    'league_wage_percentile': league_wage_pct,
    'global_wage_percentile': global_wage_pct,
    'performance_percentile': perf_pct,
    'quadrant': quadrant,
    'league_avg_wage': r(float(league_wages.mean()), 0) if len(league_wages) > 0 else 0,
    'league_median_wage': r(float(league_wages.median()), 0) if len(league_wages) > 0 else 0,
    'position_avg_wage': r(float(league_wages.mean()), 0) if len(league_wages) > 0 else 0,
}

```

`\_next\_season\_label` (def) - '2024-2025' -> '2025-2026'. Returns None if unparseable.

```

def _next_season_label(season):
    """'2024-2025' -> '2025-2026'. Returns None if unparseable."""
    try:
        a, b = str(season).split('-')
        return f"{int(a) + 1}-{int(b) + 1}"

```

```
except Exception:
    return None
```

`cmd\_get\_market\_value` (def) - The market-value trajectory and the heuristic breakdown for a player.

```
def cmd_get_market_value(req):
    season = req.get('season')
    league = req.get('league')
    team = req.get('team')
    player_name = req.get('player')
    if not all([season, league, team, player_name]):
        return {'error': 'season, league, team, player required'}

    df = load_players(season, league, team)
    player_rows = df[df['player'] == player_name]
    if player_rows.empty:
        return {'error': f'Player {player_name} not found'}

    player = player_rows.iloc[0]
    player_dict = player.to_dict()

    # Merge supplementary (smart: tries player+team, then falls back to player name only)
    supp = load_supplementary(season)
    if not supp.empty:
        match = supp[(supp['player'] == player_name) & (supp['team'] == team)]
        if match.empty:
            match = supp[supp['player'] == player_name]
        if not match.empty:
            for col in ['market_value_eur', 'weekly_wage_eur', 'contract_expiry']:
                if col in match.columns:
                    player_dict[col] = match.iloc[0].get(col)

    method = 'heuristic' # heuristic-only valuation (single, readable formula)

    mv, mv_est = get_market_value(player_dict, method=method)

    # Build trajectory from history. Each season goes through get_market_value,
    # which returns a verified market value when one is on file, otherwise the
    # heuristic estimate. So only the seasons we had to estimate are modelled.
    history = load_player_history(player_name)

    # Per-season verified market values: index supplementary by season so each
    # trajectory point can use a real value when one exists for THAT season
    # (preferring the row whose team matches the history row), instead of always
    # estimating. No cross-season fallback - an old season stays estimated if it
    # has no verified value of its own.
    supp_hist = load_player_supplementary_history(player_name)
    supp_by_season = {}
    if not supp_hist.empty and 'season' in supp_hist.columns and 'market_value_eur' in
    supp_hist.columns:
        for _, sr in supp_hist.iterrows():
            mvv = safe_float(sr.get('market_value_eur', 0))
            if mvv > 0:
                supp_by_season.setdefault(str(sr.get('season')), []).append(
                    (_norm_key(sr.get('team', '')), mvv))

    trajectory = []
    if not history.empty:
        for s in sorted(history['season'].unique()):
            srows = history[history['season'] == s]
            row_dict = srows.iloc[0].to_dict()
            cands = supp_by_season.get(str(s), [])
```

```

        if cands:
            tkey = _norm_key(row_dict.get('team', ''))
            real = next((mv for tk, mv in cands if tk == tkey), cands[0][1])
            row_dict['market_value_eur'] = real
        val, is_est = get_market_value(row_dict, method=method)
        trajectory.append({
            'season': str(s),
            'age': parse_age(row_dict.get('age', 25)),
            'actual_mv': r(val, 0) if not is_est else None,
            'estimated_mv': r(val, 0) if is_est else None,
            'display_mv': r(val, 0),
            'is_estimated': is_est,
        })

# Force the selected season's point to equal the headline value, so the
# chart can never contradict the "Current Market Value" card. The headline
# is merged with supplementary verified values above; raw history rows from
# league_season_team_player_data usually are not, which is why a player can
# show a verified 70M headline yet an all-estimated trajectory otherwise.
cur_label = str(season)
cur_point = {
    'actual_mv': r(mv, 0) if not mv_est else None,
    'estimated_mv': r(mv, 0) if mv_est else None,
    'display_mv': r(mv, 0),
    'is_estimated': mv_est,
}
matched = next((p for p in trajectory if p['season'] == cur_label), None)
if matched is not None:
    matched.update(cur_point)
else:
    trajectory.append({'season': cur_label,
                       'age': parse_age(player_dict.get('age', 25)),
                       **cur_point})
    trajectory.sort(key=lambda p: p['season'])

# Career trend + a regression forecast for one season beyond the last.
trend_direction = 'Stable'
forecast = mv
if len(trajectory) >= 2:
    vals = [p['display_mv'] for p in trajectory]
    x = np.arange(len(vals))
    try:
        slope, intercept, r_val, _, _ = stats.linregress(x, vals)
        forecast = max(0, intercept + slope * len(x))
        trend_direction = 'Rising' if slope > 0 else 'Declining' if slope < 0 else 'Stable'
    except:
        pass

# The "next" figure is relative to the SELECTED season, not the latest, so
# viewing an old season doesn't surface a far-future number. If the season
# after the selected one is already in the trajectory, show its real value
# (known, not a guess); only when the selected season is the latest do we
# fall back to the regression forecast.
seasons_sorted = [p['season'] for p in trajectory]
next_value = forecast
next_is_forecast = True
next_season = _next_season_label(cur_label)
if cur_label in seasons_sorted:
    i = seasons_sorted.index(cur_label)
    if i + 1 < len(trajectory):
        nxt = trajectory[i + 1]
        next_value = nxt['display_mv']
        next_is_forecast = False

```



```

        next_season = nxt['season']

    return {
        'player': player_name,
        'season': cur_label,
        'current_value': r(mv, 0),
        'current_value_label': format_eur(mv),
        'is_estimated': mv_est,
        'method': method,
        'method_label': 'Heuristic',
        'trajectory': trajectory,
        'trend': trend_direction,
        'next_predicted': r(next_value, 0),
        'next_predicted_label': format_eur(next_value),
        'next_is_forecast': next_is_forecast,
        'next_season': next_season,
    }

```

`cmd\_get\_moneyball\_score` (def) - The 0.5/0.3/0.2 moneyball blend for one player.

```

def cmd_get_moneyball_score(req):
    season = req.get('season')
    league = req.get('league')
    team = req.get('team')
    player_name = req.get('player')
    if not all([season, league, team, player_name]):
        return {'error': 'season, league, team, player required'}

    df = load_players(season, league, team)
    player_rows = df[df['player'] == player_name]
    if player_rows.empty:
        return {'error': f'Player {player_name} not found'}

    player = player_rows.iloc[0]
    player_dict = player.to_dict()
    position = str(player.get('primary_position', player.get('position', 'MF')))

    # Merge supplementary (smart: tries player+team, then falls back to player name only)
    supp = load_supplementary(season)
    if not supp.empty:
        match = supp[(supp['player'] == player_name) & (supp['team'] == team)]
        if match.empty:
            match = supp[supp['player'] == player_name]
        if not match.empty:
            for col in ['weekly_wage_eur', 'annual_wage_eur', 'market_value_eur', 'contract_expiry',
'release_clause_eur']:
                if col in match.columns:
                    player_dict[col] = match.iloc[0].get(col)

    # Performance = the full composite index (0.4 z-aggregate + 0.3 style + 0.3
    # league power), matching the Day 4 formula and the Hidden Gems detector.
    # The plain style-category average under-rates output-dominant specialists
    # (e.g. Haaland: style-avg ~76 vs composite ~91, because averaging in his
    # weaker creation/link-up categories masks his elite output z-score).
    league_df = load_league_data(season, league)
    league_df = merge_supplementary(league_df, season)
    pos_df = league_df[league_df['primary_position'] == position].copy() if 'primary_position' in
league_df.columns else league_df.copy()
    style_cats = get_playing_style_categories()
    composite = None
    perf_zagg = perf_style = perf_power = None
    if len(pos_df) > 1:
        pos_ci = calculate_composite_index(pos_df, position, style_cats)

```

```

crow = pos_ci[(pos_ci['player'] == player_name) & (pos_ci['team'] == team)]
if crow.empty:
    crow = pos_ci[pos_ci['player'] == player_name]
if not crow.empty:
    c0 = crow.iloc[0]
    composite = safe_float(c0.get('composite_index', None))
    perf_zagg = safe_float(c0.get('zscore_comp', None))
    perf_style = safe_float(c0.get('style_pctile_avg', None))
    perf_power = safe_float(c0.get('power_norm', None))
# Fallback: style-category average if the composite couldn't be computed.
if not composite:
    scores = calculate_category_scores(player, pos_df, style_cats, position, method='percentile')
    composite = float(np.mean(list(scores.values()))) if scores else 50

# Value efficiency = (performance / wage percentile) x 50, capped 0-100.
wage, wage_est = get_wage_value(player_dict)
wages_series = pos_df.apply(lambda row: get_wage_value(row.to_dict())[0], axis=1) if not
pos_df.empty else pd.Series([500])
wage_pctl = stats.percentileofscore(wages_series.dropna(), wage, kind='rank')
wage_pctl = max(wage_pctl, 1)
value_ratio_raw = (composite / wage_pctl) * 50
# Match the Hidden Gems detector (Day 3): a value ratio is only meaningful
# with a REAL wage on file. When the wage is estimated we can't judge
# under/overpayment, so this component is neutralised to 50 - keeping Day 4
# in sync with Day 3 for the ~88% of players without a verified wage.
value_eff = 50.0 if wage_est else min(100, max(0, value_ratio_raw))

# Contract opportunity (urgency + release-clause discount)
contract = contract_opportunity_breakdown(player_dict)
contract_score = contract['total']

# Moneyball
moneyball = calculate_moneyball(composite, value_eff, contract_score)

method = req.get('method', 'heuristic')
if method not in ('heuristic', 'linear', 'gbm'):
    method = 'heuristic'
mv, mv_est = get_market_value(player_dict, method=method)

return {
    'player': player_name,
    'position': position,
    'season': str(season),
    'moneyball_score': r(moneyball, 1),
    'performance_score': r(composite, 1),
    'perf_zaggregate': r(perf_zagg, 1) if perf_zagg is not None else None,
    'perf_style': r(perf_style, 1) if perf_style is not None else None,
    'perf_power': r(perf_power, 1) if perf_power is not None else None,
    'value_efficiency': r(value_eff, 1),
    'value_ratio_raw': r(value_ratio_raw, 1),
    'value_capped': bool(value_ratio_raw > 100),
    'wage_percentile': r(wage_pctl, 0),
    'contract_opportunity': r(contract_score, 1),
    'contract_months': contract['months'],
    'contract_urgency': contract['urgency'],
    'contract_clause': contract['clause'],
    'wage': r(wage, 0),
    'wage_label': format_eur(wage) + '/wk',
    'wage_estimated': wage_est,
    'market_value': r(mv, 0),
    'market_value_label': format_eur(mv),
    'mv_estimated': mv_est,
}

```

`cmd\_get\_squad\_optimizer` (def)

```
def cmd_get_squad_optimizer(req):
    season = req.get('season')
    league = req.get('league')
    budget = req.get('budget', 500000)
    constraint_type = req.get('constraint_type', 'wage')
    positions_needed = req.get('positions', {'GK': 1, 'DF': 4, 'MF': 3, 'FW': 3})
    if not season:
        return {'error': 'season required'}

    if league:
        df = load_league_data(season, league)
    else:
        df = load_all_leagues_data(season)

    if df.empty:
        return {'squad': [], 'total_cost': 0, 'total_score': 0}

    # Merge supplementary
    df = merge_supplementary(df, season)

    style_cats = get_playing_style_categories()
    selected = []
    remaining = budget

    for pos, count in positions_needed.items():
        pos_pool = df[df['primary_position'] == pos].copy() if 'primary_position' in df.columns else df.copy()
        if pos_pool.empty:
            continue

        # Calculate composite + moneyball for this position
        composites = []
        for _, row in pos_pool.iterrows():
            scores = calculate_category_scores(row, pos_pool, style_cats, pos, method='percentile')
            composites.append(float(np.mean(list(scores.values())) if scores else 0)
            pos_pool['composite_index'] = composites

        # Cost
        if constraint_type == 'wage':
            pos_pool['_cost'] = pos_pool.apply(lambda row: get_wage_value(row.to_dict())[0],
axis=1).fillna(500)
        else:
            pos_pool['_cost'] = pos_pool.apply(lambda row: get_market_value(row.to_dict())[0],
axis=1).fillna(100000)

        # Moneyball
        wages = pos_pool['_cost']
        wage_pctiles = wages.apply(lambda w: max(stats.percentileofscore(wages.dropna(), w,
kind='rank'), 1))
        pos_pool['value_eff'] = ((pos_pool['composite_index'] / wage_pctiles) * 50).clip(0, 100)
        pos_pool['contract_opp'] = pos_pool.apply(lambda r:
calculate_contract_opportunity_score(r.to_dict()), axis=1)
        pos_pool['moneyball_score'] = pos_pool.apply(
            lambda r: calculate_moneyball(r.get('composite_index', 50), r.get('value_eff', 50),
r.get('contract_opp', 20)), axis=1
        )

        # Efficiency = score / cost
        pos_pool['_efficiency'] = pos_pool['moneyball_score'] / pos_pool['_cost'].clip(lower=1)
        pos_pool = pos_pool.sort_values('_efficiency', ascending=False)
```

```

picked = 0
for _, p in pos_pool.iterrows():
    if picked >= count:
        break
    cost = p['_cost']
    if cost <= remaining:
        selected.append({
            'player': str(p.get('player', '')),
            'team': str(p.get('team', '')),
            'league': str(p.get('league', '')),
            'position': pos,
            'age': parse_age(p.get('age', 0)),
            'cost': r(cost, 0),
            'cost_label': format_eur(cost) + ('/wk' if constraint_type == 'wage' else ''),
            'moneyball_score': r(p.get('moneyball_score', 0), 1),
            'composite': r(safe_float(p.get('composite_index', 0)), 1),
            'goals': safe_float(p.get('goals', 0)),
            'assists': safe_float(p.get('assists', 0)),
        })
        remaining -= cost
        picked += 1

total_cost = sum(p['_cost'] for p in selected)
total_score = sum(p['moneyball_score'] for p in selected)

return {
    'squad': selected,
    'total_cost': r(total_cost, 0),
    'total_cost_label': format_eur(total_cost),
    'total_score': r(total_score, 1),
    'remaining_budget': r(remaining, 0),
    'remaining_label': format_eur(remaining),
    'constraint_type': constraint_type,
}

```

`cmd\_check\_update` (def) - Fast status read against league_season_team_player_data. After migration 001 this is a regular table (post: relkind='r'); before that it was a materialised view (relkind='m').

```

def cmd_check_update(req):
    """Fast status read against league_season_team_player_data. After
    migration 001 this is a regular table (post: relkind='r'); before
    that it was a materialised view (relkind='m'). Same query works
    on both. last_update is pulled from player_stats since the
    aggregated table doesn't carry that column."""
    import time as _time
    target_season = req.get('season', '2024-2025')
    t0 = _time.time()

    db_counts = {}
    last_update = None

    # Per-league counts from the main table.
    try:
        with get_connection() as conn:
            with conn.cursor() as cur:
                cur.execute(
                    """
                    SELECT league, COUNT(*) AS players
                    FROM league_season_team_player_data
                    WHERE season = %s
                    GROUP BY league
                    """,

```

```

        (target_season,),
    )
    for r in cur.fetchall():
        db_counts[str(r[0])] = int(r[1])
except Exception as e:
    print(f"[check_update] counts query failed: {e}", file=sys.stderr, flush=True)

# last_update lives on the raw player_stats source table - separate
# query so a missing column on either side never zeros the counts.
try:
    with get_connection() as conn:
        with conn.cursor() as cur:
            cur.execute(
                "SELECT MAX(updated_at) FROM player_stats WHERE season = %s",
                (target_season,),
            )
            row = cur.fetchone()
            if row and row[0] is not None:
                last_update = str(row[0])
except Exception as e:
    print(f"[check_update] last_update query failed (non-fatal): {e}", file=sys.stderr,
flush=True)

results = []
for league_key, cfg in LEAGUE_MAP.items():
    label = league_key.replace('-', ' ').title()
    current_rows = db_counts.get(league_key, 0)
    sources = ['FBref']
    if cfg.get('understat'): sources.append('Understat')
    if cfg.get('capology'): sources.append('Capology')
    if cfg.get('tm'): sources.append('Transfermarkt')
    results.append({
        'league': label,
        'league_key': league_key,
        'current_rows': current_rows,
        'sources': ', '.join(sources),
    })

elapsed = _time.time() - t0
print(f"[check_update] season={target_season} → {len(db_counts)} leagues,
{sum(db_counts.values())} rows in {elapsed:.2f}s", file=sys.stderr, flush=True)

return {
    'leagues': results,
    'target_season': target_season,
    'last_update': last_update,
    'total_rows': sum(db_counts.values()),
}

```

`\_snapshot\_season\_counts` (def) - Per-league row counts for the requested season. Used to build the verification table both before and after a refresh.

```

def _snapshot_season_counts(season):
    """Per-league row counts for the requested season. Used to build
    the verification table both before and after a refresh."""
    snap = {}
    try:
        with get_connection() as conn:
            vdf = pd.read_sql(
                """
                SELECT league, COUNT(*) as players
                FROM league_season_team_player_data
                WHERE season = %s GROUP BY league ORDER BY league
            """

```

```

        """
        conn,
        params=[season],
    )
    for _, row in vdf.iterrows():
        snap[str(row['league'])] = int(row['players'])
except Exception:
    pass
return snap

```

Imports.

```

import threading
import uuid as _uuid

```

`\_update\_jobs\_lock` (module constant)

```
_update_jobs_lock = threading.Lock()
```

`\_update\_jobs` (module constant)

```
_update_jobs = {} # job_id -> {'status', 'season', 'steps', 'errors', 'started_at', 'finished_at', 'result'}
```

`\_new\_job` (def)

```

def _new_job(season):
    job_id = str(_uuid.uuid4())
    with _update_jobs_lock:
        _update_jobs[job_id] = {
            'job_id': job_id,
            'season': season,
            'status': 'running',
            'steps': [],
            'errors': [],
            'started_at': time.time(),
            'finished_at': None,
            'result': None,
            'cancelled': False,
        }
    return job_id

```

`\_is\_cancelled` (def)

```

def _is_cancelled(job_id) -> bool:
    with _update_jobs_lock:
        job = _update_jobs.get(job_id)
        return bool(job and job.get('cancelled'))

```

`\_record\_step` (def)

```

def _record_step(job_id, step_name, status, detail):
    with _update_jobs_lock:
        job = _update_jobs.get(job_id)
        if not job:
            return
        # Update existing step if present, otherwise append
        for s in job['steps']:
            if s['step'] == step_name:
                s['status'] = status
                s['detail'] = detail
                return
        job['steps'].append({'step': step_name, 'status': status, 'detail': detail})

```

`\_finish\_job` (def)

```
def _finish_job(job_id, status, result=None, errors=None):
    with _update_jobs_lock:
        job = _update_jobs.get(job_id)
        if not job:
            return
        job['status'] = status
        job['finished_at'] = time.time()
        if result is not None:
            job['result'] = result
        if errors:
            job['errors'] = errors
```

`\_run\_update\_worker` (def) - Body of the background thread that does the actual scrape + sync. Imports are inside the function so a missing dependency only blows up the job, not the whole scout server.

```
def _run_update_worker(season, job_id):
    """Body of the background thread that does the actual scrape + sync.
    Imports are inside the function so a missing dependency only blows up
    the job, not the whole scout server."""
    try:
        # Make legacy_scrapers + sync_view importable
        legacy_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'legacy_scrapers')
        if legacy_dir not in sys.path:
            sys.path.insert(0, legacy_dir)
        scout_dir = os.path.dirname(os.path.abspath(__file__))
        if scout_dir not in sys.path:
            sys.path.insert(0, scout_dir)

        from run_update import run_update as _scrape_run_update

        _record_step(job_id, 'Initialising', 'completed', f'Starting scrape for {season}')

        def _progress(step, status, detail):
            _record_step(job_id, step, status, detail)

        # The inline per-league sync inside run_update.py now does the
        # job sync_for_season used to do at the end. No separate
        # "Sync to lab table" step here.
        scrape_result = _scrape_run_update(
            season,
            _progress,
            cancel_check=lambda: _is_cancelled(job_id),
        )

        if _is_cancelled(job_id):
            _finish_job(
                job_id,
                'cancelled',
                result={
                    'season': season,
                    'scrape': scrape_result,
                    'verification_after': _snapshot_season_counts(season),
                },
                errors=scrape_result.get('errors', []),
            )
            return

        # Aggregate per-league sync diffs into a single rollup the UI
        # can render. stats['sync'] is {league_label: {...sync_for_league result...}}.
```

```

sync_per_league = (scrape_result.get('stats') or {}).get('sync', {})
rollup = {'inserted': 0, 'updated': 0, 'unchanged': 0, 'deleted': 0}
sample_diffs_all = []
for lg_label, r in sync_per_league.items():
    for k in rollup:
        rollup[k] += r.get(k, 0)
    for d in r.get('sample_diffs', [])[0:5]: # cap per-league to keep payload small
        sample_diffs_all.append(**d, 'league': lg_label)

_finish_job(
    job_id,
    'completed',
    result={
        'season': season,
        'scrape': scrape_result,
        'sync_totals': rollup,
        'sample_diffs': sample_diffs_all[0:50], # cap total
        'sync_per_league': sync_per_league,
        'verification_after': _snapshot_season_counts(season),
    },
    errors=scrape_result.get('errors', []),
)
except Exception as e:
    print(f'[run_update worker] {job_id} failed: {e}', file=sys.stderr, flush=True)
    _record_step(job_id, 'Fatal', 'error', str(e))
    _finish_job(job_id, 'error', errors=[str(e)])

```

`cmd\_run\_update` (def) - Kick off a real run_update in a background thread. Returns immediately with a job_id; UI polls cmd_get_update_progress.

```

def cmd_run_update(req):
    """Kick off a real run_update in a background thread. Returns
    immediately with a job_id; UI polls cmd_get_update_progress."""
    target_season = req.get('season', '2024-2025')
    before = _snapshot_season_counts(target_season)
    job_id = _new_job(target_season)

    # Stash before-counts on the job so the progress endpoint can
    # surface deltas without re-querying.
    with _update_jobs_lock:
        _update_jobs[job_id]['before'] = before

    t = threading.Thread(
        target=_run_update_worker,
        args=(target_season, job_id),
        daemon=True,
    )
    t.start()

    print(f'[run_update] dispatched job={job_id} season={target_season}', file=sys.stderr,
    flush=True)
    return {
        'job_id': job_id,
        'season': target_season,
        'status': 'running',
        'before': before,
    }

```

`cmd\_get\_update\_progress` (def) - Read the live state of a job started by cmd_run_update.

```

def cmd_get_update_progress(req):

```



```

"""Read the live state of a job started by cmd_run_update."""
job_id = req.get('job_id')
if not job_id:
    return {'error': 'job_id required'}
with _update_jobs_lock:
    job = _update_jobs.get(job_id)
    if not job:
        return {'error': 'job not found'}
    snapshot = dict(job)
    snapshot['steps'] = list(job['steps']) # shallow copy
# Always include current after-counts so the UI can show live deltas
snapshot['after'] = _snapshot_season_counts(job['season'])
return snapshot

```

`cmd\_retry\_league` (def) - Re-run the FBref + Understat + Capology + Transfermarkt scrape + sync for a single league. Same job-state machinery as cmd_run_update so the existing progress polling + cancel work unchanged.

```

def cmd_retry_league(req):
    """Re-run the FBref + Understat + Capology + Transfermarkt scrape +
    sync for a single league. Same job-state machinery as cmd_run_update
    so the existing progress polling + cancel work unchanged.

    Useful when one league has missing source data while others succeeded
    (e.g. FBref's standard-stats page didn't snapshot to Wayback for
    Eredivisie 2025-26 but the other pages did)."""
    season = req.get('season')
    league_key = req.get('league_key')
    if not season or not league_key:
        return {'error': 'season + league_key required'}
    if league_key not in LEAGUE_MAP:
        return {'error': f'unknown league: {league_key}'}

    before = _snapshot_season_counts(season)
    job_id = _new_job(season)
    with _update_jobs_lock:
        _update_jobs[job_id]['scope'] = f'retry:{league_key}'
        _update_jobs[job_id]['before'] = before

    def _worker():
        try:
            legacy_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'legacy_scrapers')
            if legacy_dir not in sys.path:
                sys.path.insert(0, legacy_dir)
            scout_dir = os.path.dirname(os.path.abspath(__file__))
            if scout_dir not in sys.path:
                sys.path.insert(0, scout_dir)

            from run_update import run_one_league

            _record_step(job_id, 'Initialising', 'completed', f'Retrying {league_key} for {season}')

            def _progress(step, status, detail):
                _record_step(job_id, step, status, detail)

            # run_one_league mutates stats + errors in place; we provide
            # the same dicts run_update would use so the result shape is
            # consistent with a full run.
            stats = {'fbref': {}, 'understat': {}, 'capology': {},
                    'transfermarkt': {}, 'supplementary': {}}
            errors: list = []
            sync_res = run_one_league(
                season,

```

```

        int(season.split('-')[0]),
        league_key,
        _progress,
        stats,
        errors,
    )

    # Aggregate into the same shape cmd_run_update produces so
    # the UI's rendering code doesn't need a different branch.
    sync_per_league = stats.get('sync', {})
    rollup = {'inserted': 0, 'updated': 0, 'unchanged': 0, 'deleted': 0}
    sample_diffs_all = []
    for lg_label, r in sync_per_league.items():
        for k in rollup:
            rollup[k] += r.get(k, 0)
        for d in r.get('sample_diffs', []):20]:
            sample_diffs_all.append(**d, 'league': lg_label})

    final_status = 'cancelled' if _is_cancelled(job_id) else 'completed'
    _finish_job(
        job_id,
        final_status,
        result={
            'season': season,
            'league_key': league_key,
            'scrape': {'season': season, 'stats': stats, 'errors': errors},
            'sync_totals': rollup,
            'sample_diffs': sample_diffs_all,
            'sync_per_league': sync_per_league,
            'verification_after': _snapshot_season_counts(season),
        },
        errors=errors,
    )
except Exception as e:
    print(f'[retry_league worker] {job_id} failed: {e}', file=sys.stderr, flush=True)
    _record_step(job_id, 'Fatal', 'error', str(e))
    _finish_job(job_id, 'error', errors=[str(e)])

threading.Thread(target=_worker, daemon=True).start()
print(f'[retry_league] dispatched job={job_id} season={season} league={league_key}',
file=sys.stderr, flush=True)
return {'job_id': job_id, 'season': season, 'league_key': league_key,
        'status': 'running', 'before': before}

```

`cmd\_get\_coverage` (def) - Return per-league column-coverage stats for the given season. Read-only, no scraping.

```

def cmd_get_coverage(req):
    """Return per-league column-coverage stats for the given season.
    Read-only, no scraping. Used by the admin dashboard's always-visible
    Coverage Overview panel - admins check this before deciding which
    leagues to re-run."""
    season = req.get('season') or '2025-2026'
    legacy_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'legacy_scrappers')
    if legacy_dir not in sys.path:
        sys.path.insert(0, legacy_dir)
    scout_dir = os.path.dirname(os.path.abspath(__file__))
    if scout_dir not in sys.path:
        sys.path.insert(0, scout_dir)
    import sync_view

    leagues = []
    for league_key in LEAGUE_MAP.keys():

```

```

    try:
        cov = sync_view.compute_coverage_only(season, league_key)
        leagues.append({
            'league_key': league_key,
            'coverage': cov,
        })
    except Exception as e:
        print(f"[get_coverage] {league_key} failed: {e}", file=sys.stderr, flush=True)
        leagues.append({
            'league_key': league_key,
            'coverage': None,
            'error': str(e),
        })

return {'season': season, 'leagues': leagues}

```

`cmd\_run\_leagues` (def) - Run the scrape + sync for a subset of leagues, chosen by the admin via checkboxes in the UI. Same job-state machinery as cmd_run_update so progress polling + cancel work unchanged.

```

def cmd_run_leagues(req):
    """Run the scrape + sync for a subset of leagues, chosen by the
    admin via checkboxes in the UI. Same job-state machinery as
    cmd_run_update so progress polling + cancel work unchanged.

    Designed for the workflow: a full update partially succeeds, the
    UI shows which leagues are at <100% column coverage, the admin
    ticks only those and re-runs - no need to re-scrape leagues
    that are already complete."""
    season = req.get('season')
    league_keys = req.get('league_keys') or []
    if not season or not isinstance(league_keys, list) or not league_keys:
        return {'error': 'season + non-empty league_keys list required'}
    unknown = [k for k in league_keys if k not in LEAGUE_MAP]
    if unknown:
        return {'error': f'unknown leagues: {"", ".join(unknown)}'}

    before = _snapshot_season_counts(season)
    job_id = _new_job(season)
    with _update_jobs_lock:
        _update_jobs[job_id]['scope'] = f'leagues:{"", ".join(league_keys)}'
        _update_jobs[job_id]['before'] = before

def _worker():
    try:
        legacy_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'legacy_scrapers')
        if legacy_dir not in sys.path:
            sys.path.insert(0, legacy_dir)
        scout_dir = os.path.dirname(os.path.abspath(__file__))
        if scout_dir not in sys.path:
            sys.path.insert(0, scout_dir)

        from run_update import run_one_league

        _record_step(
            job_id, 'Initialising', 'completed',
            f'Running {len(league_keys)} league(s) for {season}: {"", ".join(league_keys)}',
        )

        def _progress(step, status, detail):
            _record_step(job_id, step, status, detail)

        stats = {'fbref': {}, 'understat': {}, 'capology': {},

```

```

        'transfermarkt': {}, 'supplementary': {}, 'sync': {}
errors: list = []
for lk in league_keys:
    if _is_cancelled(job_id):
        _record_step(job_id, 'Cancelled', 'completed', f'Stopped before {lk}')
        break
    run_one_league(
        season,
        int(season.split('-')[0]),
        lk,
        _progress,
        stats,
        errors,
    )
    # 60s cooldown - matches run_update.py. Wayback throttles
    # back-to-back league batches; the gap lets its per-source
    # rate budget reset.
    time.sleep(60)

sync_per_league = stats.get('sync', {})
rollup = {'inserted': 0, 'updated': 0, 'unchanged': 0, 'deleted': 0}
sample_diffs_all = []
for lg_label, r in sync_per_league.items():
    for k in rollup:
        rollup[k] += r.get(k, 0)
    for d in r.get('sample_diffs', [])[:20]:
        sample_diffs_all.append({'*d', 'league': lg_label})

final_status = 'cancelled' if _is_cancelled(job_id) else 'completed'
_finish_job(
    job_id,
    final_status,
    result={
        'season': season,
        'league_keys': league_keys,
        'scrape': {'season': season, 'stats': stats, 'errors': errors},
        'sync_totals': rollup,
        'sample_diffs': sample_diffs_all,
        'sync_per_league': sync_per_league,
        'verification_after': _snapshot_season_counts(season),
    },
    errors=errors,
)
except Exception as e:
    print(f'[run_leagues worker] {job_id} failed: {e}', file=sys.stderr, flush=True)
    _record_step(job_id, 'Fatal', 'error', str(e))
    _finish_job(job_id, 'error', errors=[str(e)])

threading.Thread(target=_worker, daemon=True).start()
print(
    f'[run_leagues] dispatched job={job_id} season={season} '
    f'leagues={league_keys}',
    file=sys.stderr, flush=True,
)
return {
    'job_id': job_id, 'season': season, 'league_keys': league_keys,
    'status': 'running', 'before': before,
}

```

`cmd\_cancel\_update` (def) - Mark a running job as cancelled. The worker thread checks the flag between leagues / between sources and exits cleanly.

```
def cmd_cancel_update(req):
```

```

"""Mark a running job as cancelled. The worker thread checks the
flag between leagues / between sources and exits cleanly. In-flight
HTTP calls (FBref / Understat etc.) keep running to completion in
their own daemon thread - Python can't kill them - but the
pipeline won't start the next source/league. Status becomes
'cancelled'."""
job_id = req.get('job_id')
if not job_id:
    return {'error': 'job_id required'}
with _update_jobs_lock:
    job = _update_jobs.get(job_id)
    if not job:
        return {'error': 'job not found'}
    if job['status'] != 'running':
        return {
            'job_id': job_id,
            'status': job['status'],
            'message': f'Job is already {job["status"]}; nothing to cancel.',
        }
    job['cancelled'] = True
print(f'[cancel_update] flagged job={job_id} for cancellation', file=sys.stderr, flush=True)
return {'job_id': job_id, 'status': 'cancelling',
        'message': 'Cancellation requested. The worker will stop at the next checkpoint.'}

```

Main Loop

``cmd_export_main_csv`` (def) - Dump the main Data Scout table
(league_season_team_player_data) to a CSV in the repository (data-scout/).
Admin/pipeline-gated.

```

def cmd_export_main_csv(req):
    """Dump the main Data Scout table (league_season_team_player_data) to a CSV
    in the repository (data-scout/). Admin/pipeline-gated. Uses Postgres COPY so
    it streams straight to disk without loading the whole table into memory."""
    out_path = os.path.join(
        os.path.dirname(os.path.abspath(__file__)),
        'league_season_team_player_data.csv',
    )
    with get_connection() as conn:
        with conn.cursor() as cur:
            with open(out_path, 'w', encoding='utf-8', newline='') as f:
                cur.copy_expert(
                    "COPY (SELECT * FROM league_season_team_player_data) TO STDOUT WITH CSV HEADER",
                    f,
                )
            cur.execute("SELECT COUNT(*) FROM league_season_team_player_data")
            rows = cur.fetchone()[0]
    size = os.path.getsize(out_path)
    return {
        'success': True,
        'rows': rows,
        'bytes': size,
        'path': 'data-scout/league_season_team_player_data.csv',
    }

```

``COMMANDS`` (module constant)

```

COMMANDS = {
    'health': lambda req: {'status': 'ok'},
    'export_main_csv': cmd_export_main_csv,
    'get_leagues': cmd_get_leagues,
}

```

```

'get_teams': cmd_get_teams,
'get_players': cmd_get_players,
'get_player_profile': cmd_get_player_profile,
'get_position_benchmark': cmd_get_position_benchmark,
'get_playing_style': cmd_get_playing_style,
'get_hidden_gems': cmd_get_hidden_gems,
'get_gem_trajectory': cmd_get_gem_trajectory,
'backtest_gems': cmd_backtest_gems,
'train_mv_models': cmd_train_mv_models,
'get_mv_model_info': cmd_get_mv_model_info,
'get_similar_players': cmd_get_similar_players,
'compare_players': cmd_compare_players,
'get_career_history': cmd_get_career_history,
'get_contract_opportunities': cmd_get_contract_opportunities,
'get_wage_benchmark': cmd_get_wage_benchmark,
'get_market_value': cmd_get_market_value,
'get_moneyball_score': cmd_get_moneyball_score,
'get_squad_optimizer': cmd_get_squad_optimizer,
'check_update': cmd_check_update,
'run_update': cmd_run_update,
'get_update_progress': cmd_get_update_progress,
'cancel_update': cmd_cancel_update,
'retry_league': cmd_retry_league,
'run_leagues': cmd_run_leagues,
'get_coverage': cmd_get_coverage,
}

```

`main` (def)

```

def main():
    # Quick DB connectivity check
    try:
        with get_connection() as conn:
            with conn.cursor() as cur:
                cur.execute("SELECT 1")
        print(" DB connection OK", file=sys.stderr, flush=True)
    except Exception as e:
        print(f" DB connection failed: {e}", file=sys.stderr, flush=True)

    print("scout_server ready", file=sys.stderr, flush=True)
    print(json.dumps({"status": "ready"}), flush=True)

    for line in sys.stdin:
        line = line.strip()
        if not line:
            continue
        try:
            req = json.loads(line)
            cmd = req.get('command', '')
            handler = COMMANDS.get(cmd)
            if handler:
                result = handler(req)
            else:
                result = {'error': f'Unknown command: {cmd}'}
        except Exception as e:
            result = {'error': str(e)}

        # Ensure JSON serializable
        try:
            output = json.dumps(result, default=str)
        except Exception:
            output = json.dumps({'error': 'Serialization error'})

```

```
sys.stdout.write(output + "\n")
sys.stdout.flush()
```

```
if __name__ == "__main__":
    main()
```

Glossary

- **Composite index** - the single 0-100 score for a player (40% z-aggregate + 30% style + 30% league power).
- **Percentile rank** - where a value sits in the pool, 0-100. Outlier-proof, unlike min-max scaling.
- **z-score** - how many standard deviations above/below the pool mean a value is.
- **AHP** - Analytic Hierarchy Process: turns per-position importance priorities into a consistent set of criterion weights.
- **TOPSIS** - ranks options by closeness to an ideal profile; closeness in [0, 1].
- **Consistency ratio (CR)** - AHP's self-check that the priorities aren't contradictory (< 0.1 = fine).
- **Moneyball score** - performance + value-for-money + contract leverage, blended 0.5 / 0.3 / 0.2.
- **DataFrame** - a pandas table; rows are players, columns are metrics.
- **Endpoint** - a URL the browser can call, like /api/data-scout.
- **COMMANDS** - the engine's dictionary mapping a command name to the function that answers it.